תכנות מערכות
מני אדלר

---

0. קווים לדמותו של הקורס

תכנות מערכות

תוכן: נראה בהמשך ההרצאה, מבט רחב על מגוון נושאים הקשורים לעיצוב של מערכת תוכנה.

חשיבות התרגול.

שפות תכנות: Java, C++, Python, SQL

<u>חובות הקורס</u>:

70% מבחן
30% תרגילי תכנות

1. מבוא – סביבת זמן ריצה

<u>מוטיבציה</u>: בניית מערכת לחיפוש והורדה של סרטים

Get movie search properties
Search for the movie in the web
Display results, get user selection
Download
Play (read, decode, play)

מימוש:

- מימוש GUI (לא מעניין, קיימות ספריות, ניתן ללמוד זאת לבד)
- <u>חיפוש ברשת</u>: חיפוש (מבנה נתונים, אלגוריתמים), ברשת (**תקשורת בין תהליכים**)
- הורדת הסרט: שאלות אחסון, ב**זיכרון**, בקבצים, ב**בסיסי נתונים**.
- הקרנת הסרט: **מקביליות**, הרצה במקביל של מספר משימות (download, read, encode, play) תוך קביעת האילוצים ביניהם.

 נושאי הקורס: ניהול זיכרון (C++), מקביליות (ספריות חדשות ב Java), תקשורת (שילוב C++ ו Java), בסיסי נתונים (Python, SQL)

1.1 מושגי יסוד

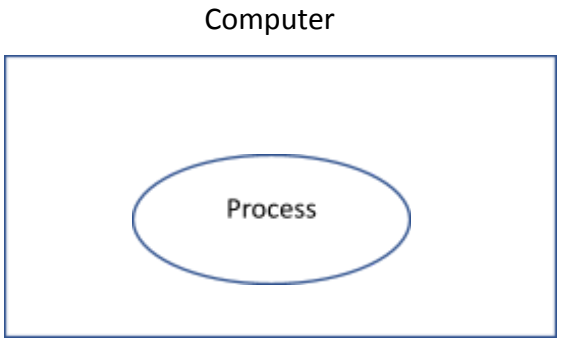
מחשב (Computer)
יחידה לוגית היכולה לבצע באופן עצמאי משימה
(יכול לכלול כמה מחשבים פיזיים, כל עוד הם מנוהלים על ידי מערכת הפעלה אחת)

תוכנית (Program)
הגדרה פורמאלית, בשפה כלשהי, של משימה

תהליך (Process)
תוכנית בהרצה על ידי מחשב

דוגמאות: בישול, תוכניות Java

מבנה כללי:



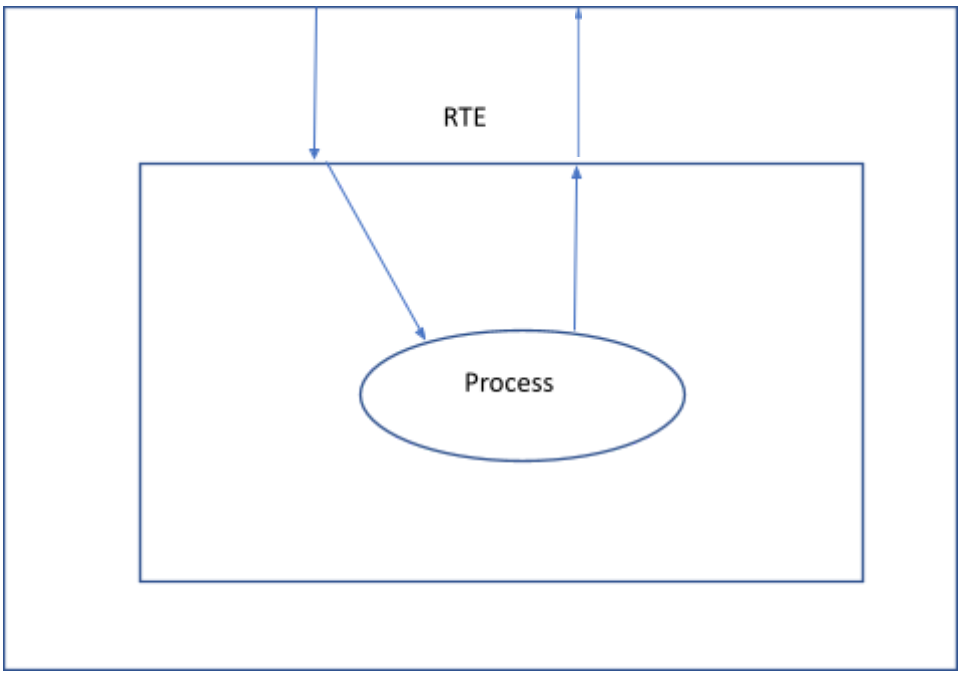
1.2 סביבת זמן ריצה (Run Time Environment)

הציור הנ"ל, בו התהליך רץ 'ישירות' על המחשב, מטריד בכמה מובנים:

- התוכנית צריכה להיות כתובה בשפה שהמחשב מבין. יותר מדי ספציפי (מעבד, דיסק, כרטיס רשת). חסרה הפשטה.
- ממשקים לא נוחים
- פונקציוליות מוגבלת של החומרה (הרצת מספר תוכניות יותר ממספר המעבדים, הרצת תוכנית הדורשת יותר זיכרון ממה שקיים בחומרה)

 נוסיף סביבת זמן ריצה שתתווך בין התוכנית לתשתית הקיימת (במקרה זה חומרת המחשב):
- סביבת הריצה משמרת את תכונת התשתית הקיימת, כלומר ניתן להריץ בה תוכניות
- סביבת הריצה מספקת הפשטה, חוסר תלות במשאבים של התשתית הקיימת
- סביבת הריצה מספקת ממשקים נוחים יותר בגישה לתשתית הקיימת
- סביבת הריצה מספקת שירותים נוספים מאלה הניתנים על ידי התשתית הקיימת

Computer



1.3 דוגמאות לסביבות זמן ריצה

1.3.1 מערכת ההפעלה

מערכת ההפעלה היא סביבת זמן ריצה המתווכת בין התהליך לחומרת המחשב

1. מערכת ההפעלה תומכת בהרצת תוכניות

כאשר מריצים תוכנית תחת מערכת ההפעלה, מתרחשים השלבים הבאים:

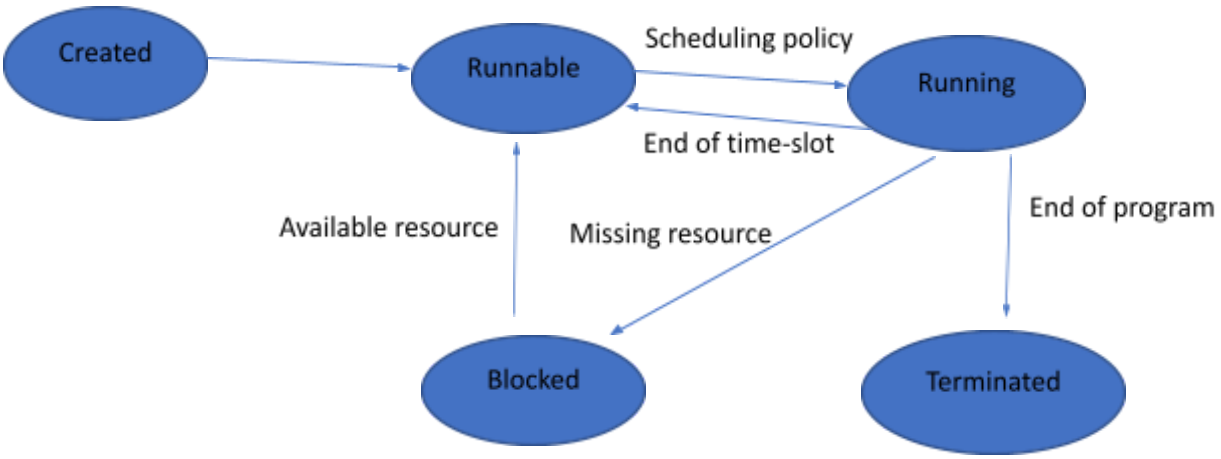
- Created, בניית התשתית להרצת התכנית (מבחינת ההפעלה)
  - המערכת בונה אובייקט לאחסון נתוני התהליך (מספר זהות id, באיזו שורת קוד אנו נמצאים כרגעpc , באיזה שלב נמצא התהליך (Created, Runnable, Running, Blocked, Terminated), נתונים סטטיסטיים על הזמן שהוא רץ עד כה,...)
  - הקצאת זיכרון לתהליך
  - טעינת קובץ התכנית לזיכרון
  - קביעת שורת הקוד הבא לביצוע (PC – Program Counter) כשורה הראשונה במתודת ה main
  - מעבר למצב Runnable

- Runnable, התהליך יכול לרוץ אך הוא ממתין כרגע
  - כאשר יגיע תור התהליך לרוץ (על פי מדיניות התזמון של מערכת ההפעלה) הוא יועבר למצב Running

- Running, התהליך רץ בפועל
  - המערכת מעבירה את השורה הבאה לביצוע (הכתובת שלה נמצאת בשדה PC) במעבד
  - ומגדירה את השורה הבאה לביצוע כשורה הבאה בקוד (הגדלת ה PC)
  - וכן הלאה, עד אשר קורה אחד משלושה דברים:
    - התוכנית הסתיימה, הגענו לשורה האחרונה מעבר למצב Terminated
    - נגמרה הקצאת זמן הריצה לשלב זה מעבר למצב Runnable
    - קיים אילוץ שלא מאפשר כרגע את הרצת השורה הבאה (לדוגמא, גישה ל i/o, כמו קריאה מקובץ או מהרשת) מעבר למצב Blocked

- Blocked, התהליך לא יכול לרוץ כרגע בשל אילוץ
  - התהליך לא נמצא בתור של התהליכים המיועדים לקבל זמן ריצה, עד אשר האילוץ יתקיים (המידע הגיע מהרשת, השורה הבאה בקובץ הגיעה לזיכרון)
  - עם הסרת האילוץ, התהליך עובר למצב Runnable

- Terminated, ניקוי משאבי התהליך ושחרורו

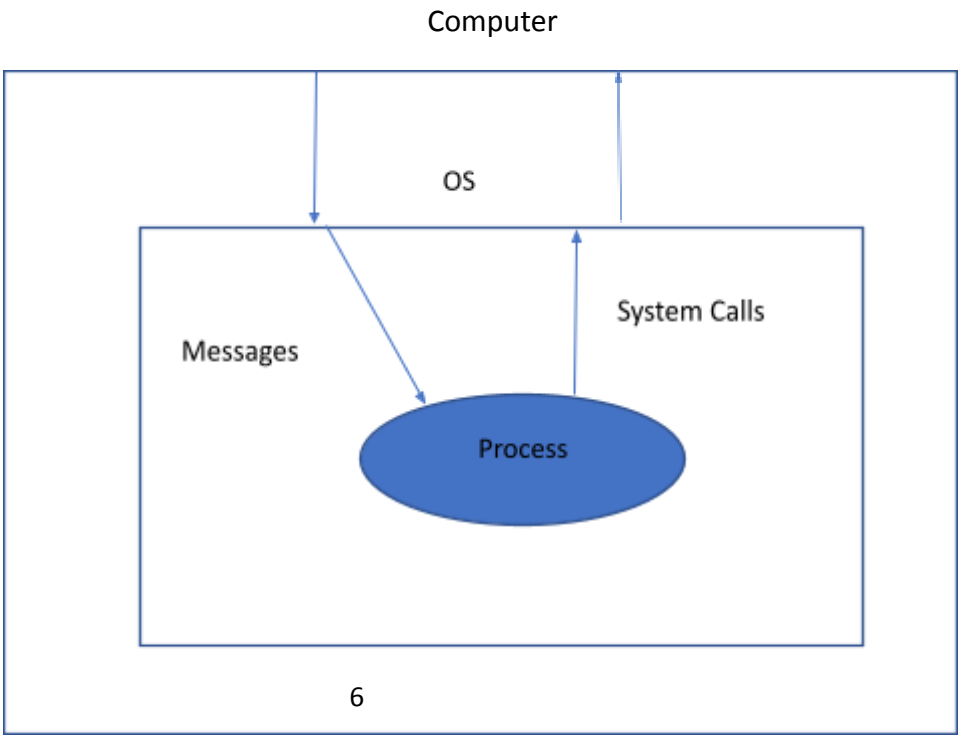


2. מערכת ההפעלה מספקת הפשטה, או חוסר תלות, במשאבי החומרה השונים

מערכת ההפעלה 'מכירה' את כל סוגי החומרה (אם יש סוג חומרה שהיא לא מכירה, צריך לספק את הדרייבר שלו כדי שהיא תכיר אותו), ויודעת לתרגם את הבקשה בתוכנית לשפה של רכיב החומרה המתאים.

3. מערכת ההפעלה מספקת ממשקים נוחים לחומרה

- התהליך יכול לפנות לרכיבי החומרה בעזרת אוסף פונקציות שמערכת ההפעלה מספקת, המכונות system calls
- אירועים שונים ברכיבי החומרה נשלחים כהודעות על ידי מערכת ההפעלה לתהליכים

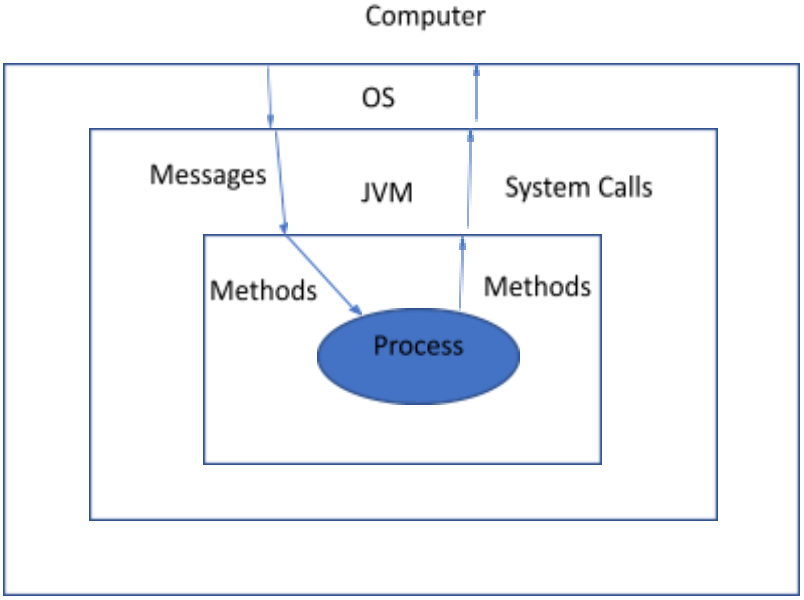


4. מערכת ההפעלה מספקת שירותים נוספים מעבר ליכולות של החומרה
   - הרצת מספר תהליכים גבוה יותר ממספר המעבדים (על פי הפרוטוקול שתואר למעלה, בו חלק מהתהליכים ממתינים לתורם)
   - הרצת תהליך הדורש יותר זיכרון מהזיכרון הפיסי הקיים בחומרה (מנגנון הזיכרון הוירטואלי – בהמשך)
   - הרשאות קבצים (קריאה/כתיבה, בן אדם מסוים, קבוצה...)

1.3.2 JVM (Java Virtual Machine)

כפי שראינו, מערכת ההפעלה מספקת הפשטה של החומרה, כך שהתהליך אינו תלוי במימוש החומרה, וכן ממשקים נוחים ושירותים נוספים.
בשנות התשעים רמת הפשטה ושירותים אלו אינם כבר מספקים. בשנים אלו התפתחה הרשת, ובפרט המושג 'הרצת תוכנית' לא התייחד דווקא להרצה של תוכנית במחשב מסוים, אלא גם להעברת קטעי קוד ממחשב למחשב והרצתם שם. כמו לדוגמא, העברת קוד משרת לריצה במחשב של כל לקוח המבקר באתר. כאשר לכל לקוח עשויה להיות מערכת הפעלה שונה נדרש חוסר תלות של הקוד במערכת ההפעלה.
הממשקים בשנות ה-90 הופכים להיות מונחי-עצמים. כך שפתאום 'הממשקים הנוחים' של מערכת ההפעלה כבר אינם כל כך נוחים: הקריאות למערכת אינן מתודות של אובייקט; כדי להגיב לאירוע בחומרה, צריך להאזין להודעות מערכת ההפעלה.
גם השירותים הנוספים הניתנים על ידי מערכת ההפעלה אינם מספקים בשנות ה-90. לדוגמא, רמת ההרשאות אינה מספקת, נדרשות הרשאות מפורטות יותר על האפשרויות השונות להריץ מתודות.

 נגדיר סביבת ריצה נוספת, אשר תתווך בין התהליך למערכת ההפעלה:



נראה כיצד מקיים ה JVM את תכונות סביבת הריצה:

1. הרצת תוכניות

ה JVM מאפשר להריץ תוכניות class, הכתובות בשפה המכונה 'byte code', על מערכת הפעלה נתונה. שפה זו אינה תלוית מערכת הפעלה. שפה זו קרובה מאוד לשפה של המעבד (אסמבלי), למעט הגישות למערכת ההפעלה שלא מתורגמות עדיין.
נמחיש זאת:
בהינתן תוכנית Java, A.java, הקמפול של התוכנית יוצר קובץ בשפת ה byte code:

```
>javac A.java
A.class
```

לא ניתן להריץ את קובץ ה class במערכת ההפעלה, כי היא לא כתובה בשפה של המעבד. כדי להריץ קובץ class יש לפנות לסביבת הריצה שיודעת להריץ קבצים כאלו, כלומר ל JVM:

```
>java A
```

פעולה זו הינה למעשה הרצה של תוכנית ה JVM כתהליך במערכת ההפעלה. תוכנית ה JVM מצפה לקבל כפרמטר ראשון את שם קובץ ה class שהוא צריך להריץ.
פונקציית ה main של ה JVM מבצעת את הפעולות הבאות:

- קמפול סופי של המחלקה A, בהתאם למערכת ההפעלה הנוכחית, כך שקובץ ה class הכתוב ב byte code הופך לקובץ הכתוב בשפה של המעבד בהתאם למערכת ההפעלה. תהליך זה, של קמפול מחלקה בזמן ריצה, מתרחש כל פעם שנדרשת מחלקה חדשה בתוכנית (just in-time compilation)
- השמת שדה ה pc של תהליך ה JVM במערכת ההפעלה כך שיצביע לשורה הראשונה במתודת ה main של המחלקה A

2. הפשטה, חוסר תלות במערכת ההפעלה

ע"פ המנגנון שתואר לעיל: שני שלבים של קמפול קוד ה Java:
Offline: לשפת byte code, שאינה תלוית מערכת הפעלה (קובץ ה class). נעשה ע"י הקומפיילר של Java) javac) פעם אחת.
Online: לשפת המעבד בהתאם למערכת ההפעלה הנוכחית. נעשה כל פעם שהתוכנית רצה, ע"י ה java) JVM) בזמן ריצה

כדי לחדד זאת, נראה לשם השוואה מה קורה בתהליך הקמפול בשפה כמו C++:

נתונה מחלקה A.cpp עם פונקציית main. כדי להריץ אותה יש לקמפל:

```
<g++ A.cpp
A.o
```

הקובץ המתורגם A.o כתוב כבר בשפת האסמבלי של המעבד, ע"פ מערכת ההפעלה הקיימת. כלומר, הוא ספציפי למערכת זו (לינוקס), ולא ירוץ על מערכת הפעלה אחרת (כמו ווינדוז).

אם רוצים קובץ ריצה של התוכנית עבור Windows, יש לקמפל את הקוד במערכת הפעלה זו:

```
>cl A.cpp
A.exe
```

3. ממשקים נוחים

כל הקריאות למערכת והתגובות לאירועים הם בממשקי OO (כמו לדוגמא המחלקה FileInputStream שתומכת בגישה לקבצים)

4. שירותים נוספים

לדוגמא: הגדרה מורכבת של הרשאות לביצוע מתודות במחלקה מסוימת.

2. ניהול זיכרון

2.1 מבוא – נקודות מבט שונות על הזיכרון

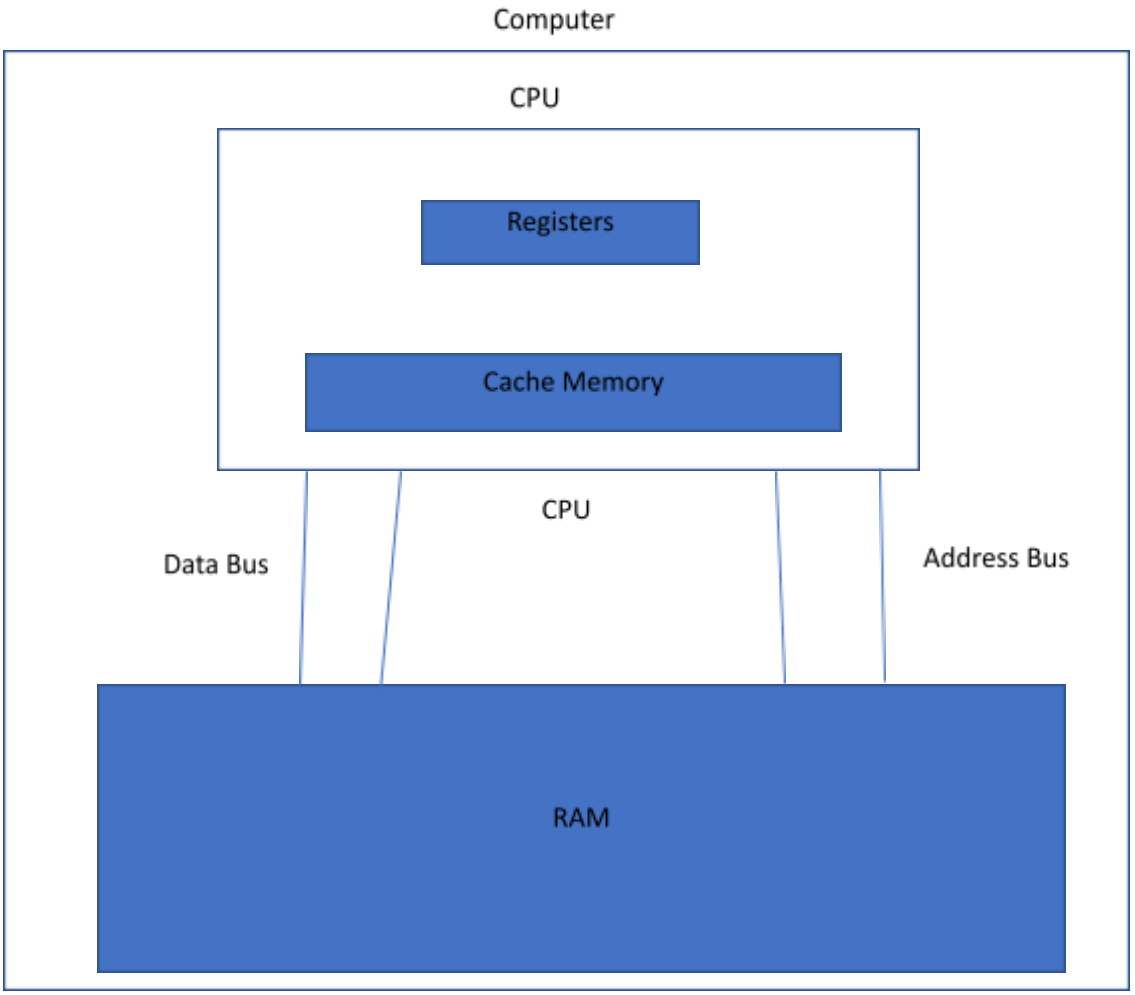
מסתבר, כי כמו כל דבר בחיים, המושג זיכרון ניתן להתפרש באופנים שונים, בהתאם לנקודות מבט שונות ('רשומון')

2.1.1 חומרה

מבחינת החומרה מושג הזיכרון קשור לטכנולוגיות שונות לאחסון מידע, ולפעולות עליהן.

<u>טכנולוגיות</u> / רכיבי זיכרון שונים במחשב:

- זיכרון על השבב של המעבד, חשמלי
  - רגיסטרים: מערך של 'תאים' לאחסון מידע. זמן הגישה אליהם הוא פולס אחד.
  - זיכרון 'מטמון' (cache memory), מרחב זיכרון (חשמלי) במעבד עם גישה עקיפה יותר (3 פולסים) אשר נועד לשמירה של תאי זיכרון 'פופולריים'
- RAM
  - מערך של תאי זיכרון (חשמלי), הנמצא ברכיב נפרד מהמעבד (סדר גודל של 10 פולסים זמן גישה)
- DISK
  - מחוץ ל'לוח האם', מגנטי, (סדר גודל של 10,000 פולסים), SSD (סמי-חשמלי, מהיר יותר). נשמר לאורך זמן
- TAPE
  - טכנולוגיה אופטית, גישה איטית, נשמר לאורך זמן



ניתן להבחין את הטכנולוגיות השונות ע"פ מספר פרמטרים:
- גודל הזיכרון
- מהירות הגישה
- מחיר
- האם המידע נשמר לאורך זמן, או שהוא נמחק כאשר סוגרים את החשמל

אבחנה: ככל שהגישה למידע מהירה יותר, יכולת האחסון קטנה ויקרה יותר.

רכיבי הזיכרון השונים משמים מטרות שונות, ומתאימים לצרכים שונים:
- רגיסטרים
  - אחסון המידע שהמעבד צריך לפעולה הנוכחית שהוא מבצע
  - אחסון מידע שהמעבד צריך כל הזמן, כמו כתובת השורה הבאה לביצוע בקוד (יש רגיסטר קבוע ל pc) או כתובת המחסנית של התהליך הנוכחי (יש רגיסטר קבוע ל sp)
- זיכרון המטמון
  - עותק של תאים 'פופולריים' מה RAM, שנקראו/נכתבו לאחרונה, לשם צמצום זמן הגישה אליהם.
- RAM
  - שמירת המידע הנדרש במהלך הרצת התהליך (בפרט, משתני התוכנית)
- DISK
  - שמירת מידע שהמשתמש צריך לאורך זמן
- Tape
  - גיבוי של מידע

מספר הביטים ב data bus (מספר ה'חוטים' בו, 'רוחב הפס'), מגדיר את כמות המידע שניתן לקרוא/לכתוב בפעולה אחת של המעבד מ/ל הזיכרון. נכנה גודל מידע זה כ'תא' בזיכרון, או 'מילה' בזיכרון. זהו גם הגודל של תא ברגיסטרים.
מספר הביטים ב address bus קובע כמה זיכרון נגיש בפועל למעבד. כלומר, ייתכן שרכיב ה RAM הוא גדול ביותר, אך לא ניתן לגשת לרוב הכתובות שלו, כי הבקשה לקרוא נעשית על יד מתן הכתובת של התא, וניתן לתאר כתובת של תא רק ע"י כמות הביטים ב address bus.

<u>פעולות</u>

לכל מעבד יש אוסף פעולות שהוא יודע לבצע, המכונה instruction set.
דוגמא לפעולה שכזו: ADD

```
ADD R1 R2 R3
```

בפעולה זו, המעבד לוקח את התוכן של הרגיסטר R1 ומחבר אותו לתוכן של הרגיסטר R2 וכותב את הסכום לרגיסטר R3.

בנוסף יש למעבד שתי פעולות בסיסיות בזיכרון: כתיבה ל RAM וקריאה מה RAM

```
READ  100 R1
WRITE R2 200
```

קריאת התוכן של תא מספר 100 ב RAM אל רגיסטר 1R
כתיבת התוכן של רגיסטר 2R לתא מספר 200 ב RAM

נתרגם קוד Java פשוט לפעולות אלו:

```
int i = 10;
int j = 20;
i = i + j;
```

נניח כי ידוע שכתובת הזיכרון (ב RAM) של i היא 3000, וכתובת הזיכרון (ב RAM) של j היא 4000. התרגום קוד ה Java לפעולת של המעבד (אסמבלי) יהיה:

```
WRITE 3000 10
WRITE 4000 20
READ 3000 R1
READ 4000 R2
ADD R1 R2 R3
WRITE 3000 R3
```

2.1.2 מערכת ההפעלה

מבחינת מערכת ההפעלה, העיסוק בזיכרון שונה בתכלית:

- הקצאת זיכרון לתהליך: כיצד מוצאים מקום פנוי של תאי זיכרון ב RAM, מה קורה אם אין מקום פנוי, האם תאי הזיכרון עבור התהליך צריכים להיות <u>רצופים פיזית</u>, כיצד ניתן להרחיב תוך כדי ריצה את מרחב הזיכרון של התהליך, וכו'.
- כיצד מספקים זיכרון לשני תהליכים, כאשר אין מקום לשניהם (context switch)
- כיצד מספקים לתהליך יותר זיכרון ממה שקיים ב RAM (זיכרון וירטואלי)

2.1.3 שפת התכנות

מבחינת שפת התכנות קיים זיכרון שסופק ע"י מערכת ההפעלה. השאלות הן אחרות:

- כיצד לארגן את הזיכרון שמערכת ההפעלה סיפקה, אילו מבני נתונים תומכים בדפוס העבודה בזיכרון של התוכניות (Stack, Heap)
- פונקציות: כיצד ממומשת קריאה לפונקציה וחזרה ממנה מבחינת הזיכרון.
- משתנים: מה הם מסמלים, כיצד הם ממומשים בזיכרון.
- אובייקטים: כיצד מיוצגים אובייקטים בזיכרון, כיצד הם מאותחלים, נמחקים, מועברים, מועתקים, מושמים.
- ירושה ופולימורפיזם: כיצד ממומשים בזיכרון הקונספטים של ירושה ומימוש ממשקים.

2.2 מודלים של זיכרון: Stack, Heap

ראינו כי מערכת ההפעלה מקצה אזור בזיכרון עבור כל תהליך. כלומר מערך של תאי זיכרון, שניתן לגשת אליהם לקריאה או לכתיבה על ידי ציון האינדקס/הכתובת שלהם (מבחינת התהליך הזיכרון רציף, תא אחרי תא, אך לא בהכרח מבחינה פיזית)
מסתבר, כי שפות התכנות מארגנות מרחב זיכרון זה, תחת מבני נתונים המתאימים לדפוסים שונים של שימוש בזיכרון במהלך הרצת תוכנית.

<u>דוגמא</u>: דפוסי השימוש באולם הקולנוע בסינמטק במהלך היום

 שני דפוסים של שימוש בזיכרון

- קריאה לפונקציה וביצועה ו/או הגדרת משתנה שגודלו ידוע בזמן קומפילציה
שימוש סטטי בזיכרון: הזיכרון המוקצה ידוע מראש ('זמן קומפילציה'), השימוש בו מוגבל מראש, ההקצאה והשחרור מתבצעים באופן אוטומטי (ע"י המערכת), ההקצאה יעילה (למקום האחרון שפנוי), והשחרור ודאי (רק בתום השימוש)
יתרון: יעילות, חיסרון: גמישות
- הגדרת מידע בזיכרון שגודלו אינו ידוע בזמן קומפילציה (אובייקט המממש באופן כל שהוא ממשק, מערך בגודל לא ידוע), או שזמן השימוש בו אינו ידוע מראש.
שימוש דינאמי בזיכרון: רק בזמן ריצה ידוע כמה זיכרון צריך, ההקצאה והשחרור נעשים ביוזמת המתכנתים, ההקצאה דורשת חיפוש בזמן ריצה אחר מקומות פנויים, שחרור הזיכרון תלוי ברצון המשתמש (עשוי לגרום ל'חורים בזיכרון')
יתרון: גמישות, חיסרון: יעילות

 הזיכרון הניתן ע"י מערכת ההפעלה מאורגן תחת שני מבני נתונים: Stack, Heap
ה Stack תומך בדפוס השימוש הסטטי, והיעיל
ה Heap תומך בדפוס השימוש הדינאמי, והגמיש

Heap

מבנה נתונים המוגדר ע"פ הפעולות הבאות:

new – הקצאה של סדרת תאים רצופים בזיכרון (מספר התאים ניתן כפרמטר)
כלומר, לכל תא בזיכרון של ה Heap יש תכונה בוליאנית של מוקצה/לא-מוקצה. פעולת new תסמן סדרת תאים כ'מוקצה'

delete – שחרור של סדרת תאים בזיכרון. כלומר הפיכת שדה ההקצאה שלהם ל'לא מוקצה'.

read – קריאת תא מוקצה מהזיכרון (אם התא לקריאה, ע"פ כתובתו, אינו מוקצה – שגיאת זמן ריצה)

write – כתיבת לתא מוקצה בזיכרון (אם התא לכתיבה, ע"פ כתובתו, אינו מוקצה – שגיאת זמן ריצה)

Stack

createAF – התחלת אזור חדש במחסנית (אזור המכונה Activation Frame)
כלומר, בניגוד למחסנית שנלמדה בקורס מבנה נתונים, המחסנית 'שלנו' אינה כוללת רק תאים, אלא גם מארגנת תאים אלו תחת 'אזורים שונים'
[אם היינו צריכים לממש זאת, היינו מוסיפים למחלקה של המחסנית בקורס מבנה נתונים, רשימה של מספרים המציינים את כתובות ההתחלה של האזורים השונים. פעולת createAF תוסיף במימוש זה, את הכתובת של ראש המחסנית כרגע לרשימה זו]

removeAF – חזרה לאזור הקודם במחסנית
[במימוש שהוצע קודם, נסיר פשוט את הכתובת האחרונה מרשימת הכתובות המציינות את תחילת האזורים השונים במחסנית]

push – הוספת תא אחד בראש המחסנית, כלומר לאזור הנוכחי, כלומר ל Activation Frame.

pop – הסרת תא אחד מראש המחסנית, כלומר מהאזור הנוכחי, כלומר מה Activation Frame

read – קריאה מאחד התאים באזור הנוכחי, ב Activation Frame [בניגוד למחסנית ממבנה נתונים, בה ניתן לקרוא רק את התא האחרון]

write – כתיבה לאחד התאים באזור הנוכחי, ב Activation Frame [בניגוד למחסנית ממבנה נתונים, בה ניתן לכתוב רק לתא האחרון]

נתאר כעת את השימוש ב Stack וב Heap במהלך הרצת קוד של תוכנית, ע"פ שני דפוסי השימוש שהוזכרו (כך יתרגם הקומפיילר את הקוד הנתון לפעולות בזיכרון על ה Stack ועל ה Heap)

עיקרון כללי: בכל מהלך הרצת התוכנית, הגישה לזיכרון (בין במחסנית ובין ב Heap) אפשרית אך ורק דרך ה Activation Frame של המחסנית.
נבחן כעת, כיצד מתרגם הקומפיילר את הפעילות שלנו בזיכרון בקוד לפעולות על המחסנית וה Heap:

1. משתנים
   - הגדרת משתנה בקוד התכנית תתורגם תמיד לפעולות push ע"פ גודלו הידוע בזמן קומפילציה.
   - בתום כל בלוק של קוד (סוף {} בג'אווה), ביצוע pop לכל המשתנים שהוגדרו בבלוק זה.
   - ניתן לגשת לכל אחד מהמשתנים ב AF. גישה למשתנה, קריאה או כתיבה, תתורגם לפעולות read/write על התא של המשתנה במחסנית. <u>המיקום של כל משתנה ב Activation Frame ביחס לראש המחסנית ידוע בזמן קומפילציה</u> (לשם כך הקומפיילר מתחזק משתנה המציין היכן ראש המחסנית כרגע, ואף מאחסן אותו על אחד הרגיסטרים)

2. קריאה לפרוצדורה
   - פתיחת אזור חדש במחסנית (createAF)
   - [ביצוע push לכתובת השורה הבאה בקוד (כדי שהפרוצדורה תדע לאן לחזור בסוף), נכנה אותו 'return address' או בקיצור 'ra']
   - [במידה ומדובר במתודה, כלומר פרוצדורה של אובייקט, יבוצע push ל this, כלומר לכתובת של האובייקט עליו מבוצעת המתודה, כדי לאפשר למתודה לגשת לשדות שלו (שאינם ב AF)]
   - ביצוע push לכל הארגומנטים הנשלחים לפרוצדורה, וכתיבת הערכים שלהם במקומם
   - [מעבר לקוד הפרוצדורה ייעשה על ידי השמת כתובת קוד הפרוצדורה ל pc]

3. הדרת פונקציה
   - בתום הפרוצדורה הערך המוחזר ייכתב למקום זמני (נניח לרגיסטר מסוים שנקבע מראש), לשם העתקתו אח"כ למקום המתאים ב AF הקודם.
   - בתום הפרוצדורה (חוץ מה pop הנלווה לכל סוף בלוק, כבסעיף א, כולל הפרמטרים שנשלחו אליה), חזרה ל AF הקודם (removeAF)

- [חזרה לקוד של מי שקרא לפרוצדורה, ע"י השמת ה pc לכתובת הקוד שקרא לפונקציה (אשר נשלח כפרמטר)]

4. הגדרת מידע דינאמי
   - בכל מצב שבו נדרש להגדיר מידע שאופיו ידוע רק בזמן ריצה – מבחינת הגודל שלו, ומבחינת זמן השימוש בו – ההקצאה והשחרור שלו יתבצעו ע"י המתכנתים ע"י קריאה ל new על ה Heap בזמן היצירה, ול delete על ה Heap כאשר הוא אינו נדרש יותר.

<u>דוגמא</u>:

Stack

Heap

1000	2000	3000
3		
4		

```
class Pair {
   int first, second;
   Pair(int first, int second)
          { this.first = first; this.second = second; }
     int sum () {
       int ret = this.first + this.second; // code address 7000
       return ret; }
}

class Test {
    static int sum(int i, int j) {
        int ret = i+j;   // code address 6000
        return ret;
    }
    public static void main(String[] args) {
         int k = sum(1,2); ;   // code address 5000
         Pair p = new Pair(3,4);
         k = p.sum();
    }
}
```

0 stack.push // definition of k
8 stack.createAF // calling sum
16 stack.push
24 write [sp], 72 // the address of the code after the procedure
32 stack.push // a place for the first argument i
40 stack.write [sp], 1 // the value of the first argument i
48 stack.push // a place for the second argument j
56 stack.write [sp], 2 // the value of the second argument j
64 pc = 6000 // move to the code of sum(int i, int j)
72 stack.write [sp], R17
80 stack.push // for variable p
88 heap.new 2 // allocation of two cells in the heap for Pair obj
[code for writing to address in the heap of the pair to p/[sp], and a code the initialization of the pair in the heap to 3,4]
[code for calling sum(), requires the sending of 'this' as an additional parameter]

// code of Test.sum procedure
6000 stack.push // a place for the ret local variable
// code for calculating i+j
6008 stack.read [sp]+16, R1 // read parameter I to register R1
6016 stack.read [sp]+8, R2 // read parameter j to register R2
6024 add R1,R2,R3 // some registers R1 and R2 into R3
6032 stack.write R3, [sp] // write sum result (at R3) to local variable ret (at the top of the stack)
6040 stack.write [sp], R17 // save return value in a temporary place (R17)
6048 stack.write [sp+24], R16 // save return code address in a temporary place (R16)
6056 stack.pop // remove ret
6064 stack.pop // remove j
6072 stack.pop // remove i
6080 stack.pop // remove code-ret-address
6088 stack.removeAF // close AF of sum()
6096 pc = [R16] // return to the caller code address

<u>לסיכום</u>:

- ראינו בדוגמא את האופן שבו מְתֻרגמים דפוסי השימוש השונים הזיכרון בקוד לפעולות על המחסנית ועל ה Heap בהתאם.
- תיארנו גראפית תהליך זה
- עמדנו על הצורך לספק לפרוצדורה פרמטרים נלווים, כמו המקום בקוד שאליו יש לחזור בסוף (ה return address), כתובתו של האובייקט עליו מבוצעת המתודה ('this')
- כי ראינו כי לעתים נדרש לאחסן זמנית מידע (הערך החוזר, כתובת שורת הקוד לחזרה), דבר הדורש החלטה מוסכמת על מיקומו הזמני של המידע (רגיסטרים מסוימים וכדומה)
- יתרון השימוש במחסנית: יעילות ההקצאה (פעולת push פשוטה), והאוטומציה של השחרור (pop בסוף הבלוק(המבטיחה שלא ייוצרו 'חורים' בזיכרון (Memory Leak). חסרונה, גמישות (מתחייבים מראש לגודל מסוים של מידע, לפרק הזמן של הבלוק בלבד)
- יתרון השימוש ב Heap: גמישות, אין צורך להתחייב מראש על גודל המידע או על זמן השימוש בו. חסרונה: חיפוש אחר מקום פנוי בהקצאה בזמן הריצה (new), וחשש כבד לחורים בזיכרון (כאשר לא מבוצע delete)

2.3 משתנים

שפות התכנות מאפשרות לגשת לזיכרון, לקריאה או לכתיבה, בעזרת הפשטה נוחה של משתנים.

```
int i;
 …
 i = 4; // write i
 i++;
 …
 j = i;  // read i, write j
```

בשפות תכנות שונות, יש למשתנים אף טיפוס, הקובע בין היתר את גודלו, ובמקרה של אובייקטים גם את מבנהו.
בסעיף זה נבחן מה בדיוק מסמל המשתנה, כיצד הוא מיוצג בזיכרון, כיצד הוא מועבר כפרמטר, וכיצד הוא חוזר מפרוצדורה כערך מוחזר.

2.3.1 הגדרת משתנים

כאשר מגדירים משתנה עבור מידע מסוים, יש להחליט שתי החלטות:

א. מה מייצג בדיוק המשתנה: האם הוא מייצג תוכן של תאים בזיכרון, או כתובת של תא בזיכרון המכיל מידע (או שהמידע מאוחסן בתאים החל ממנו)?

2. האם מוגדר משתנה חדש במחסנית (המייצג תוכן/כתובת) או שמדובר בשם נוסף, כינוי (reference), למשתנה קיים המייצג תוכן/כתובת זו.

ב Java, שאלות אלו הוכרעו כבר ע"י מעצבי השפה, כך שהמתכנתים פטורים מלהרהר בהן:

1. משתנים מטיפוסים פשוטים (int, long, boolean,…) מייצגים תוכן
   משתנים מטיפוס אובייקט (class) מייצגים כתובת
2. לא ניתן לתת כינוי למשתנה קיים (אין reference)

```
Pair p1 = new Pair(3,4);
Pair p2 = p1;
p1 = new Pair(8,0);
```

ב C++ המתכנתים יכולים לבחור כל שילוב של שתי האופציות:

Heap

100	200	300	400	500	600	700	800	900	1000
16	8								
	9								

Stack

Address	Data	Var
9860		
9868		
9876		
9884		
9892		
9900		
9908		
9916		
9924		
9932		

Code

```
 // simple type, content   [V java]
int i(5);
i++;

// object, content [X java]
Pair m(3,4);
m.sum();  //  return 7

// simple type, address [X java]
int* pi = new int(15);
[pi++; //XXX increase pi, i.e., the address represented by pi, not the value 6]
cout << i; // 6
cout << pi; // 100
// in order to access the content of an address, we use the '*' operator
cout << (*pi);  // 15
(*pi)++;
// we can find the address of a content with the operator '&'
cout << (&i); // 10000

// object, address [V java]
Pair* pm = new Pair(8,9);
pm.first(); // error, pm is not an Object
(*pm).first(); // 8. Syntactic sugar: pm->first();

// simple type, content, ref [X java]
int& j = i;
j++;
cout << (&j) // 10000

// object, content, ref [X java]
Pair& n = m;
cout << n.second(); // 4

// simple type, address, ref [X Java]
```

9940		
9948		
9956		
9944		
9952		
9960		
9968	200	pm, pn
9976	100	pi,pj
9984	3	m,n
9992	4	
10000	7	i,j

כיצד נבחר בפועל את סוג המשתנה הרצוי (כתובת/תוכן, כינוי)?

1. תוכן או כתובת

<u>הרעיון הכללי</u>: אם זה אפשרי, תמיד נעדיף להגדיר את המידע, או התוכן, על המחסנית, מבלי לערב את ה Heap. ראינו כבר כי ההקצאה יעילה הרבה יותר, והשחרור אוטומטי וודאי.
 נעדיף תמיד, אם אפשר, להגדיר משתנה המייצג תוכן. שהרי המשתנה נמצא תמיד על המחסנית, ואם הוא מייצג תוכן, הרי שהתוכן נמצא על המחסנית.

מתי זה אפשרי?
כאשר גודל המידע ידוע בזמן קומפילציה (לא מערך בגודל לא ידוע, לא ירושה לא ידועה של ממשק), וגם השימוש בו נדרש אך ורק עבור הבלוק הקיים, נגדיר את המשתנה שלו כמשתנה תוכן, אחרת כמשתנה כתובת / מצביע.

2. מתי נדרש להגדיר ref?

הרעיון הכללי: ה reference מאפשר לתמוך בשני קונספטים:

- הימנעות מההעתקה מיותרת
- אי חשיפת הכתובת של אובייקט ה-heap ל-AF שאינו אחראי על מחיקתו

תרחישים (יודגם בפירוט בהמשך):

- אובייקט ב heap המועבר כפרמטר לפרוצדורה
- שדה במחלקה המתייחס למידע של אובייקט אחר
- Getter במחלקה המחזיר אובייקט

2.3.2 העברת משתנים/פרמטרים לפרוצדורה

ראינו כי קריאה לפרוצדורה כרוכה בהעברת על המידע הנדרש לפרוצדורה, כלומר יש להעביר מה AF הנוכחי ל AF החדש של הפרוצדורה את הפרמטרים.

קיימות שתי דרכים להעברת הפרמטרים:

By Value: מה שמייצג המשתנה (תוכן/כתובת), משוכפל ומועתק ל AF החדש כמשתנים חדשים

By Reference: הפרמטרים לא עוברים ל AF החדש אלא נשארים במקומם. מה שיש ב AF החדש זו גישה למידע שנמצא במקום אחר, כלומר הפרצדורה מקבלת כינויים נוספים למשתנים הקיימים במקום אחר.

ב Java יש רק העברה By Value, ב C++ המתכנתים בוחרים כיצד להעביר כל פרמטר.

נדגים זאת:

Heap

100 [Pair]	200	300	400	500	600	700	800	900	1000
3									
4									

tack

Address	Data	Var
9860		
9868		
9876		
9884		
9892		
9900		
9908		
9916		
9924		
9932		
9940		
9948		
9956		
9944		

Code

```
int sum1(int i, int j) { return i + j; }
int sum2(Pair n) { return n.first() + n.second(); }
int sum3(int* pi, int* pj) { return (*pi) + (*pj); }
int sum4(Pair* pn) { return (*pn).first() + (*pn).second(); }

int sum5(int& i,int& j) { return i + j; }
int sum6(const Pair& n) { return n.first() + n.second(); }
int sum7(int*& pi, int*& pj) { return (*pi) + (*pj); }
int sum8(Pair*& pn) { return (*pn).first() + (*pn).second(); }

{
     int a(5);
     int b(6);
     Pair* pm = new Pair(3,4);

     cout << sum1(a,b);
     cout << sum2(*pm);
     cout << sum3(&a,&b);
     cout << sum4(pm);

     cout << sum5(a,b);
     cout << sum6(*pm);
     cout << sum7(&a,&b);
     cout << sum8(pm);

     delete pm;
}
```

9952		
9976		
		pn
9984	100	pm
9992	6	b
10000	5	a

ראינו כי ברירת המחדל היא העברה by value, כדי 'להעביר' פרמטר by reference יש לציין זאת ע"י & ליד הטיפוס (נשים לב כי זו רק בחירה של אופן ההעברה ולא שינוי הטיפוס, כל שאר הקוד נשאר כשהיה).

כיצד אם כן נבחר את אופן ההעברה של הפרמטרים השונים לפרוצדורות?

<u>הרעיון הכללי</u>: אנו רוצים למקסם את כל שלושת הנקודות הבאות:

- חסכון בהעתקה. עדיף להימנע משכפול אובייקטים או מערכים.
- אי חשיפה של הכתובת בזיכרון לפרוצדורה, שאינה אחראית למוחקו (היא רק משתמשת במידע)
- תמיכה בעדכון של שינויים שהפרוצדורה מבצעת על המידע המקורי (side-effect, בדרך כלל לא מומלץ) / מניעת הפרוצדורה מלבצע שינויים.

 האם הפרמטר מיועד לשינוי בפרוצדורה?

כן: העברה של הפרמטר by reference

לא: טיפוסים פשוטים – by value, אובייקטים – by const reference

2.3.3 החזרת ערך מפונקציה

ניתן להחזיר ערך מפונקציה בשתי דרכים:

By Value: הערך המוחזר (כתובת/תוכן) מועתק, משוכפל, מה Activation Frame של הפונקציה ל Activation Frame הקודם.

By reference: הערך המוחזר (כתובת/תוכן) נשאר במקומו. מה שחוזר זה רק 'כינוי', גישה אליו מה Activation Frame הקודם.

ב Java ניתן להחזיר רק by value. ב C++ המתכנתים בוחרים את אחת משתי האפשרויות.

גם כאן ברירת המחדל היא החזרה by value. ניתן לבחור החזרה by reference על ידי ציון & ליד טיפוס הערך המוחזר.

דוגמאות:

Heap

100 [Number]	200	300	400	500	600	700	800	900	1000
3									

Stack

Address	Data	Var
9860		
9868		
9876		
9884		
9892		
9900		
9908		
9916		
9924		
9932		
9940		
9948		
9956		
9944		

Code

```
int sum9(int i, int j) { int ret(i+j); return ret; }
Number sum10(int i, int j) { Number ret(i+j); return ret; } // VX
int& sum11(int i, int j) { int ret(i+j); return ret; }   // XXX
Number& sum 12(int i, int j) { Number ret(i+j); return ret; }  // XXX
Number* sum13(int i, int j) { Number* ret = new Number(i+j);
                                                    return ret; }
{
     int a = sum9(1,2);
     Number n = sum10(1,2);
     int& b = sum11(1,2);
     Number& m = sum12(1,2);
     Number* pn = sum13(1,2);

     ...
     delete pn;
}
```

9952		
9960		
9968		
9976		m
9984		b
9992	3	n
10000	3	a

כיצד נבחר את אופן ההחזרה?

גם כאן, ננסה להימנע משכפול אובייקטים, ומצד שני להימנע מהחזרת ref למקום שלא קיים כבר בזיכרון.

מקרה א: הפונקציה מחזירה מידע חדש שיוצר בה (רוב הפונקציות)

הערך המוחזר הוא טיפוס פשוט - נחזיר אותו by value

הערך המוחזר הוא אובייקט – לא נרצה להחזיר אותו by value כי זה לא יעיל. מצד שני, לא ניתן להחזיר אותו by reference כי הוא נמחק מהמחסנית בסוף הפונקציה. זהו מקרה שבו מיוצר מידע שאורך חייו חורג מגבולות הבלוק שבו הוא מוגדר נגדיר אותו ב Heap ונחזיר את הכתובת שלו (מי שקרא לפונקציה הוא זה שיהיה אחראי למוחקו, והוא גם היחיד שיש לו את הכתובת שלו. תבנית זו מכונה Factory, נעצב אותה טוב יותר בהמשך)

מקרה ב: הפונקציה מחזירה מידע שכבר קיים. הדוגמא המובהקת: getter של מחלקה.

טיפוסים פשוטים – by value

אובייקטים – by (const) reference

דוגמא: המחלקה Student מיד

[מערכים ב C++ – ראו בתרגול]

2.4 אובייקטים

שפות תכנות שונות תומכות בהגדרת מידע בעל מבנה מסוים, ואף באובייקט הכולל מתודות.
נבחן כיצד קונספט זה בא לידי ביטוי מבחינת הזיכרון:

- מה המבנה של אובייקט בזיכרון
- כיצד הוא מאותחל
- כיצד הוא משוכפל
- כיצד מבוצעת עליו השמה
- כיצד הוא נמחק מהזיכרון

2.4.1 מבנה אובייקט בזיכרון

כאשר אובייקט מוגדר (על המחסנית או על ה Heap) מוקצה עבורו זיכרון ע"פ הגודל שלו. כאשר המבנה הפנימי של התאים המוקצים, מוגדר מראש על פי שיטה קבועה (כך שהקומפיילר יכול לתרגם את הגישה לשדות השונים למקום המדויק שלהם בזיכרון ביחס לכתובת האובייקט).
המבנה (כרגע) הוא: השדות מסודרים בזה אחר זה (כל שדה מתחיל בתא חדש), על פי סדר הגדרתם במחלקה, כאשר השדה הראשון מתחיל בכתובת של האובייקט (הקטנה). כאשר המבנה הפנימי של כל שדה מוגדר רקורסיבית באותו אופן.

דוגמא: המחלקה Student

Heap

100 [Student]		200	300	400	500	600	700	800	900	1000
id										
name	chars									
	length									
bag										
books	lst									
	size									
univ										

Stack

Address	Data	Var	
9900			
9908			
9916			
9924			
9932			
9940			
9948			
9956			
9944		100	ps
9952		id	s
9960		chars	name
9968		length	
9976		bag	
9984		lst	books
9992		size	
10000		univ	

Code

```
class Student {

  private:
       const int id;
       string name;
       Bag* bag;
       vector<Book*> books;
       const University& univ;

     public:
         int getId() const
            { return id; }
         const string& getName() const
             { return name; }
         const Bag& getBag() const
             { return *bag; }
         const vector<Book*>& getBooks() const
             { return books; }
        const University& getUniv() const
             { return university; }
};

{
     Student s(----);
     Student* ps = new Student(----);
}
```

2.4.2 אתחול אובייקט בזיכרון

ציינו כבר, מספר פעמים, כי הגדרה של אובייקט בזיכרון (ב Stack או ב Heap), מתורגמת ע"י הקומפיילר לשתי פעולות מרכזיות:

- הקצאת זיכרון (ב Stack ע"י push בהתאם לגודל האובייקט; ב Heap ע"י new, כלומר חיפוש בזמן ריצה אחר תאים פנויים בהתאם לגודל האובייקט)
- הפעלת מנגנון אתחול, המבטיח שכל החלקים של האובייקט עוברים אתחול (באופן אוטומטי):
  - הפעלה אוטומטית של מנגנון האתחול על כל השדות באובייקט שהם אובייקטים בעצמם (לא טיפוסים פשוטים, לא כתובות של אובייקטים)
  - ביצוע מתודת הבנאי של האובייקט

מתודת הבנאי עבור המחלקה Student, ניסיון ראשון:

```
Student(int id, const string& name, Bag* bag, const University& univ)    {
    _id = id;
    _name = name;
    _bag = bag;
    _univ = univ;
    _books.push_back(new DigitalBook()
    _books.push_back(new HardCopyBook());
}
```

שני חסרונות:
- חוסר יעילות: השדות מסוג אובייקט עבור אתחול (ריק) + השמה. עדיף לבצע אתחול אחד עם ההשמה הנדרשת.
- עבור שדות שהם const, וכן שדות שהם reference, לא ניתן לבצע את ההשמה לאחר ההגדרה.

נקבע מראש כיצד לאתחל את השדות השונים:

```
Student(int id, const string& name, Bag* bag, const University& univ)
    : _id(id), _name(name), _bag(bag),  _univ(univ)
{
    _books.push_back(new DigitalBook());
    _books.push_back(new HardCopyBook());
}
```

100 [Student]		200 [String]	300 [Bag 7]	400 [string]	500 [Bag 5]	600	700	800	900	1000
id 8		ringo	.	jango	.					
name	chars 400	240 [DigitalBook]	.	440 [DigitalBook]	550 [Link]					
	Length 5	.	350 [Link]	.	440					
bag 500		.	240	.	560					
books	lst 550	260 [HardCopyBook]	360	460 [HardCopyBook]	560 [Link]					
	size 2	.	360 [Link]	.	460					
univ		.	260	.	0					
		.	0	.						

Stack

Address	Data	Var		
9940				
9948				
9956				
9944	100			ps
9952	7	id		s
9960	200	chars	name	
9968	5	length		
9976	300	bag		
9984	350	lst	books	
9992	2	size		

10000		univ	

```
class Student {

  private:
       int _id;
       string _name;
       Bag* _bag;
       vector<Book*> _books;
       const University& _univ;

     public:
       Student(int id, const string& name,
         Bag* bag, const University& univ)
           : _id(id), _name(name), _bag(bag),
             _univ(univ)
       {
             _books.push_back(new Infi());
             _books.push_back(new
HardCopyBook());
       }
};

{
     University univ(…);
     …
     Student s(7, “ringo”, new Bag7(), univ);
     Student* ps =
      new Student(8, jango, new Bag5(), univ);
}
```

הערה: תבנית העיצוב Factory

מוטיביציה:

- לעתים נדרש לא רק לאתחל את השדות, כמו בדוגמא שלנו, אלא גם לבצע פעולות אתחול אחרות: התחברות לשרת ורישום שם (כמו רישום הסטודנט בבסיס הנתונים של האוניברסיטה, ועוד. דוגמא בהמשך הקורס). ברצוננו להפריד בין יצירת סטודנט חוקי בזיכרון, לבין פעולות נלוות אח"כ (נראה בפרק הבא מדוע, הרעיון הכללי: איננו רוצים לשלוח לדוגמא לשרת סדונט שלא הסתיימה עדיין פעולת הבנאי שלו)
- לעתים, פעולת הבנאי עשויה לזרוק Exception. ייתכן שזה יקרה אחרי שהבנאי ייצר מידע ב Heap, כך שהכתובת שלו אינה ידוע, כי היצירה של האובייקט שהחזיק את הכתובת נכשלה.

 תבנית ה Factory

- נגדיר את הבנאי כ private (כלומר לא ניתן ליצור מבחוץ מופע של המחלקה ע"י קריאה לבנאי)
- נגדירה מתודה סטטית המחלקה המייצרת מופע של המחלקה ב Heap ומחזירה את הכתובת שלו.

חיסרון: הגדרת המופע בהכרח ב Heap.

```
class Student {

    private:
        Student(…) { …}
   …
   public:
        static Student* create(….) {
               Student* ret = new Student(…);
               // additional initialization code
              UniversitySever.register(*ret);
        }
}

Student* s = Student.create(…);
…
delete s;
```

2.4.3 מחיקת אובייקט מהזיכרון

הקומפיילר מתרגם את המחיקה של אובייקט מהזיכרון – בעקבות סיום הבלוק שבו הוא הוגדר כאובייקט על המחסנית, או בעקבות ביצוע delete על הכתובת שלו אם הוא הוגדר על ה Heap – לשתי פעולות:

- הפעלת מנגנון ההריסה
  - ביצוע מתודת הדיסטרקטור
  - הפעלה אוטומטית של מנגנון ההריסה עבור כל השדות מטיפוס אובייקט
- שחרור הזיכרון של האובייקט (מהמחסנית, או מה Heap)

באופן זה, אין צורך אף פעם לדאוג לשחרור של אובייקט על המחסנית. נדרש רק לבצע delete כל משתני הכתובת לפני שהם יורדים מהמחסנית, כלומר נדרש ליזום את המחיקה של האובייקטים המאוחסנים ב Heap. ע"פ הצעת העיצוב שלנו, מי שמחזיק את משתנה הכתובת הוא זה שצריך לבצע עליו delete.

דוגמא: למטה

 במתודת הדיסטרקטור, נבצע delete על המצביעים שבאחריות המחלקה (בגלובל, כל המצביעים במחלרה, עד כדי אילוצים טכניים דוגמת הקושי להגידר ווקטור של רפרנסים)

100 [Student]		200 [String]	300 [Bag 7]	400 [string]	500 [Bag 5]	600	700	800	900	1000
id 8		ringo	.	Jango	.					
name	chars 400	240 [DigitalBook]	.	440 [DigitalBook]	550 [Link]					
	length 5	.	350 [Link]	.	440					
bag 500		.	240	.	560					
books	lst 550	260 [HardCopyBook]	360	460 [HardCopyBook]	560 [Link]					
	size 2	.	360 [Link]	.	460					
univ		.	260	.	0					
		.	0	.						

Stack

Address	Data	Var		
9940				
9948				
9956				
9944	100			ps
9952	7	id		s
9960	200	chars	Name	
9968	5	length		
9976	300	bag		
9984	350	lst	Books	
9992	2	size		

10000		univ	

```
class Student {

  private:
       int _id;
       string _name;
       Bag& _bag;
       vector<Book*> _books;
       const University& _univ;

     public:
        Student(int id, const string& name,
                       Bag* bag, const
University& univ)
            : _id(id), _name(name), _bag(bag),
              _univ(univ)
       {
              _books.push_back(new
DigitalBook());
              _books.push_back(new
HardCopyBook());
       }
       ~Student() {
           delete _bag;
           for (int i=0; i<_books.size(); i++)
                  delete _books.at(i)
       }
};

{
   University univ(…);
   …
   Student s(7, “ringo”, new Bag7(), univ);
   Student* ps =
     new Student(8, jango, new Bag5(), univ);
    delete ps;
}
```

2.4.4 העתקת אובייקט

ראינו כי ב C++ ניתן להעביר אובייקט לפרוצדורה ByValue או להחזיר אותו מפרוצדורה By Value.
כלומר, האובייקט משוכפל ומועתק מ AF אחד ל AF אחר (תרחיש זה אינו קיים ב Java, כי שם אובייקט תמיד מייצג כתובת)
כיצד נקבעת הסמנטיקה/המשמעות של פעולת ההעתקה עבור אובייקטים (עבור טיפוסים פשוטים או כתובות המשמעות ברורה)? האם צריך העתקה עמוקה לדוגמא או אולי משהו אחר? וכדומה
ב C++ המתכנתים קובעים את המשמעות של ההעתקה ע"י מימוש של בנאי ייעודי לכך, המכונה Copy Constructor. בכל פעם שנדרשת ההעתקה של אובייקט (בפרט, בהעברה או בהחזרה ל/מ פרוצדורה) הקומפיילר מתרגם זאת לקריאה לבנאי זה.
מפאת חשיבותו, כמגדיר משמעות העתקת האובייקטים מהמחלקה הנתונה, הקומפיילר מגדיר ברירת מחדל עבורו בכל מחלקה. לדוגמא, עבור המחלקה Student:

```
class Student {
     …
     Student(const Student& other) : _id(other._id), _name(other._name),
_bag(other.bag),
                                     _books(other._books),
_univ(other._univ)
     {}
};

void F(Student s) { … }
{
     Student s(7, “ringo”, new Bag7(), univ);
     F(s);
     cout << (*s.getBag());
}
```

אם אין שדות מסוג מצביעים, ניתן להסתפק בברירת המחדל. אך אם יש מצביעים, נדרש לממש בנאי-מעתיק המייצר עותקים בזיכרון של האובייקטים המוצבעים ע"י שדות אלו.

```
class Student {

     …
     Student(const Student& other) :_id(other._id), _name(other._name(,
_univ(other._univ) {
           _bag =  other._bag->clone();
           for (int i=0; i<other.books.size(); i++)
                _books.push_element(other.books.at(i)->clone());
     }
};

class Bag7 : public Bag {
   …
   virtual Bag* clone() { return new Bag7(*this); }
};
```

```
class DigitalBook : public Book {
    …
    virtual Book* clone() { return new DigitalBook(*this); }
};
```

100	200 [String]	300 [Bag 7]	400 [string]	500 [Bag 7]	600	700	800	900	1000
	ringo	.	ringo	.					
	240 [DigitalBook]	.	440 [DigitalBook]	.					
	.	350 [Link]	.	550 [Link]					
	.	240	.	440					
	260 [HardCopyBook]	360	460 [HardCopyBook]	560					
	.	360 [Link]	.	560 [Link]					
	.	260	.	460					
	.	0	.	0					

Stack

Address	Data	Var	
9916	7	id	s
9924	400	chars	
9932	5	length	

9940	500	bag		
9948	550	lst		
9956	2	size		
9944		univ		
9952	7	id		s
9960	200	chars	name	
9968	5	length		
9976	300	bag		
9984	350	lst	books	
9992	2	size		
10000		univ		

```
class Student {

  private:
       const int _id;
       string _name;
       Bag* _bag;
       vector<Book*> _books;
       const University& _univ;

  public:
Student(const Student& other) :
         _id(other._id), _name(other._name(,
         _univ(other._univ)
{
  _bag = other._bag ->clone();
  for (int i=0; i<other.books.size(); i++)
     _books.push_element
       (other.books.at(i)->clone());
}
};

F(Student s) { … }

{
     Student s(7, "ringo", new Bag7(), univ);
     F(s);
     cout << (*s.getBag());
}
```

2.4.5 השמת אובייקט

ב C++ ניתן לבצע השמה בין אובייקטים (ב Java זה לא אפשרי, כי אובייקטים ב Java הם כתובות)

```
Student s1(7, “ringo”, new Bag7(), univ);
…
Student s2(8, “jango”, new Bag5(), univ);
…
s1 = s2; // s1.opreator=(s2);
```

מה המשמעות של של פעולת השמה זו? עבור טיפוסים פשוטים או כתובות משמעות פעולת ההשמה ברורה, אך עבור אובייקטים זה לא ברור (השמה עמוקה או לא, וכדומה)?
ב C++ משמעות פעולת ההשמה מוגדרת ע"י המתכנתים, ע"פ מתודה מיוחדת המכונה 'אופרטור השמה'.
מפאת חשיבותו, האופרטור מוגדר כבר ע"י הקומפיילר כברירת מחדל באופן הבא:

```
class Student {
     …
     void operator=(const Student& other) {
           if (&other != this) {
                 _id = other._id;
                 _name = other._name;
                 _bag = other._bag;
                 _ books = other._books;
                 _univ = other._univ;
           }
    }
};
{
     Student s1(7, “ringo”, new Bag7(), univ);
     …
    {
          Student s2(8, “jango”, new Bag5(), univ);
           …
           s1 = s2;
           …
    }
    cout << (*s1.getBag());
}
```

בעיות:
- נזילת זיכרון (אי-מחיקת bag ואובייקטי ה Book)
- שני האובייקטים חולקים אותו תיק ואותם ספרים. מחיקה של האחד מהזיכרון תעלים את התיק והספרים של השני הקיים.

גם כאן, לא ניתן להסתפק בברירת המחדל כאשר יש שדות מטיפוס מצביע – במקרה זה, נממש את אופרטור ההשמה כך שיעתיק רק את התוכן של המידע ולא את הכתובות.

```
class Student {
      …
      void operator=(const Student& other) {
            if (&other != this) {
                  _id = other._id;
                  _name = other._name;
                  delete  _bag;
                  _bag = other._bag->clone();
                  // delete old books
                  for (int i=o; i<_books.size(); i++)
                        delete _books.at(i);
                  // clear old books addresses
                  _books.clear();
                  // add other books
                  for (int i=o; i<other._books.size(); i++)
                        _books.push_back(other._books.at(i)->clone());
                  univ = other._univ;
            }
     }
};
```

הערה: כדי לתמוך ב a=b=c

נגדיר את אופרטור ההשמה כמבצע השמה לשדותיו ע"פ other המועבר כפרמטר (כמו קודם) **ובנוסף** כך שיחזיר את עצמו, למקרה של השמה מרובה.

```
a = b = c;    □□    a.operator=(b.operator=(c));
```

```
class Student {
      …
      Student& operator=(const Student& other) {
            if (&other != this) {
                  _id = other._id;
                  _name = other._name;
                  delete  _bag;
                  _bag = other._bag->clone();
                  // delete old books
                  for (int i=o; i<_books.size(); i++)
                        delete _books.at(i);
                  // clear old books addresses
                  _books.clear();
                  // add other books
                  for (int i=o; i<other._books.size(); i++)
                        _books.push_back(other_books.at(i)->clone());
                  univ = other._univ;
```

```
        }
        return *this;
};
```

100	200 [String]	300 [Bag 7]	400 [string]	500 [Bag 5]	600	700	800	900	1000
	jango	.	jango	.					
	240 [DigitalBook]	.	440 [DigitalBook]	550 [Link]					
	.	350 [Link]	.	440					
	.	240	.	560					
	260 [HardCopyBook]	360	460 [HardCopyBook]	560 [Link]					
	.	360 [Link]	.	460					
	.	260	.	0					
	.	0	.						

Stack

Address	Data	Var	
9916	8	Id	s2
9924	400	chars	
9932	5	length	
9940	500	bag	
9948	550	lst	
9956	2	size	
9944		univ	
9952	8	id	s1

9960	200	chars	name	
9968	5	length		
9976	500	bag		
9984	350	lst	books	
9992	2	size		
10000		univ		

```
class Student {
...
Student& operator=(const Student& other) {
   if (&other != this) {
     _id = other._id;
     _name = other._name;
     delete  _bag;
     _bag = other._bag->clone();
     // delete old books
     for (int i=0; i<_books.size(); i++)
        delete _books.at(i);
     // clear old books addresses
     _books.clear();
     // add other books
     for (int i=0; i<other._books.size(); i++)
            _books.push_back(_books.at(i));
            univ = other._univ;
     }
     return *this;
};

{
     Student s1(7, "ringo", new Bag7(), univ);
    {
     Student s2(8, "jango", new Bag5(), univ);
     s1 = s2;
    }
    cout << (*s1.getBag());
}
```

הערה: Operator Overloading

בדומה להגדרת אופרטור ההשמה, ניתן בכל מחלקה להגדיר את הסמנטיקה של פעולות סטנדרטיות שונות, כמו: +,-,*,/,<,>,==,+=,*=,[],<< ועוד. זהו עיצוב אלגנטי ומומלץ, אך לא נתעכב עליו. נגענו רק באופרטור ההשמה, כי הוא מגדיר הקונספט הבסיסי בשפות תכנות של השמה.

מימוש הדיסטרקטור, הבנאי המעתיק ואופרטור ההשמה במחקה נתונה מכונה **rule-of-3**. מימוש זה הכרחי כי הוא מגדיר את הסמנטיקה של המחלקה תחת תרחישים בסיסיים של שפת התכנות: מחיקה מהזיכרון, העברה by value והשמה.

אופטימיזציית move

קיימת אופטימיזציה על העתקת/השמת אובייקט זמני (r-value, שאין לו תא עם כתובת בזיכרון), בה השדות מטיפוס כתובת לאובייקט ב-heap אינם משוכפלים אלא מנוצלים, כי האובייקט שממנו הם נלקחים הוא זמני ולא ייעשה בהם שימוש.

עבור בנאי מעתיק

```
Student s1(Student(1,”yosi”,new Bag7(),univ));
```

במקרה זה נרצה לממש גרסה נוספת של copy constructor שבה ההעתקה אינה עמוקה, כך שהשדה _bag של s1 יצביע לאובייקט ה-_bag של הסטודנט הזמני, והספרים בווקטור _books של s1 יצביעו לספרים של הסטודנט הזמני. ובנוסף נדאג לכך שאובייקטים זה לא יימחק בדיסטרקטור של הסטודנט הזמני על ידי השמת ל-nullptr.

האבחנה בין שתי הגרסאות ניתנת על ידי הגדרת הפרמטר other:

Student**&** - עבור פרמטר שאינו זמני, l-reference
Student**&&** - עבור פרמטר זמני, r-reference

```
Student(Student&& other) :_id(other._id), _name(other._name(,
_univ(other._univ),
   // ‘steal’ other’s bag
   _bag(other._bag)
   // ‘steal’ other’s books
   _books(other._books)
{
     // neutralize the deletion of the bag by the distructor of other
     other._bag = nullptr;
     // neutralize the deletion of the books by the distructor of other
     other._books.clear();
}
```

עבור אופרטור השמה

```
s1 = Student(2,”danny”, new Bag5(),univ));
```

במקרה זה נרצה לממש גרסה נוספת של assignment operator שבה ההעתקה אינה עמוקה, כך שהשדה _bag של s1 יצביע לאובייקט ה-_bag של הסטודנט הזמני, והספרים בווקטור _books של s1

יצביעו לספרים של הסטודנט הזמני. ובנוסף נדאג לכך שאובייקטים זה לא יימחק בדיסטרקטור של הסטודנט הזמני על ידי השמת ל-nullptr.

האבחנה בין שתי הגרסאות ניתנת על ידי הגדרת הפרמטר other:

Student**&** - עבור פרמטר שאינו זמני, l-reference
Student**&&** - עבור פרמטר זמני, r-reference

```
Student& operator=(const Student&& other) {

          if (&other != this) {
                _id = other._id;
                _name = other._name;
                univ = other._univ;

                delete  _bag;
                // 'steal' other's bag
                _bag = other._bag;
                // neutralize the deletion of the bag by the distructor
of other
               other._bag = nullptr;

                // delete old books
                for (int i=o; i<_books.size(); i++)
                      delete _books.at(i);
                // 'steal' other's books
                 _books = other._books;
                 // neutralize the deletion of the books by the
distructor of other
                 other._books.clear();
          }
           return *this;
    };
```

מימוש שתי מתודות נוספות בנוסף לשלוש הקודמות מכונה **rule-of-5**.

Smart Pointers 2.4.6

לסיום העיסוק באובייקטים, נדגים את פעולות המחיקה/העתקה/השמה דרך תבנית עיצובית המכונה Smart Pointers.

<u>המטרה</u>: הגדרת מחלקה כללית לניהול הזיכרון של מידע ב Heap המיוצג ע"י מצביע (=הכתובת שלו), ובפרט מחיקתו כאשר אף אחד לא צריך יותר מידע זה.

דוגמא:

```
{
   Student* s1(new Student(...));
```

```
   F(s1);
   Student* s2(new Student(...));
   …
   s1 = s2;
   …
   delete s1 ???
   delete s2 ???
}

void F(Student* s) { … }
```

לא ברור האם צריך למחוק את s1,s2 לפני תום הבלוק (כדי למנוע נזילות זיכרון) או שאסור זאת (כדי למנוע גישה לזיכרון מחוק, מאחר שאיננו בטוחים מה קרה ב F, או בעקבות ההשמה של s1=s2)
 נעטוף את המצביעים בקוד במחלקה המנהלת את הזיכרון שלהם, ובפרט מוחקת אותם כשצריך.

```
{
   smart_ptr<Student*> s1(new Student(...));
   F(s1);
   smart_ptr<Student*> s2(new Student(...));
   …
   s1 = s2;
   …
}

void F(smart_ptr<Student*> s) { … }
```

כיצד נעצב מחלקה זו (smart_ptr)?
- המחלקה מכילה את הכתובת שהיא צריכה לנהל (מקבלת אותה בבנאי), וכן מונה המציין כמה משתנים אחרים מכירים כתובת זו (בהתחלה 1)
- בכל פעם שמשתנה נוסף מכיר את הכתובת (כאשר הוא מועתק ל AF חדש, וכאשר הוא מושם למצביע אחר) נגדיל את המונה, ובכל פעם שמשתנה אחד פחות מכיר את המצביע (בזמן ההריסה) נקטין את המונה
- כאשר המונה שווה ל-0 נמחק את המידע ב-Heap בכתובת הנתונה.

<u>מימוש</u>: המימוש יבוסס בעיקר על ה rule-of-3 של המחלקה smart_prt, שהרי שלושת התרחישים של שינוי המונה הם שלושת המתודות: דיסטרקטור, בנאי העתקה ואופרטור ההשמה.

<u>הערה</u>: ברצוננו להגדיר מחלקה כללית לכל סוג של מצביע (Student*, int*, Animal*,…) נגדיר תבנית כללית של מחלקה, כאשר לפני שמשתמשים בה מגדירים מה לשבץ בשדה של טיפוס המצביע. ניתן לעשות זאת באמצעות תחביר ה template ב C++.
תחביר זה מאפשר להגדיר תבניות של קוד עם אפשרות לשיבוץ של טיפוסים שונים בקוד. שימושי עבור:
מבני נתונים: vector, map
אלגוריתמים: sort

```
template <typename T>
class smart_ptr {
     private:
           T _ptr;
           int* _refcount;

         void checkIfDelete() {
               if ((*_refcount) == 0) {
                   delete _ptr;
                   delete _refcount;
              }
           }
   public:
           smart_ptr(T ptr) : _ptr(ptr), _refcount(new int(1)) { }

           smart_ptr(const smart_ptr& other) : _ptr(other._ptr),
_refcount(othr._refcount) {
                     (*_refcount)++;
           }

          ~smart_ptr() {
                 (*_refcount)--;
                 checkIfDelete();
        }

        smart_ptr& operator=(const smart_ptr& other) {
              if (this != &other) {
                 (*_refcount)--;
                  checkIfDelete();
                 _ptr = other._ptr;
                 _refcount = other._refcount;
                 (_refcount)++;
              }
               return *this;
        }

       T getPtr() { return _ptr; }
};
```

100	200	300	400

Address	Data	Var
9960		
9968		
9976		
9984		
9992		
10000		

```
template <typename T>
class smart_ptr {
     private:
           T _ptr;
           int* _refcount;

          void checkIfDelete() {
                if ((*_refcount) == 0) {
                    delete _ptr;
                    delete _refcount;
               }
           }
   public:
           smart_ptr(T ptr) : _ptr(ptr), _refcount(new int(1)) { }

           smart_ptr(const smart_ptr& other) :
              _ptr(other._ptr), _refcount(othr._refcount) {
                     (*_refcount)++;
           }

          ~smart_ptr() {
                 (*_refcount)--;
                 checkIfDelete();
          }

          smart_ptr& operator=(const smart_ptr& other) {
                if (this != &other) {
                   (*_refcount)--;
                    checkIfDelete();
                   _ptr = other._ptr;
                   _refcount = other._refcount;
                   (_refcount)++;
               }
                return *this;
          }

         T getPtr() { return _ptr; }
};
```

2.5 ירושה ופולימורפיזם

2.5.1 מבוא – הגדרת סוגי יחסים בין טיפוסים

שפות תכנות שונות, מאפשרות להגדיר יחסים שונים בין טיפוסים בתוכנית (כלומר, קיים תחביר השפה המאפשר הגדרה של יחסים אלו):

A **is-a** B

Student **is-a** Person

ב Java:

```
class Person {
    protected final int _id;
    protected String _name;
    …
}

class Student extends Person {
     protected  Bag _bag;
     protected List<Book> _books;
     protected final University _univ;
     …
}
```

ב C++:

```
class Person {
   protected:
    const int _id;
    string _name;
    …
};

class Student : public Person {
   protected:
     Bag* _bag;
     vector<Book*> _books;
     const University& _univ;
}
```

A **is-implemented-in-terms-of** B
Living **is-implemented-in-terms-of** Person
Loving **is-implemented-in-terms-of** Person

Learning **is-implemented-in-terms-of** Student

ב Java:

```
interface Living {
     void live();
}

interface Loving {
    void love();
}

interface Learning {
    void learn();
}
class Person implements Living, Loving {
    final int _id;
    String _name;
    …

    void live() { … }
    void love() { … }
}

class Student extends Person implements Learning {
     Bag _bag;
     List<Book> _books;
     final University _univ;
     …
     void learn() { … }
     void live() { … }
}
```

ב C++ אין לצערנו אבחנה תחבירית בין מימוש יכולת / מימוש ממשק (יחס is-implemented-in-terms-of) לבין ירושה (יחס is-a), ניתן רק לרשת מספר מחלקות.
בכל זאת, אנחנו נקפיד על עיצוב המבחין בין שני יחסים אלו:
- את המחלקה של היחס is-a נגדיר כמחלקת האב הראשונה
- את שאר מחלקות האב נגדיר כמחלקות **אבסטרקטיות** (כלומר כמחלקות הכוללות רק הצהרות של מתודות, ללא מימוש וללא שדות)

```
class Living {
     virtual void live() =0;
};

class Loving {
     virtual void love() =0;
};
```

```
class Learning {
     virtual void learn() =0;
};
class Person : public Living, public Loving {
    const int _id;
    string _name;
    …
    void live() { … }
    void love() { … }
};

class Student : public Person, public Learning {
     Bag* _bag;
     vector<Book*> _books;
     const University& _univ;
     …
     void learn() { … }
     void live() { …}
};
```

A **has-a** B
Student **has-a** Bag

ממומש ב Java וב C++ ע"י composition, כלומר הגדרת שדה במחלקה A מטיפוס B.

2.5.2 מבנה, אתחול, הריסה, העתקה והשמה במחלקה יורשת

<u>מבנה</u>: מה המבנה של מחלקה כמו Student היורשת מחלקות אחרות?

מבנה מחלקת האב הראשונה
מבנה מחלקת האב השניה
…
מבנה מחלקת האב האחרונה
מבנה מחלקת הבן
[הכל רקורסיבי]

אתחול

אתחול אוטומטי של מחלקת האב הראשונה
אתחול אוטומטי של מחלקת האב השניה
…
אתחול אוטומטי של מחלקת האב האחרונה
אתחול אוטומטי של מחלקת הבן

אם רוצים לקרוא לבנאי ספציפי של כל אחת ממחלקות האב יש לציין זאת בשורת האתחול

הריסה

מנגנון הריסה אוטומטי של מחלקת הבן
מנגנון הריסה אוטומטי של מחלקת האב האחרונה
...
מנגנון הריסה אוטומטי של מחלקת האב הראשונה

העתקה

קריאה אוטומטית לבנאי של מחלקת האב הראשונה. אם לא הוגדרה קריאה לבנאי המעתיק בשורת האתחול של מחלקת הבן (עם other) ייקרא הבנאי הריק.
קריאה אוטומטית לבנאי של מחלקת האב השניה. כנ"ל
...
קריאה אוטומטית לבנאי של מחלקת האב האחרונה. כנ"ל
ביצוע הבנאי המעתיק עבור הבן (העתקת שדות)

[הערה לגבי אופטימיזציית rule-of-5: אם רוצים להעביר את other לבנאי המעתיק של מחלקת האב כr-reference צריך להשתמש ב std::move]

השמה

<u>ברירת מחדל</u>: ביצוע גוף אופרטור ההשמה של מחלקת הבן

בדרך כלל נדרש להפעיל את אופרטור ההשמה של מחלקות האב. יש ליזום את הקריאות במפורש כי זה לא קורה אוטומטית. כלומר, מתוך גוף אופרטור ההשמה:

קריאה לאופרטור ההשמה של מחלקת האב הראשונה (עם other)
קריאה לאופרטור ההשמה של מחלקת האב השניה (עם other)
...
קריאה לאופרטור ההשמה של מחלקת האב האחרונה (עם other)
ביצוע גוף אופרטור ההשמה של מחלקת הבן (השמת שדות)

[הערה לגבי אופטימיזציית rule-of-5: אם רוצים להעביר את other לבנאי המעתיק של מחלקת האב כ r-reference צריך להשתמש ב std::move]

<u>דוגמא</u>:

```
class Person : public Living, public Loving {
    const int _id;
    string _name;
    ...
    void live() { ... }
    void love() { ... }

    Person(int id, const string& name) : _id(id), _name(name)
};
```

```
class Student : public Person, public Learning {
     Bag* _bag;
     vector<Book*> _books;
     const University& _univ;
     …
     void learn() { … }
     void live() { …}

    Student(int id, const String& name, Bag* bag, const University& univ)
          : Person(id, name), _bag(bag), _univ(univ) {

            _books.push_back(new DigitalBook());
           _books.push_back(new HardCopyBook());
    }

    Student(const Student& other)
          : Person(other), _bag(other._bag->clone()), _univ(univ) {

         …
    }
Student& operator=(const Student& other) {
    if (this != &other) {
         Person::operatror=(other);
         delete _bag;
         _bag = other._bag->clone();
         …
    }
     return *this;
}

[virtual] ~Student() {
     delete _bag;
     …
}

};
```

Address	Data	Var		
9944				
9952				
			Living	s
			Loving	

9960		id		Person	
9968		chars	name		
9976		length			
				Learning	
9984		bag		Student	
9992		lst	books		
10000		size			
		univ			

```
{
    Student s(7, "ringo", new Bag7(), univ);
}
```

2.5.3 מימוש הפולימורפיזם בזיכרון

כזכור, קוד התוכנית נטען בשלב האתחול של התהליך על ידי מערכת ההפעלה לזיכרון התהליך (לאזור בזיכרון המכונה code section). בפרט, כל אחת מהפונקציות מאוחסנת בכתובת מסוימת בזיכרון.
ראינו כי קריאה לפונקציה מתורגמת לפעולת השמה של ה program counter לכתובת שבה נמצא קוד הפונקציה.
פעולה זו (תרגום הקריאה לפונקציה להשמה של ה pc לכתובתה(מכונה Binding.

מתי מתבצע ה Binding?
אפשרות א: בזמן קומפילציה, static binding
אפשרות ב: בזמן ריצה, dynamic binding

ב Java, ה binding תמיד דינאמי, בזמן ריצה.
ב C++, המתכנתים קובעים עבור כל פונקציה האם תרגום הקריאות אליה סטטי (בזמן קומפילציה) או דינאמי (בזמן ריצה). ברירת המחדל: בזמן קומפילציה.

מדוע נרצה שהתרגום יהיה רק בזמן ריצה?

```
class A {
    virtual void F() { cout << “A”; }     // address 1000
};
class B : public A {
     virtual void F() { cout << “B”; }   // address 2000
};
{
     A* a;
     …
     a = new B();
     …
     a->F(); // pc = 1000, “A” ----- pc = <formula for runtime>, “B”
}
```

כדי לתמוך בפולימורפיזם, ה binding של הפונקציה צריך להיות דינאמי.
כיצד אומרים זאת לקומפיילר ב C++? המילה השמורה virtual מציינת את הדרישה שהקריאות לפונקציה זו יתורגמו רק בזמן ריצה.

<u>הערות</u>:

- תכונת ה virtual של פונקציה עוברת בירושה
- כעת מובן מדוע פונקציות אבסטרקטיות חייבות להיות וירטואליות: אי אפשר לדרוש מהקומפיילר לתרגם את הקריאות אליהן בזמן קומפילציה כי אין להן מימוש.

```
Learning* l;
l = new Student(…);
l->learn(); // ???
```

- הדיסטרקטור צריך להיות וירטואלי, כדי לוודא שמחיקה של אובייקט ממחלקה יורשת תחולל את מנגנון ההריסה של מחלקת האב

```
Person* p;
p = new Student(...);
delete p;
```

מימוש ה dynamic binding

<u>מבנה נתונים</u>:

- הקומפיילר מגדיר עבור כל מחלקה שיש בה מתודה וירטואלית טבלה בזיכרון המכילה את הכתובות בזיכרון שבהן נמצא קוד המימוש של המתודות הוירטואליות במחלקה. טבלה זו מכונה v-table.
  לדוגמא, עבור המחלקות Person, Student

Person v-table, at address 100: [7000, 4000]

Student v-table, at address 200: [8000, 4000, 9000]

כלומר: קוד מימוש המתודה live במחלקה Person הוא בכתובת 7000
קוד מימוש המתודה love במחלקה Person הוא בכתובת 4000
קוד מימוש המתודה live במחלקה Student הוא בכתובת 8000
קוד מימוש המתודה love במחלקה Person הוא בכתובת 4000
קוד מימוש המתודה learn במחלקה Student הוא בכתובת 9000

- כל מופע של מחלקה שיש לה v-table (כלומר של מחלקה עם מתודות וירטואליות) כולל תא נוסף בזיכרון שלו (בתחילתו) המכיל את הכתובת של ה v-table שלו. תא זה יאותחל בפעולת ה new/push בהתאם לטיפוס בזמן הריצה.

באופן זה, הקומפיילר יכול להכין 'נוסחה' בזמן קומפילציה לאיתור הכתובת של המתודה בהתאם לסוג המופע בזמן ריצה, ע"פ ה v-table שלו:
כתובת האובייקט היא התא שבו נמצא ה v-table, התוכן שלו היא כתובת ה v-table, המתודה המדוברת מאוחסנת המערך המכיל את כתובות המתודות ע"פ סדרן, יש לגשת לתא במערך עם הכתובת של המתודה.
לדוגמא:

```
{
  Person* p;
  p = new Student (7, “ringo”, new Bag7(), univ);
  p->live();
}
```

הקריאה p->live() תתורגם ל (pc = (*((*p) + 0)*))

Heap

Address	Data	Var		
9936				
9944				
9952	200	v-table		p
			Living	
			Loving	
9960		id		Person
9968		chars	name	
9976		length		
			Learning	
9984		bag		Student
9992		lst	books	
10000		size		
		univ		

```
{
 Person* p;
 P = new Student (7,"ringo",new Bag7(),univ);
 p->live();
}
```

חסרונות:
זיכרון – אחסון הטבלאות הוירטואליות, תא נוסף לכל מופע
זמן – שתי גישות נוספות לזיכרון בכל הפעל של מתודה

The Double-Dispatch Design Patter 2.5.4

נסיים את הפרק בתבנית עיצוב הממחישה את מימוש הפולימורפיזם – **double dispatch**

<u>מוטיבציה</u>:
- נתונים שני ממשקים, לכל ממשק מספר מימושים.
- קיים קוד שונה לכל שילוב בין המימושים של שני הממשקים
- בהינתן שני מופעים של ממשקים אלו, יש להגיע לקטע הקוד המתאים עבורם.

דוגמא:

```
class Printer {
	virtual void print(const Doc& doc)=0;
}

class HP : public Printer {
	virtual void print(const Doc& doc) { … }
}

class Xerox : public Printer {
	virtual void print(const Doc& doc) { … }
}
```

```
class Doc {
      …
}

class PDF : public Doc {
     …
}

class Word : public Doc {
   …
}

{
     Printer* hp;
     Printer* xerox;
     Doc* pdf;
     Doc* word;
     …
     hp = new HP();
     xerox = new Xerox();
     pdf = new PDF();
     word = new Word();
    …
     hp->print(*pdf);  // should be directed to HP-PDF code
     hp->print(*word); // should be directed to HP-WORD code
     xerox->print(*pdf); // should be directed to XEROX-PDF code
     xerox->print(*word); // should be directed to XEROX-WORD code
     …
     delete hp;
     delete xerox;
     delete word;
     delete pdf;
}
```

כיצד יתרגם הקומפיילר את הקריאה: hp->print(pdf);

המתודה print וירטואלית המחלקה Printer, כך שהקומפיילר יתרגם את הקריאה אליה לנוסחה כמו:

pc = (*)(*hp(+ 0))

כלומר, נגיע למימוש של print במחלקה HP

אמרנו, כי קיים קוד שונה להדפסה עבור כל מסמך. כיצד נדע באיזה מסמך מדובר (הוא הרי מיוצג במתודה כ Doc)?

אפשרות א: נבדוק בתוך הקוד מה הטיפוס שלו (instanceof ב Java, dynamic_cast אם זה C++)

```
class HP : public Printer {
      virtual void print(const Doc& doc) {
```

```
            if (dynamic_cast<PDF*>(doc)) {  // code for HP-PDF }
             if (dynamic_cast<Word*>(doc)) {  // code for HP-Word }
      }
}

class Xerox : public Printer {
      virtual void print(const Doc& doc) {
            if (dynamic_cast<PDF*>(doc)) {  // code for Xerox-PDF }
             if (dynamic_cast<Word*>(doc)) {  // code for Xerox-Word }
      }
}
```

חיסרון: כל פעם שיש מימוש חדש ל Doc צריך להיכנס לכל מקום בקוד של כל המימושים של Printer המתייחסים לסוג המסמך, לעדכן אותם עם סוג המסמך החדש, לקמפל, להריץ טסטים.

הצעה: נוסיף בממשק של Printer חתימות נוספות למתודת ה print עבור כל סוג של מסמך. באופן זה, העדכון הנדרש בכל מדפסת כאשר יש מסמך חדש, הוא ברמת מימוש מתודה, ולא עדכון קוד קיים.

```
class Printer {
      virtual void print(const Doc& doc) =0;
      virtual void print(const PDF& pdf) =0;
      virtual void print(const Word& word) =0;
}

class HP : public Printer {
      virtual void print(const Doc& doc) { … }
      virtual void print(const PDF& pdf) { // code for HP-PDF}
      virtual void print(const Word& word) { // code for HP-Word }
}

class Xerox : public Printer {
      virtual void print(const Doc& doc) { … }
      virtual void print(const PDF& pdf) { // code for Xerox-PDF}
      virtual void print(const Word& word) { // code for Xerox-Word }
}
```

בעיה: הקריאה hp->print(pdf) לא תגיע לגרסה של print במחלקה HP המקבלת PDF אלא לזו המקבלת Doc

אמנם החיפוש יהיה בזמן ריצה ע"פ ה v-table של Printer, אך מה מחפשים, כלומר איזו חתימה של פונקציה, נקבע בנוסחה כבר בזמן קומפילציה ע"פ הטיפוס של הפרמטר בזמן קומפילציה (במקרה שלנו, pdf הוא Doc ולא PDF).

כלומר, מנגנון של ה dynamic binding המממש את הפולימורפיזם, דואג להפעלה של המתודות במחלקה המתאימה ע"פ טיפוס האובייקט עליו מופעלת המתודה בזמן ריצה, אך לא ע"פ הסוג הספציפי של הפרמטר של המתודה בזמן ריצה.

נפעיל שוב את המנגנון, ע"י ביצוע של מתודה כלשהי על הפרמטר:

```
class Doc {
     virtual void printMe(const Printer& printer) =0;
      …
}

class PDF : public Doc {
    virtual void printMe(const Printer& printer) { printer.print(*this); }
     …
}

class Word : public Doc {
    virtual void printMe(const Printer& printer) { printer.print(*this); }
    …
}
class HP : public Printer {
      virtual void print(const Doc& doc) {  doc.printMe(*this); }
      virtual void print(const PDF& pdf) { // code for HP-PDF }
      virtual void print(const Word& word) { // code for HP-Word }
}

class Xerox : public Printer {
      virtual void print(const Doc& doc) { doc.printMe(*this); }
      virtual void print(const PDF& pdf) { // code for Xerox-PDF}
      virtual void print(const Word& word) { // code for Xerox-Word }
}
{
     …
     hp->print(*pdf);  // should be directed to HP-PDF code
     hp->print(*word); // should be directed to HP-WORD code
     xerox->print(*pdf); // should be directed to XEROX-PDF code
     xerox->print(*word); // should be directed to XEROX-WORD code
    …
}
```

3. מקביליות (Concurrency)

3.1 מוטיבציה

נחזור לדוגמא מתחילת הקורס: הקרנה של קובץ וידיאו
ראינו כי פעולה זו מורכבת למעשה משלושה שלבים: read, decode, play
כיצד נריץ את שלושת השלבים הללו:

- באופן סדרתי
בתוכנית תהיה קריאה לשלושת הפונקציות read, encode, play בזו אחר זו, כאשר כל פונקציה משתמשת בערך שחזר מהפונקציה הקודמת.
חסרון: זמן המתנה מרובה עד אשר נזכה לראות שניה מהסרט. אין ניצול של העובדה כי המעבד הולך לעבור שוב ושוב למצב blocked במהלך הפעולות האלו (הרבה גישה ל i/o). כלומר, בשיטה זו ננצל מעט מאוד את הזמן שהתהליך קיבל עם המעבד. יתרה מזאת, אם יש מספר מעבדים פנויים במחשב, הם לא ינוצלו.
- באופן מקבילי כשלושה תהליכים שונים
נריץ את שלושת הפעולות האלה במקביל, בתהליכים שונים. תוכנית תבצע read, תוכנית שניה תבצע encode, תוכנית שלישית תבצע play. במודל זה, כאשר אחד התהליכים נתקע (=עובר למצב blocked), יהיה מעבר לתהליך אחר.
חיסרון: המעבר בין תהליך לתהליך הוא כבד
  - הפסקת הריצה של התהליך הנוכחי
  - שמירת המצב (context) בזיכרון של התהליך הנוכחי, הכולל בין היתר: קוד התוכנית, המחסנית והערימה, מבני הנתונים של מערכת ההפעלה לניהול התהליך.
  - בחירת התהליך לריצה
  - שחזור המצב שלו בזיכרון
  - עדכון זיכרון המטמון במעבד
  - הרצת התהליך הנבחר

בזמן המעבר הזה, המכונה context switch, המעבד לא עושה דבר בתהליכים אלו.

יתרה מזאת, מעבר זה אינו נדרש במקרה שלנו כי המשימות השונות קשורות לאותם נתונים (הסרט)

- באופן מקבילי, כשלוש משימות בתהליך אחד
במודל זה, רצות שלוש משימות במקביל על אותו זיכרון. כך שהמעבר ביניהן פשוט הרבה יותר. באופן זה, כאשר המעבד נתקע במשימה אחת (בשל גישה ל i/o) הוא יוכל לעבור למשימה אחרת באותו תהליך על אותו זיכרון.
חיסרון: נראה במהלך הפרק...

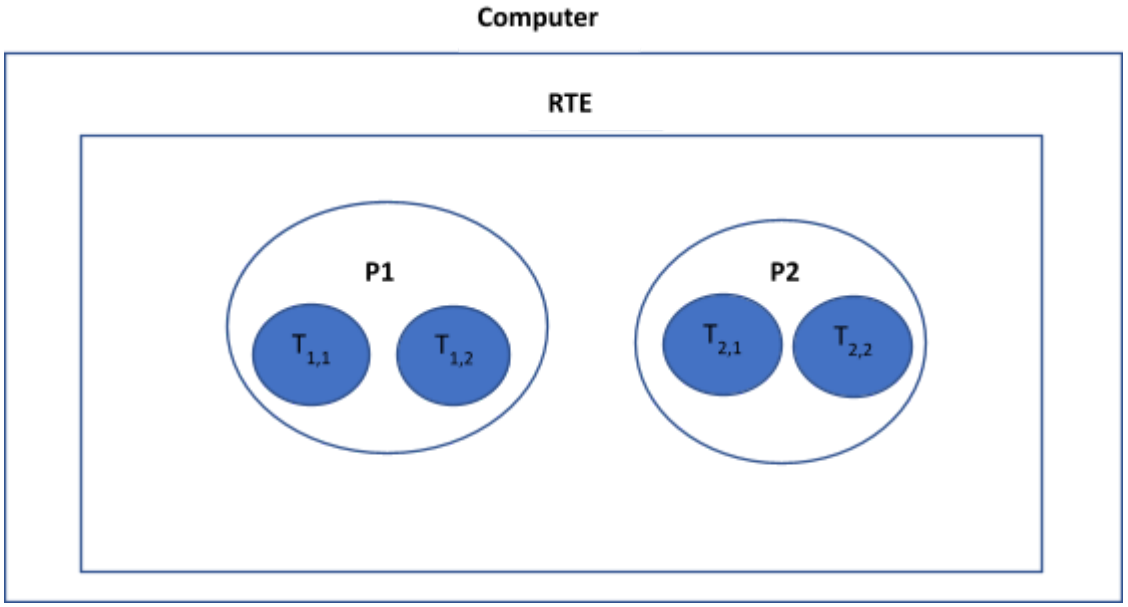
דוגמא נוספת: פס ייצור של מכוניות

3.2 מודל הרצה מקבילי בתהליך אחד

3.2.1 הגדרת Thread

נגדיר ת'רד כיחידת הפעלה בסיסית של המעבד, הכוללת:
- program counter (=משימה לביצוע)
- registers (=מצב המעבד המבצע את קוד המשימה)
- stack (=הזיכרון הרלבנטי, אוסף המשתנים הנדרש כרגע לביצוע הקוד הנוכחי)
מעבר מת'רד אחד לת'רד שני, כלומר מעבר ממשימה אחת למשימה שניה על אותו מעבד, כרוך בהחלפת שלושת הנתונים הללו בלבד (בניגוד להחלפה הכבדה במעבר מתהליך לתהליך)

נשים לב, כי לכל ת'רד יש מחסנית משלו, אך כל הת'רדים בתהליך חולקים את אותו Heap.



סוגיה במערכות הפעלה: האם הת'רדים הם 'ישויות' בסביבת זמן הריצה, כלומר האם מערכת ההפעלה מכירה את הת'רדים, מייצרת אותם, מתזמנת אותם, סוגרת אותם וכו'. או שהת'רדים מנוהלים פנימית ע"י קוד התהליך. בפרק זה, נניח כי הת'רדים מנוהלים על ידי סביבת זמן הריצה.

בהמשך לסוגיה הנ"ל, מבחינים בין שתי שיטות תזמון:

Preemptive scheduling – בגישה זו ההחלטה על המעבר מתהליך אחד לתהליך שני נמצאת בידי סביבת זמן הריצה, כחלק ממדיניות התזמון. כלומר, לאחר כל פעולה אטומית בקוד התהליך ייתכן מעבר לתהליך או לת'רד אחר.

Non-preemptive scheduling – היוזמה למעבר לתהליך או ת'רד מגיעה באופן יזום מקוד התוכנית. על ידי קריאה למערכת עם בקשה לעבור לתהליך/ת'רד אחר.

3.2.2 הרצת ת'רד ב Java

כאשר תוכנית רצה, סביבת הריצה מריצה ת'רד אחד שמשימתו היא מתודת ה main. ניתן במהלך ריצת התכנית להריץ משימות נוספות במקביל ע"י ת'רדים נוספים.

כדי להריץ משימה במקביל ע"י ת'רד בתהליך, נדרש:
א. הגדרת משימה
ב. בקשה מסביבת הריצה (כלומר, קריאה למערכת) ליצור ת'רד שיריץ משימה זו (כאשר המשימה תסתיים, הת'רד ייסגר ע"י הסביבה)

נממש זאת בג'אווה:

1. הגדרת משימה ב ג'אווה

הגדרת משימה בג'אווה ניתנת על ידי מימוש הממשק Runnable, הכולל מתודה אחת run:

```
interface Runnable {
    void run();
}
```

לדוגמא, נגדיר משימה של הדפסת מחרוזת על ערוץ פלט:

```
class MsgPrinter implements Runnable {
     String msg;
     PrintStream out;

     MsgPrinter(String msg, PrintStream out) { this.msg = msg; this.out =
out; }

    public void run() { out.print(msg); }
}
```

מופע של מחלקה זו, יכול להוות משימה עבור ת'רד.
לדוגמא: נגדיר שני מופעים של משימה זו, האחד עבור הדפסת Hello והשני עבור הדפסת Goodbye:

```
class Test {
     public void main(String[] args) {
           Runnable taskHello = new MsgPrinter(“Hello”, System.out);
           Runnable taskGoodbye = new MsgPrinter(“Goodbye”, System.out);
           …
     }
}
```

2. הרצת המשימה כת'רד

מה יקרה אם נבצע במתודת ה main את ה run של כל משימה?
הרצה סדרתית של שתי המשימות, ע"י הת'רד הראשי (והיחיד כרגע) של התוכנית. כך שהפלט יהיה: Hello Goodbye

Heap

100 [MsgPrinter]	200 [MsgPrinter]	300	400	500	600	700	800	900	1000
Hello	Goodbye								
System.out	System.out								

T0

Address	Data	Var
9940		
9948		
9956		
9944		
9952		
9960		

9968		
9976		
9984		
9992	200	taskGoodbye
10000	100	taskHello

כדי להריץ משימות אלו במקביל, יש לבקש מסביבת הריצה ליצור ת'רד עבור כל משימה.

לשמחתנו, אנו כותבים קוד בג'אווה כך שכל הקריאות לסביבה נעשות דרך ממשקים נוחים של מחלקות. בפרט, בקשות הקשורות לת'רדים נעשות בנוחות בעזרת המחלקה Thread.

- נגדיר אובייקט מטיפוס Thread, המקבל בבנאי את המשימה לביצוע.
- נקרא למתודה start, אשר תתורגם ע"י ה JVM לבקשה ממערכת ההפעלה ליצירת ת'רד (לא ניכנס לזה, אך ייתכן שהת'רד ינוהל ע"י ה JVM מבלי לערב את מערכת ההפעלה).

```
class Test {
    public void main(String[] args) {
        Runnable taskHello = new MsgPrinter("Hello", System.out);
class Test {
     public void main(String[] args) {
           Runnable taskHello = new MsgPrinter("Hello", System.out);
           Runnable taskGoodbye = new MsgGoodbye("Hello", System.out);
            Thread threadHello = new Thread(taskHello);
           Thread threadGoodbye = new Thread(taskGoodbye);
           threadhHello.start();
           threadGoodbye.start();
     }
}
```

Heap

100 [MsgPrinter]	200 [MsgPrinter]	300 [Thread]	400 [Thread]
Hello	Goodbye	.	.
System.out	System.out	.	.
		100	200
		.	.

T1 **T2**

Address	Data	Var
9940		
9948		

9956		
9944		
9952		
9960		
9968		
9976		
9984		
9992		
10000		

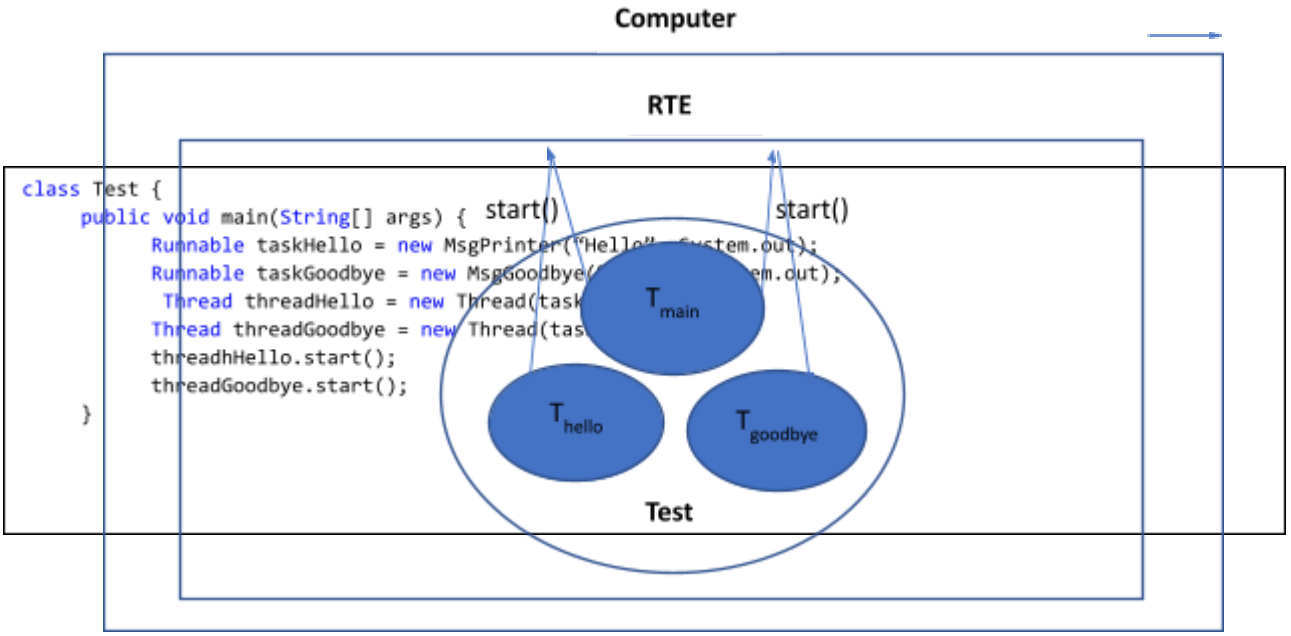
T0

Address	Data	Var	
9940			
9948			
9956			
9944			
9952			
9960			
9968			
9976	400	threadGoodbye	
9984	300	threadHello	
9992	200	taskGoodbye	
10000	100	taskHello	

Address	Data	Var
9940		
9948		
9956		
9944		
9952		
9960		

9968		
9976		
9984		
9992		
10000		

בעקבות הבקשות, סביבת הריצה תייצר בזמנה החופשי שני ת'רדים, האחד עבור המשימה taskHello והשני עבור המשימה taskGoodbye. לכל אחד מהת'רדים תוקצה מחסנית משלו. השדה pc של כל אחד מהת'רדים יצביע לשורה הראשונה במתודת ה main במשימה שלו. מעבר מת'רד אחד לשני על אותו מעבד, יהיה כרוך בהחלפת ה pc, ה sp, ותוכן הרגיסטרים המעבד (נדגים זאת בקרוב).



מה יהיה פלט התוכנית?

ראינו כי בהרצה סדרתית מתקבל פלט אחד: Hello Goodbye

בהרצה מקבילית, לעומת זאת, ייתכנו מספר פלטים, בהתאם למדיניות התזמון של הסביבה:

- הרצת קוד מתודת ה run של הת'רד Hello, הרצת קוד מתודת ה run של הת'רד Goodbye: Hello Goodbye
- הרצת קוד מתודת ה run של הת'רד Hello, הרצת קוד מתודת ה run של הת'רד Goodbye: Goodbye Hello
- הרצת חלק מקוד מתודת ה run של הת'רד Hello, הרצת חלק מקוד מתודת ה run של הת'רד Goodbye:

GHeololdlbyeo, HellGoodobyeo, ...
בהמשך נשאל, מה אנחנו מצפים שיהיה הפלט בסוג כזה של תוכניות, וכיצד ניתן להבטיח תוצאה חוקית של הרצה סדרתית.

<u>פעולות נוספות הניתנות על ידי המחלקה Thread</u>:

מלבד הבקשה מהסביבה להרצת משימה נתונה ע"י ת'רד חדש, בעזרת המתודה start, המחלקה Thread מאפשרת פעולות שכיחות נוספות:

- sleep
  ת'רד הנתון עובר למצב blocked לפרק זמן נתון (כלומר הוא מפסיק לרוץ לפחות לפרק זמן זה). אחרי פרק זמן זה הוא חוזר למצב runnable/ready:

```
threadHello.sleep(1000);
```

- join
  ת'רד הנתון עובר למצב blocked עד אשר ת'רד אחר מסיים משימתו (כלומר את מתודת ה run שלו)

```
threadHello.join();
```

- stop
  הפסקת הריצה של ת'רד הנתון. <u>אסור בתכלית האיסור להשתמש במתודה זו</u>! תחת זו יש להתשמש במנגנון ה interrupt

- interrupt
  הודעה לת'רד הנתון כי מומלץ להפסיק לעבוד. קוד המשימה של ת'רד יגדיר מה הוא עושה במצב כזה (נראה בהמשך כיצד ת'רד שרץ נחשף להודעה זו)

<u>הערה</u>: המקביליות אינה מוחלטת. כלומר, ברגע נתון לא בהכרח כל המשימות רצות:
- בדרך כלל יש יותר ת'רדים ממספר המעבדים
- גם אם יש מעבד נפרד לכל ת'רד, קיימות פעולות אקסלוסיביות הניתנות לביצוע רק על ידי מעבד אחד, כמו קריאה או כתיבה לזיכרון.
המושג מקביליות אומר, שהמשימות אינן רצות בזו אחר זו, אלא בין כל שתי פעולות אטומיות ייתכן מעבר לת'רד אחד.

3.2.3 מתי לעצב תוכנית עם ת'רדים

הרצה מקבילית של משימות עשויה להיות כבדה (כפי שנראה בהמשך), וכן העיצוב של הקוד שלה אינו טריוויאלי כלל ועיקר, הדיבוג של הריצה מסובך לא נריץ תוכנית עם ת'רדים אלא כאשר זה נדרש.
קיימים שלושה תרחישים מרכזיים הדורשים הרצה מקבילית של משימות ע"י ת'רדים:
- תוכנית כבדה, מספר מעבדים זמינים: עיצוב התוכנית כמספר משימות מקבילות הרצות ע"י ת'רדים שונים, ייעל את זמן הריצה.
- תוכנית הניגשת רבות ל i/o: עיצוב התוכנית כמספר משימות הרצות במקביל ע"י ת'רדים שונים יאפשר מעבר למשימה אחרת כאשר אחת המשימות 'נתקעת' (עוברת למצב blocked) כתוצאה מגישה ל i/o.
- תוכנית הנותנת שירות למספר לקוחות: עיצוב הטיפול בכל לקוח כמשימה עצמאית הרצה ע"י ת'רד

יאפשר שירות הוגן ושווה לכל הלקוחות.

מלבד זאת, גם אם נחליט שלא להריץ את התוכנית באופן מקבילי, עדיין עיצוב התוכנית כאוסף משימות מקבילות ככל האפשר הוא ערך עיצובי מהמעלה הראשונה.

3.3 בטיחות ונכונות של תוכנית מקבילית

3.3.1 מבוא / מוטיבציה

ראינו כי הרצה מקבילית של תוכנית המדפיסה Hello ו Goodbye בשתי משימות שונות, עשויה לייצר מגוון רחב של פלטים:

Hello Goodbye
Goodbye Hello
HelGoodlobye
…

מהן התוצאות החוקיות עבורנו? האם ניתן להבחין ברמות שונות של תוצאות שגויות? כיצד ניתן להגדיר זאת פורמאלית?

<u>דוגמא נוספת</u>: תוכנית המגדירה מספר זוגי, ושתי משימות עליו – קידומו ב-2, וקידומו ב-2.

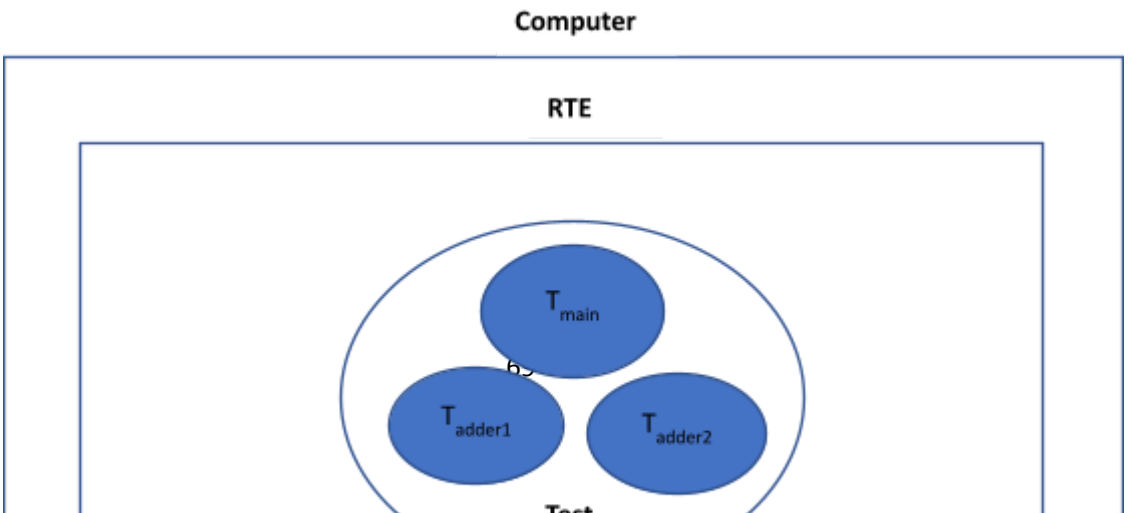
הגדרת מחלקה המייצגת מספר זוגי:

```
class Even {
   int val;
   Even(int val) { this.val = val; }
   public int get() { return val;  }
   public void next() { val++; val++; }
}
```

תוכנית המגדירה מספר זוגי ומקדמת אותו פעמיים באופן מקביל ע"י שני ת'רדים:

```
class Test {
   public static void main(String[] args) {
         Even e = new Even(0);
         new Thread(()-> { e.next; } ).start();
         new Thread(()-> { e.next(); } ).start();
    }
}
```

<u>הרצה</u>:



Heap

100 [Even]	200	300	400
0			

T1 **T2**

Address	Data	Var
9940		
9948		
9956		
9944		
9952		
9960		
9968		
9976		
9984		
9992		
10000		

T0

Address	Data	Var
9940		
9948		
9956		
9944		
9952		
9960		
9968		

9976		
9984		
9992		
10000	100	e

Address	Data	Var
9940		
9948		
9956		
9944		
9952		
9960		
9968		
9976		
9984		
9992		
10000		

```
class Test {
   public static void main(String[] args) {
         Even e = new Even(0);
         new Thread(()-> { e.next; }).start();
         new Thread(()-> { e.next; }).start();
    }
}
```

כמה פעולות אטומיות יש במתודת ה run של כל אחד משני הת'רדים? כלומר, כמה פעולות אטומיות יש במתודה next של המחלקה Even?

val++;

```
read 100,R1
inc R1
write R1, 100
```

כלומר במתודת ה next יש שש פעולות אטומיות:

```
read 100,R1
inc R1
write R1, 100
read 100,R1
inc R1
write R1, 100
```

בין כל שתי פעולות אטומיות, ייתכן מעבר מת'רד לת'רד:

T1

```
read 100,R1
inc R1
write R1, 100
read 100,R1
inc R1
write R1, 100
```

T2

```
read 100,R1
inc R1
write R1, 100
read 100,R1
inc R1
write R1, 100
```

תרחיש/תזמון ראשון:

T1

```
1. read 100,R1          // 0
2. inc R1               // 1
3. write R1, 100        // 1
4. read 100,R1          // 1
5. inc R1               // 2
6. write R1, 100        // 2
```

T2

```
7. read 100,R1          // 2
8. inc R1               // 3
9. write R1, 100        // 3
10. read 100,R1         // 3
11. inc R1              // 4
12. write R1, 100       // 4
```

בתרחיש זה חל המעבר 0->2->4 בערך המספר הזוגי

תרחיש/תזמון שני:

T1

```
1. read 100,R1          // 0
2. inc R1               // 1
3. write R1, 100        // 1
7. read 100,R1          // 2
9. inc R1               // 3
10. write R1, 100       // 3
```

T2

```
4. read 100,R1          // 1
5. inc R1               // 2
6. write R1, 100        // 2
8. read 100,R1          // 2
11. inc R1              // 3
12. write R1, 100       // 3
```

בתרחיש זה חל המעבר 0->3 בערך המספר הזוגי

תרחיש/תזמון שלישי:

T1

```
1. read 100,R1          // 0
3. inc R1               // 1
4. write R1, 100        // 1
7. read 100,R1          // 1
9. inc R1               // 2
10. write R1, 100       // 2
```

T2

```
2. read 100,R1          // 0
5. inc R1               // 1
6. write R1, 100        // 1
8. read 100,R1          // 1
11. inc R1              // 2
12. write R1, 100       // 2
```

בתרחיש זה חל המעבר 0->2 בערך המספר הזוגי

קיבלו שלוש תוצאות שונות: 2,3,4

אלו מהתוצאות נכונות? כנראה רק 4, מדוע, יש להגדיר זאת?

האם ניתן להבחין בין רמות שונות של שגיאה בתוצאות 2,3? מדוע, יש להגדיר זאת?

3.3.2 בטיחות של אובייקטים, הרצה בטוחה של תוכנית

מטרתנו: לאפיין, פורמלית, את הבעייתיות בתרחישים כמו קידום המספר הזוגי 0 ל-3

נגדיר תחילה את המושגית 'תכונה נשמרת', ו'תנאי התחלה וסיום'.

תכונה נשמרת (invariant, 'אינווריאנטה'): בהינתן ממשק, נגדיר את התכונה הנשמרת שלו כאוסף המצבים החוקיים, על פי השאילתות בממשק (getters), בהם יכול להימצא מופע שלו. ניתן לחשוב על התכונה הנשמרת, כמגדירה את המצבים התקניים של האובייקט.

דוגמא: הממשק Even

```
interface Even {
	int get();
	void next();
}
```

ה'מצב' של אובייקט מטיפוס Even מוגדר ע"פ המידע החוזר מהשאילתות שלו, כלומר ע"פ הערך החוזר מהמתודה get().
מבחינה תחבירית, get עשויה להחזיר כל מספר שלם (בתחום של int), כולל מספרים אי-זוגיים. כלומר, אוסף המצבים האפשריים עבור אובייקט מטיפוס Even הוא: {Integer.MIN_VALUE,…, -3, -2, -1, 0, 1, 2, 3,…, Integer.MAX_VALUE}
האינווריאנטה של הממשק Even תצמצם קבוצה זו, לקבוצה קטנה יותר הנכונה מבחינת הסמנטיקה/המשמעות: {Integer.MIN_VALUE,…, ~~-3~~, -2, ~~-1~~, 0, ~~1~~, 2, ~~3~~,…, Integer.MAX_VALUE-1, ~~Integer.MAX_VALUE~~}
כלומר, האינווריאנטה מגדירה את משמעות הממשק (באופן פורמאלי יותר, ממתן שם לממשק המלמד על אופייו, או מטקסט חופשי בהערה או בתיעוד של הממשק).

כיצד נגדיר 'פורמאלית' את האינווריאנטה?
האינווריאנטה תוגדר כתנאי הערכים החוזרים מהשאילתות של הממשק. לדוגמא: get() % 2 == 0
טכנית, נכתוב את תנאי האינווריאנטה כהערה בממשק עם התיוג @INV

```
interface Even {
	// @INV: get() % 2 == 0
	int get();
	void next();
}
```

כל מימוש עתידי של הממשק, חייב לעמוד בקריטריון האינווריאנטה (בדרך כלל יוגדרו טסטים המוודאים זאת – להלן)

ראינו כי התכונה הנשמרת מגדירה את המשמעות של הממשק בכללו (לכל ממשק יש תכונה נשמרת אחת). תנאי התחלה וסיום (Pre/Post Conditions) מגדירים את המשמעות של כל אחת מהמתודות בממשק.

תנאי ההתחלה (Pre-condition) מגדיר את קריטריון הסף לביצוע מתודה נתונה, כתנאי על הערכים החוזרים משאילתות:
- אילוץ על מצב האובייקט עליו מופעלת המתודה, מעבר לאינווריאנטה (לדוגמא, מתודת הנסיעה ברכב

אוטונומי ממקום למקום תדרוש כתנאי התחלה שיש מספיק דלק ברכב)
- אילוץ על הפרמטרים המועברים למתודה (לדוגמא, ניתן לנסוע רק ליעדים בתחום ישראל)

תנאי הסיום (Post-condition) מגדיר את התוצאה של הפעלת המתודה, על פי השאילתות:
- מצב האובייקט בתום ביצוע המתודה (לדוגמא, מתודת המיקום של הרכב בתום הפעלת מתודת הנסיעה ברכב אוטונומי ממקום למקום, צריכה להיות שווה לפרמטר היעד)
- אפיון הערך המוחזר (לדוגמא, חישוב העלות של הרכב נכון לעכשיו, על פי מאפייניו כפי שהם ניתנים בשאילתות)

```
interface Even {

      // @INV: get() % 2 == 0

      // @PRE: none
      // @POST: trivial (simple getter)
      int get();

      // @PRE: none
      // @POST: get() == @PRE(get()) + 2
      void next();
}
```

דוגמא נוספת: הממשק Stack

```
interface Stack<T> {
    void push(T obj);
    T pop();
    boolean isEmpty();
    boolean isFull();
}
```

הגדרת התכונה הנשמרת של המחסנית

אמרנו, כי התכונה הנשמרת היא אילוץ על הערכים החוזרים מהשאילתות. נבחן כל שאילתא בפני עצמה, וכן את השילובים בין השאילתות, כדי לראות האם יש ערכים שאינם תואמים את משמעות המחסנית.

isEmpty – מחסנית יכולה להיות ריקה או לא-ריקה, כך שאין שום אילוץ מעבר לאילוץ התחבירי של החזרת ערך בוליאני true או false (אם היינו רוצים להגדיר מחסנית שאינה ריקה אף פעם, היינו כוללים באינווריאנטה את הדרישה isEmpty() == false)

isFull – כנ"ל, המחסנית יכולה להיות מלאה או לא מליאה, אין כל אילוץ על שאילתא זו מעבר לתחביר.

pop – את המשמעות שהאובייקט המוחזר הוא האחרון שנדחף נגדיר בתנאי הסיום של המתודה. האם קיים אילוץ כללי על סוג האובייקט החוזר (מעבר לכך שהוא T, כפי שמוגדר בתחביר)? לכאורה לא.
האם היא יכולה להחזיר null, או במילים אחרות, האם המחסנית צריכה להכיל רק אובייקטים שאינם null? נניח שלא ניתן לאחסן null במחסנית: pop() != null

isEmpty- isFull - האם המחסנית יכולה להיות גם ריקה וגם מלאה, או במילים אחרות, האם יכולה להיות מחסנית ללא קיבולת? נניח שלא: !(isEmpty() && isFull())

```
interface Stack<T> {
    //@INV: !(isEmpty() && isFull()) && pop() != null

    void push(T obj);
    T pop();
    boolean isEmpty();
    boolean isFull();
}
```

תנאי התחלה וסיום

<u>push</u>

תנאי התחלה
- הפרמטר obj אינו יכול להיות :null =! obj @param null
- המחסנית אינה יכולה להיות מלאה: !isFull()

תנאי סיום
- המחסנית אינה ריקה: !isEmpty()
- תוכן המחסנית שהיה בתחילת המתודה לא השתנה: ??? (אין שאילתות 'טהורות', שאינן משנות את המחסנית, שיכולות לבטא זאת)
- האובייקט שניתן כפרמטר נמצא בראש המחסנית: ??? (לא ניתן לתאר זאת על ידי שאילתא 'טהורה', כלומר, שאילתא שאינה משנה את המחסנית)

<u>pop</u>

תנאי התחלה
- המחסנית אינה יכולה להיות ריקה: !isEmpty()

תנאי סיום
- המחסנית אינה מלאה: !isFull()
- תוכן המחסנית בתחילת המתודה מלבד האיבר האחרון לא השתנה: ??? (אין שאילתות 'טהורות' שיכולות לבטא זאת)
- האובייקט החוזר היה בראש המחסנית (כלומר, האובייקט האחרון שבוצע עליו push (: ??? (לא ניתן לתאר זאת על ידי שאילתא 'טהורה')

עיקרון עיצובי:

א. לעתים יש להרחיב את הממשק הקיים, כדי שניתן יהיה להגדיר את תנאי ההתחלה והסיום והתכונה הנשמרת באופן מדויק יותר.

ב. יצירת תשתית בסיסית להגדרת התנאים השונים
- נדאג להפריד בממשק בין <u>שאילתות</u> שאינן משנות את האובייקט, ובין <u>פקודות</u> שמשנות את האובייקט אך לא מחזירות דבר. כלומר, ברובד הבסיסי של הממשק לא ניתן במתודה אחת גם לשנות האובייקט

וגם להחזיר משהו (כמו המתודה pop כרגע)
- נדאג להבחין בין שאילתות ופקודות בסיסיות, ובין שאילתות ופקודות נגזרות, כלומר כאלה שהן בסה"כ יישום ספציפי וטריוויאלי של שאילתות/פקודות בסיס.

בפרט עבור הממשק Stack:

1. נוסיף שלוש מתודות

```
T itemAt(int i);
int size();
int capacity();
```

2. הגדרת מתודות עבור התשתית הבסיסית
   - הפרדה בין שאילתות לפקודות
   במקום/לצד pop נגדיר שאילתה ופקודה

```
T top();
void remove();
```

אבחנה בין שאילתות/פקודות בסיס לשאילתות/פקודות נגזרות

המתודות הבאות נגזרות ממתודות בסיסיות (כלומר ניתנות למימוש על ידי שימוש פשוט בהן):

```
top() □ itemAt(size()-1)
 pop() □ top() && remove()
 isEmpty() □ size() == 0
 isFull() □ size() == capacity()
```

בשורה התחתונה: נגדיר את האינווריאנטה ואת תנאי ההתחלה והסיום של הפקודות הבסיסיות, על פי השאילתות הבסיסיות.

```
interface Stack<T> {
    //@INV: 0 >= size() <= capacity() &&
    //      Vi < size() -1, itemAt(i) != null

   T itemAt(int i);
    int size();
    int capacity();
    // @PRE: size() > 0
    // @POST: size() == @PRE(size()) – 1 &&
                        Vi < size() -1, itemAt(i) == @PRE(itemAt(i))
    void remove() throws Exception;
    // @PRE: @param obj != null && size() < capacity()
    // @POST: size() == @PRE(size()) +1 && itemAt(size()-1) == @param obj
&&
    //        Vi < size(), itemAt(i) == @PRE(itemAt(i))
```

```
    void push(T obj) throws Exception;

    T top();
    boolean isEmpty();
    boolean isFull();
    T  pop();
}
```

הגדרת בטיחות (Safety)

נאמר כי אובייקט נתון הוא בטוח, אם האינווריאנטה של כל אחד מהממשקים שהוא מממש מתקיימת.
נאמר כי הרצת תוכנית הינה בטוחה, אם כל האובייקטים בתוכנית בטוחים במהלך ריצתה.
 הריצה שהגיעה למספר זוגי בעל ערך 3 אינה בטוחה, אך הריצה שהגיעה לאובייקט זוגי בעל ערך 2 כן בטוחה.
האם הריצה בה קיבלנו HellGoodboye בטוחה? מאחר שלא הגדרנו תכונה נשמרת עבור ערוץ הפלט (כמו לדוגמא, שהוא חייב להכיל מילים מהמילון האנגלי) התוכנית בטוחה בהגדרה.

3.3.3 נכונות של ריצה מקבילית

כיצד אם כן נגדיר את הבעייתיות של התוצאות 'HellGoodboye ', ו'2' בשתי התוכניות?

הגדרת נכונות (correctness)

מושג הנכונות נועד לאפיין את נכונות הרצה של תוכנית, מֵעבר לתקינות של האובייקטים השונים בה.

עבור הרצה סדרתית: נגדיר הרצה כנכונה, אם כל ביצוע של מתודה מקיים את תנאי הסיום שהוגדרו עבורה. כלומר, כל הפעולות שבוצעו בקוד היו תקינות.

עבור הרצה מקבילית, הגדרה זו אינה מספקת. לדוגמא: ביצוע מקבילי של next על מספר זוגי משותף. בתרחיש השלישי שתואר למעלה, הערך של getVal() התחילת המתודה next(), עבור כל אחד מהת'רדים, היה 0, והערך שלו הסיום, עבור כל אחד מהת'רדים היה 2. כלומר, תנאי הסיום של המתודה התקיים עבור כל אחד מהת'רדים, אך התוכנית בכללה לא היתה נכונה – הגענו ל-2 במקום ל-4.

לכן נגדיר את נכונות הרצה מקבילית באופן הבא:

נגדיר מחלקה נתונה כנכונה, אם הרצה מקבילית של כל קומבינציה של מתודה אחת שלה מייצרת פלט או מצב אובייקט שיכול להתקבל בהרצה סדרתית בכל סדר של ביצוע המתודות.

דוגמאות:

- Hello Goodbye
  מתודה א: ביצוע run עבור אובייקט MessagePrinter עם שדה Hello
  מתודה ב: ביצוע run עבור אובייקט MessagePrinter עם שדה Goodbye
  הרצה סדרתית בסדר מתודות א-ב: Hello Goodbye
  הרצה סדרתית בסדר מתודות ב-א: Goodbye Hello

- אוסף התוצאות החוקיות: {Hello Goodbye, Goodbye Hello}
- ההרצה המקבילית שהגיעה ל HellGoodboye אינה נכונה, כי התוצאה אינה נכללת בקבוצת התוצאות החוקיות.

- Even

מתודה א: ביצוע next, קידום מספר זוגי ב-2
מתודה ב: ביצוע next, קידום מספר זוגי ב-2
הרצה סדרתית בסדר מתודות א-ב: 4
הרצה סדרתית בסדר מתודות ב-א: 4

- אוסף התוצאות החוקיות: {4}
- ההרצה המקבילית שהגיע ל 2 אינה נכונה, כי התוצאה אינה נכללת בקבוצת התוצאות החוקיות.

3.4 שמירה על בטיחות ונכונות

מטרתנו בסעיף זה לבחון שיטות שונות להבטחת הקיום של הבטיחות והנכונות בריצה מקבילית של אוסף משימות.
קטגורית, ניתן לחשוב על גישות מרכזיות שונות (נפגוש במימושים מגוונים שלהם במהלך הפרק):

- מניעת מצב של זיכרון משותף
  - נדאג לכך שאובייקט ייחשף רק בפני ת'רד אחד. כלומר, כל ת'רד עובד 'מודולארית' עם אובייקטים שונים.
  - נדאג לכך שכל ת'רד ייחשף לחלק אחר של אותו אובייקט (כל ת'רד ניגש לשדות אחרים של האובייקט)
  - ייצור עותקים שונים של האובייקט לכל ת'רד תוך מיזוג התוצאות בסוף
- סינכרון הגישה לאובייקט ע"י הת'רדים השונים.
- ניתן לת'רדים לעבוד במשותף על האובייקט כרצונם, ואם תתרחש תקלה (בעיית בטיחות או נכונות) נזהה זאת וננסה שוב.

3.4.1 אובייקטים שאינם ברי שינוי (Immutable Objects)

בשפות תכנות, כידוע, ניתן לבצע השמה על משתנים.

```
int i=5;
…
i++;

Student s = new Student(…);
…
s.setNickName(“jango”);
```

מסתבר כי פעולת השמה שכזו אינה הכרחית בשפת התכנות, כלומר ניתן לממש כל אלגוריתם בשפת תכנות ללא פעולת השמה (נראה זאת בקורס 'עקרונות שפות תכנות').
ניתן לעצב מחלקות שאין בהן פעולת השמה. במחלקות אלו אין בעיות בטיחות בהגדרה (בהנחה שהאינווריאנטה נבדקה בבנאי), כך שניתן לחשוף את האובייקט בפני ת'רדים שונים ללא מנגנון הגנה מיוחד.

עיצוב

- פקודות: **במקום לבצע השמה נחזיר עותק חדש מעודכן**
- שאילתות: כל השדות קבועים ופרטיים, הגישה אליהם ניתנת רק דרך שאילתות. אין חשיפה בשאילתות של שדות פנימיים מטיפוס אובייקט אם הם אינם מעוצבים גם כן כ Immutable Objects (כי אז המשתמש מקבל גישה לשדה ויכול לשנות אותו)
- לא ניתן לרשת את המחלקה (=היא מוגדרת כ final)

דוגמא:

```
final class Even {

      private final int val;

      Even(int val) throws Exception {
            if (val % 2 != 0)
                  throw new Exception("Odd number!");
            this.val = val;
     }

     public int get() { return val; }

     public Even next() {
         return new Even(val+2);
     }
}
```

שימוש

בדרך כלל, כאשר משתמשים באובייקטים כאלה, נדרש לשם שמירה על הנכונות (הבטיחות מתקיימת בכל מקרה):x

- השמת העותק החדש למשתנה בכל קריאה לפקודה (רלבנטי גם לריצה סדרתית) כמו במחלקה String, לדוגמא עבור הפעלת הפקודה replaceAll:

```
String s = "abc";
s.replaceAll("b","d");
System.out.print(s);   // abc
s = s.replaceAll("b","d");
System.out.print(s);   // adc
```

- מיזוג של התוצאות בסוף (עבור ריצה מקבילית)

דוגמא:

```
class Adder implements Runnable {
     Even e;
     Adder(Even e) { this.e = e; }
     public Even getE() { return e; }
```

```
    public void run() { e = e.next(); }
}
class Test {
   public static void main(String[] args) {
         Even e = new Even(0);
         Adder adder1 = new Adder(e);
         Adder adder2 = new Adder(e);
         Thread t1 = new Thread(adder1);
         Thread t2 = new Thread(adder2);
         t1.start();
         t2.start();
         // Merge copies:
         // 1. Wait for completion
        t1.join();
        t2.join();
         // 2. Merge
         e = new Even(adder1.getE().get() + adder2.getE().get() –
e.get());
    }
}
```

;

100 [Even]	200 [Adder]	300 [Adder]	400]Even]	500 [Even]	600 [Even]	
0	400	500	2	2	4	

מתי כדאי להשתמש בסוג כזה של אובייקטים?
אובייקטים שקל לשכפל אותם וקל למזג אותם.
לדוגמא: מספרים, מחרוזות, צבעים,...
בנימה פילוסופית, סוג כזה של טיפוסים מכונה Value Objects, כלומר אובייקטים שזהותם מוגדרת על ידי הערך שלהם ולא על פי המקום שלהם בזיכרון.

```
Integer i1 = new Integer(6);
Integer i2 = new Integer(6);
if (i1 == i2) ... // true
```

מסבר שקיימות כמה וכמה מחלקות כאלה ב Java: String, Integer, Long, Double, Boolean,...

3.4.2 סנכרון הגישה לאובייקטים

נדאג לכך ששני ת'רדים לא יבצעו על אותו אובייקט פעילות 'קריטית' באותו זמן.

3.4.2.1 סנכרון ע"י lock

בגישה זו הת'רד הראשון ניגש לאובייקט בעוד הת'רדים הבאים אחריו עוברים למצב blocked.

לשם כך נדרש שירות של סביבת הריצה: פעולות נעילה ושחרור, העברת ת'רדים למצב blocked ו runnable בהתאם.

lock(O): הת'רד הנוכחי מנסה לנעול את האובייקט O, אם האובייקט O אינו נעול עדיין הוא ייתפס ע"י הת'רד, אחרת הת'רד עובר למצב blocked (עד אשר האובייקט ישוחרר על ידי הת'רד שתופס אותו כרגע)

unlock(O): הת'רד הנוכחי משחרר את הנעילה שלו על האובייקט O, ובעקבות כך הסביבה תחזיר את כל הת'רדים שעברו למצב blocked, כתוצאה מניסיון תפיסה של O, למצב runnable.

למימוש פעולות אלו, מוגדר לכל אובייקט מבנה נתונים, הכולל שדה (מטיפוס Thread) המציין איזה ת'רד נועל אותו כרגע, ורשימה (של Thread) המציינת אלו ת'רדים ניסו לנעול את האובייקט התפוס ועברו עקב כך למצב blocked.

ב Java קיים תחביר נוח, כחלק מהשפה, לשתי פעולות אלו:

```
synchronized (o) {
    …
}
```

```
lock(o)
…
unlock(o)
```

כיצד נשתמש במנגנון זה עבור מחלקה נתונה, כך שתובטח הבטיחות והנכונות במחלקה זו, גם כאשר המופעים שלה חשופים בפני כמה ת'רדים (כלומר, כיצד להפוך אותה ל 'Thread Safe')?

<u>פרוטוקול א</u>: סנכרון מלא

כל גישה לשדות של המחלקה (קריאה או כתיבה) תֵּעָשֶׂה תחת נעילה של האובייקט.

דוגמא:

```
class Even {

      private int val;

      Even(int val) throws Exception {
            if (val % 2 != 0)
                  throw new Exception(“Odd number!”);
            this.val = val;
     }

     public int get() {
          synchronized (this) {
                return val;
           }
```

```
    }

    public void next() {
       synchronized (this) {
           val++;
           val++;
       }
    }
   public void f() { System.out.println("abc"); }
}
```

T1: e.next()
T2: e.next()
T3: e.next()
e: <T3 , [T2] > 3

T1: e.next()
T2: e.get()

e: < T2, [] > 4

מאחר שמדובר בפרוטוקול סטנדרטי ושכיח, קיים עבורו תחביר נוח (syntactic sugar) ב Java:

```
public int get() {
          synchronized (this) {
                return val;
           }
 }

public synchronized int get() {
     return val;
 }

public void next() {
   synchronized (this) {
           val++;
           val++;
       }
    }

public synchronized void next() {
      val++;
      val++;
}
```

[ה synchronized אינו חלק מהחתימה, אלא חלק מקוד המתודה. לדוגמא, לא ניתן להגדיר מתודה 'מסונכרנת' בממשק, צריך להגדיר את המתודה כמסונכרנת אם דורסים אותה במחלקת הבן]

הערה: במקרים שונים, ניתן אולי להימנע מנעילה של כל בלוק המתודה, כך שרק כמה שורות ממנה יהיו נתונות תחת בלוק הסנכרון – נראה דוגמאות לכך בהמשך הקורס. בכל מקרה צריך להיזהר עם / לדעת מה עושים.

פרוטוקול ב: סנכרון חלקי

אם ניתן לחלק את המתודות בממשק לקבוצות שונות וזרות, כך שמתודה בקבוצה אחת אינה ניגשת לשדות של המתודות בקבוצה אחרת, ניתן לסנכרן כל קבוצה של מתודות על אובייקט המייצג קבוצה זו.

דוגמא:

```
class Point {
     int x,y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

     pubic void getY() {
         synchronized(this) {
            return y;
          }
     }
     pubic void up() {
         synchronized(this) {
            y++;
          }
     }
     pubic void down() {
          synchronized(this) {
              y--;
          }
     }
     pubic void getX() {
         synchronized(this) {
            return x;
          }
     }
     pubic void right() {
         synchronized(this) {
            x++;
         }
     }
     pubic void left() {
         synchronized (this) {
             x--;
         }
     }
}
```

```
class Point {
     int x,y;
     Object lockX, lockY;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
        lockX = new Object();
        lockY = new Object();
    }

     pubic void getY() {
         synchronized(lockY) {
            return y;
          }
     }
     pubic void up() {
         synchronized(lockY) {
            y++;
          }
     }
     pubic void down() {
          synchronized(lockY) {
              y--;
          }
     }
     pubic void getX() {
         synchronized(lockX) {
            return x;
          }
     }
     pubic void right() {
         synchronized(lockX) {
            x++;
         }
     }
     pubic void left() {
         synchronized (lockX) {
             x--;
         }
     }
}
```

T1: p.up()
T2: p.left()
T3: p.right()

p: <_, []> 0,1
lockX: <_,[]>
lockY: <_,[]>

<u>פרוטוקול ג</u>: נעילה של מספר אובייקטים בפעולה אחת

נניח כי אנו מעוניינים לנעול שני אובייקטים, לדוגמא (עשוי להיות בתרגיל): הסרת איבר מרשימה אחת רק אם ניתן להוסיפה לרשימה אחרת, אם זה לא אפשרי יש להחזירו למקומו הקודם ברשימה הראשונה.
ניתן לתפוס את שתי הרשימות כאחד, ע"י נעילה של אובייקט המייצג את שתי הרשימות (כך שכל הגישות לרשימות ייעשו עם סנכרון של אובייקט זה)

```
List<Integer> l1, l2;
Object lockL1L2;
…
synchronized (lockL1L2) {
  … // do something with both L1 and L2
}
```

3.4.2.2 השלכות נוספות של מנגנון הסנכרון

ראינו כי השימוש ב synchronized מאפשר לבצע קטע קוד מבלי שת'רדים אחרים ייגשו לאובייקט בזמן זה.
מסתבר כי יש השלכה נוספת לשימוש במנגנון זה:

Visibility

כזכור, כל אחד מהמעבדים מתחזק זיכרון מטמון (cache memory), כך שהגישה לתאי זיכרון 'פופולריים' אינה מתבצעת בהכרח במקום שלהם ב RAM.
 כאשר רצים ת'רדים על מעבדים שונים, ייתכן שהם קוראים תוכן שונה של אותו תא בזיכרון, כל אחד מזיכרון המטמון של המעבד שלו, למרות שעל פי מודל המקביליות ה HEAP משותף לכל הת'רדים.

דוגמא:

```
class Test {
   public static void main(String[] args) {
         Even e = new Even(0);
         Adder adder1 = new Adder(e);
         Adder adder2 = new Adder(e);
         Thread t1 = new Thread(()->e.next());
         Thread t2 = new Thread(()->e.next());
         t1.start();
         t2.start();
         t1.join();
```

```
        t2.join();
        System.out.println(e.get());
    }
}
```

נניח כי התוכנית רצה באופן תקין. כלומר כל אחד מהת'רדים הגדיל בתורו, בזה אחר זה, את המספר ב-2, מבלי להפריע אחד לשני.
ייתכן עדיין שהתוכנית תדפיס 0! במקרה שבו המעבד של הת'רד הראשי קורא את הערך של e מה cache memory שלו, כאשר שני הת'רדים האחרים רצו על מעבד אחר. כלומר, המספר ב Heap הוא 4, אך המעבד של הת'רד הראשי לא ניגש לשם.

ביצוע **synchronized** בקוד, גורר עדכון של זיכרון המטמון של כל המעבדים עם התוכן של ה RAM.

מה נעשה אם בחרנו לשמור על הבטיחות והנכונות שלא ע"י מנגנון ה synchronized?
 הגדרת המשתנה כ volatile תכריח את המעבד לקרוא/לכתוב תמיד את הערך שלו מה RAM ולא מזיכרון המטמון.

```
class Test {
   public static void main(String[] args) {
        volatile Even e = new Even(0);
        Thread t1 = new Thread(()->e.next());
        Thread t2 = new Thread(()->e.next());
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println(e.get());
}
```

<u>הערה</u>: הגדרת משתנה כ volatile תגרום גם לקריאה וכתיבה שלו באופן אטומי. כלומר, אם הוא מתפרש על יותר מתא אחד בזיכרון, ת'רדים אחרים לא יוכלו לגשת אליו בזמן הקריאה/הכתיבה של ת'רד אחר.

Code Re-ordering]

מסתבר שהקומפיילר נוטה לעתים לשנות את סדר שורות הקוד בתוכנית.
לדוגמא:

```
int i=10;
int j=20;
i++;
j++;
```

```
int i=10;
int j=20;
```

j++;
i++;

הסיבה: לשם אופטימיזציה

write [i],10
write [j],20
read R1,[i]
inc R1
write [i], R1
read R1,[j]
inc R1
write [j], R1

□

write [i],10
write [j],20
~~read R1,[j]~~
inc R1
write [j], R1
read R1,[i]
inc R1
write [i], R1

אם קיימות בין השורות פעולות שיכולות להיות מושפעות משינוי הסדר, הקומפיילר לא יגע בסדר הנתון.
פעולות כמו: if, קריאה לפרוצדורה, ..., synchronized]

3.4.2.3 תבניות עיצוב

1. הוספת מתודה למחלקה 'בטוחה-נכונה' (ששומרת על הבטיחות והנכונות)

נתונה המחלקה Vector ב Java, המוצהרת כ 'Thread-safe'. במילים אחרות, ניתן לחשוף מופעים של מחלקה זו ללא חשש בפני כמה ת'רדים.
מסתבר, כי היות המחלקה 'בטוחה-נכונה' אינו מאפשר כל שימוש שהוא במחלקה תחת מספר ת'רדים.
דוגמא:

```
Vector<Integer> v;
... // some code, including thread creation over v
if (!v.contains(5))
    v.add(5);
```

בטיחות-נכונות המחלקה מבטיח כי ביצוע כל אחת מהמתודות תקין, אך אין כל ביטחון כי בין הבדיקה האם 5 מאוחסן בווקטור לבין פעולת הוספתו, לא הוסיף ת'רד אחר 5 לאותו ווקטור.
 נדאג לכך שת'רד אחר לא יוכל לגשת לווקטור בין שתי שורות קוד אלו:

```
Vector<Integer> v;
... // some code, including thread creation over v
synchronized (v) {
    if (!v.contains(5))
        v.add(5);
}
```

סנכרון שכזה ע"י משתמשי המחלקה מכונה client-side locking

במידה ומדובר בפעולה שכיחה, כמו תמיד נעדיף להרחיב את המחלקה עם מתודה נוספת המטפלת בשימוש שכיח זה:

```
class MyVector<T> extends Vector<T> {
      public synchronized void addIfAbsent(T obj) {
             if (!contains(obj))
                  add(obj);
      }
}

MyVector<Integer> v;
... // some code, including thread creation over v
v.addIfAbsent (5);
```

<u>בעיה</u>: מניין לנו שבטיחות-נכונות המחלקה מומשה בכלל ע"י סנכרון ת'רדים (יש שיטות אחרות)? וגם אם נתון שכן, מי אמר שהסנכרון מומש ע"י נעילה (קיימת דרך אחרת מלבד synchronized, נראה בהמשך)? גם אם נתון שהמחלקה מסונכרנת ע"י נעילה בעזרת synchronized, מי אמר שאובייקט הנעילה הוא בהכרח this?
 נדאג לכך בעצמנו שכל המתודות יסונכרנו ע"י נעילה של this:

```
class MyVector<T> extends Vector<T> {
      public synchronized void addIfAbsent(T obj) {
             if (!contains(5))
                  add(5);
      }

     public synchronized boolean contains(T obj) {
             return super.contains(obj);
     }

     public synchronized void add(T obj) {
             super.add(obj);
     }
```

```
    ... // same for all methods of Vector

}
```

בעיה: אם נשכח לדרוס את אחת המתודות, או אם בעתיד יתווספו (בגרסאות הבאות של JDK) מתודות למחלקה Vector, אז התוכנית תתקמפל. כלומר אחת המתודות תישאר מסונכרנת על אובייקט אחר מ this.

- במקום ירושה, נממש את הממשקים של Vector בעזרת ווקטור פנימי (Delegation, Composition)

```
class MyVector<T> implements List<T>, Iterable<T>...  {

      Vector<T> v;

      public MyVector() { v = new Vector(); }

      public synchronized void addIfAbsent(T obj) {
            if (!v.contains(5))
                  v.add(5);
      }

     public synchronized boolean contains(T obj) {
            return v.contains(obj)l
     }

     public synchronized void add(T obj) {
            v.add(obj)l
     }

     ... // same for all methods of Vector

}
```

התובנות מתבנית זו:
- גם כאשר יש מחלקה Thread-Safe לא ניתן לכתוב קוד 'כרגיל' (כמו בעיצוב סדרתי). בפרט, יש לסנכרן פעולות על האובייקט התלויות אחת בשניה (כמו if-else)
- אם מנגנון הבטיחות-נכונות של המחלקה אינו ידוע במדויק, יש לדאוג שההרחבות שלנו למחלקה או השימוש השימוש שלנו בה יעבדו במנגנון אחיד.

2. מעבר על מבנה נתונים

נתונה מתודה המבצעת פעולה כלשהי על האיברים במחלקה המממשת מבנה נתונים, כאשר התערבות של ת'רדים אחרים במופע זה עשויים לחבל בנכונות המתודה (בקיום תנאי הסיום שלה). לדוגמא [לשם פשטות, נניח ש Vector המקורית מסונכרנת על this]:

```
class MyVector<T> extends Vector<T> {
	public void printAll() {
		Iterator<T> it = iterator();
		while (it.hasNext())
			System.out.println(it.next());
	}
}
```

ייתכן שבין הבדיקה hasNext לביצוע של next, ת'רד אחר הסיר את האיבר מהווקטור.

דוגמא אחרת: אלגוריתם בנקאי לחישוב שערי הריבית למחר, על בסיס נתוני חשבונות הלקוחות, כאשר ייתכן שתבוצע חלקית העברה של כסף מחשבון לחשבון תוך על ידי ת'רד אחר (כך שסכום זה ייספר פעמיים).

- פתרון א: נסנכרן את הפעולה, כלומר ננעל את האובייקט לכל משך ביצוע הפועלה

```
class MyVector<T> extends Vector<T> {
	public synchronized void printAll() {
		Iterator<T> it = iterator();
		while (it.hasNext())
			System.out.println(it.next());
	}
}
```

חיסרון: נעילה לפרק זמן ממושך של האובייקט, כלומר מניעת גישה לכל האובייקטים במבנה (כמו מניעת ברור יתרה במשך הלילה ע"י לקוחות הבנק).

- פתרון ב: נבצע את הפעולה על עותק של המבנה (Snapshot)

נשכפל, תחת סנכרון, את המבנה, ונבצע את הפעולה על העותק ללא סנכרון (אף ת'רד אחר אינו מכיר אותו). ההנחה היא, שזמן שכפול האובייקט קטן לאין ערוך, מזמן ביצוע הפעולה.

```
class MyVector<T> extends Vector<T> {

	public synchronized void printAll() {
		MyVector<T> copy = clone();
		Iterator<T> it = copy.iterator();
		while (it.hasNext())
			System.out.println(it.next());
	}

	public synchronized MyVector<T> clone() {
		MyVector<T> copy = new MyVector();
```

```
            Iterator<T> it = iterator();
            while (it.hasNext())
                  copy.add(it.next().clone());
           return copy;
      }
}
```

חיסרון: עדיין יש שימוש במשאבי זיכרון (אחסון העותק) וזמן (עבור ההעתקה תחת סנכרון). ובנוסף, אם מדובר בפעולה המשנה את האובייקט, נדרש מיזוג בסוף – יכול להיות מסובך.

 פתרון ג: Optimistic Try & Fail

נבצע את המעבר על איברי המבנה מבלי לסנכרן אותו, אך נשתול מנגנון המזהה שינויים שנעשו על ידי ת'רדים אחרים בזמן המעבר, כך שייזרק Exception והמשתמשת תוכל לנסות שוב.
גישה זו מתאימה למקרים שבהם רוב הת'רדים רק קוראים מידע ולא משנים אותו.

עבור MyVector:
- מנגנון המזהה שינויים באובייקט וזורק חריגה.
 * נוסיף שדה למחלקה המציין את ה'גרסה' של הווקטור, כאשר כל מתודה המשנה את הווקטור תגדיל את הגרסה שלו.
 * נממש את האיטרטור, כך שכאשר הגרסה שהיתה בזמן יצירת האיטרטור משתנית, במתודת ה next ייזרק Exception
- תבנית של תפיסת חריגה וניסיון חוזר
 בתפיסת החריגה נבצע את השינוי החוזר.

```
class MyVector<T> extends Vector<T> {

     int version;

     MyVector() { … version = 0; }

     public synchronized int getVersion() { return version; }

      public synchronized void printAll() {
            try {
                Iterator<T> it = iterator();
                while (it.hasNext())
                     System.out.println(it.next());
             } catch (ConcurrentModificationException e) {
                   printAll();
             }
      }

      public synchronized void add(T obj) {
            super.add(obj);
            version++;
      }
```

```
        public synchronized void remove() {
              super.remove(obj);
              version++;
        }

        // same for all commands (excluding queries)

       public Iterator<T> iterator() {
              return new Iterator() {
                     int final origVersion = getVersion();
                     int index = 0;

                    public T next() throws Exception {
                         synchronized(MyVector.this) {
                             if (getVersion() != origVersion)
                                 throw new
ConcurrentModificationException();
                                 return get(index++);
                          }
                    }

                    // implementation of hasNext, remove
              }
       }
}
```

הערה: ניתן להכליל את המתודה printAll למתודה כללית המבצעת פעולה כשלהי, נתונה, על איברי המערך (לאו דווקא הדפסה)

```
interface Procedure<T> {
     void apply(T obj);
}

class MyVector<T> extends Vector<T> {
       …
       public void applyAll(Procedure<T> action) {
              try {
                  Iterator<T> it = iterator();
                  while (it.hasNext())
                       action.apply(it.next());
              } catch (ConcurrentModificationException e) {
                    applyAll();
              }
       }
}

MyVector<Integer> v;
…
v.applyAll((Integer i) -> System.out.println(i));
```

3.4.2.4 סנכרון lock-free

מסתבר כי למימוש הסנכרון ע"י נעילה (=ע"י synchronized) עשוי להיות חיסרון: כאשר בלוק הקוד המסונכרן קצר (מה שקורה כלל), הזמן שהולך על כל המעבר ל blocked ובחזרה ל runnable ארוך יותר מזמן ההמתנה שהאובייקט ישתחרר.
 במקום לעבור למצב blocked הת'רד הממתין ירוץ על לולאה חסרת תכלית עד שהת'רד השני יסיים את פעולתו הקצרה.

לשם כך, נשתמש בפעולה מורכבת בזיכרון של המעבד המכונה CompareAndSet (עד כה השתמשנו רק בשתי פעולות על הזיכרון: READ, WRITE).
פעולה זו מבצעת, אטומית, שלושה דברים: קוראת ערך של תא בזיכרון, משווה אותו לערך אחר, ואם הם שווים כותבת ערך חדש לתא זה. עבור ערך מסוג int, משהו כמו (ממומש כפעולה אחת בחומרה):

```
boolean cas(int* address, int oldVal, int newVal) {
      if ((*address) == oldVal) {
            (*address) = newVal;
            return true;
      } else
             return false;
}
```

בספריות החדשות של Java קיימות מספר מחלקות המאפשרות לקרוא לפעולה זו במעבד, עבור טיפוסים שונים של ערכים:

AtomicInteger, AtomicBoolean,…, AtomicReference<T>

ניתן לממש בעזרת פעולה זו, מנגנון סנכרון אלטרנטיבי, שאינו מעביר את הת'רדים הממתינים למצב blocked, עבור גישה למידע מטיפוסים שונים.

התבנית הכללית לביצוע שינויים בערך משותף val (המוגדר כ Atomic מהסוג של הטיפוס שלו **T**):

- נתון משתנה/שדה משותף מטיפוס **T**
- נעטוף אותו בממשק ה Atomic, כלומר הוא היה מטיפוס **T**Atomic (באופן זה הוא גם יוגדר כ volatile)
- השאילתות עבור משתנה/שדה זה יבוססו עם מתודת ה get של מחלקת ה Atomic
- התבנית של כל אחת מהפקודות במחלקה המשנות משתנה/שדה זה:

```
T oldVal;
T newVal;
do {
     oldVal = val.get();
     newVal = ____ oldVal ____;
} while (!val.compreAndSet(oldVal,newVal));
```

ניישם תבנית זו עבור סנכרון lock-free של המחלקה Even:

```
class Even {

   int val;

   Even(int val)  throws Exception {
        if (val % 2 != 0)
            throw new Exception(…);
        this.val = val;
   }
   public int get() { return val; }
   public void next() {
        val+=2;
    }
}

class Even {

   AtomicInteger val;

   Even(int val)  throws Exception {
        if (val % 2 != 0)
            throw new Exception(…);
        this.val = new AtomicInteger(val);
   }
   public int get() { return val.get(); }
   public void next() {
        int oldVal;
        int newVal;
        do {
            oldVal = val.get();
            newVal = oldVal + 2;
         } while (!val.cas(oldVal,newVal));
   }
}
```

e.val: 4

T1: e.next() oldVal=0, newVal=2
T2: e.next() oldVal=0, newVal=2

<u>דוגמא נוספת</u>: המחלקה LinkedList

```
class LinkedList<T> implements List<T> {
       Link<T> head = null;
       …
       public void add(T obj) {
              Link<T>  newLink = new Link<T>(obj,head);
```

```
                head = newLink;
        }
}
```

lst.head: 2->null

T1: lst.add(1), newLink = 1->null
T2: lst.add(2), newLink = 2->null

Correctness: { 1->2->null , 2->1->null }

□

```
class LinkedList<T> implements List<T> {
        AtomicReference<Link<T>> head = new AtomicReference<Link<T>>(null);
        …
        public void add(T obj) {
                Link<T> oldVal;
                Link<T> newVal;
                do {
                    oldVal = head.get();
                    newVal = new Link<T>(obj,oldVal);
                } while (!head.cas(oldVal,newVal));
        }
}
```

lst.head: 1->2->null

T1: lst.add(1), oldVal = 2->null, newVal = 1->2->null
T2: lst.add(2), oldVal = null, newVal = 2->null

Correctness: { 1->2->null , 2->1->null }

לא תמיד כל כך פשוט לממש את מנגנון הסנכרון הזה למתודות שונות במחלקה נתונה...
קיימים ב Java מימושים כאלה למבני נתונים קלאסיים: ConcurrentHashMap,
...,ConcurrentLinkedQueue

<u>דיון</u>: ניתן לעצב תבנית סנכרון כללית lock-free המתאימה לכל קטע קוד באשר הוא:

נגדיר שדה במחלקה מטיפוס AtomicBoolean:

```
AtomicBoolean lock = new AtomicBoolean(false);
```

נסנכרן את הקוד של כל מתודה באופן הבא:

```
while (!lock.cas(false,true));   // instead of ‘synchronized(this) {‘  /
lock
… // code
lock.set(false);   // instead of ‘}’ / unlock
```

מה ההבדל בין לבין התבנית שהצענו למעלה?

בתבנית הנוכחית, אם אחד הת'רדים עובר את 'המחסום', שורת ה while, שום ת'רד לא יכול לעבור אותה יותר עד אשר הת'רד הזה יסיים את קטע הקוד ויבצע set. בפרט, אם ייגמר לו זמן המעבד עוד לפני שהוא הגיע לקוד והוא יעבור למצב runnable או blocked כאשר הת'רדים האחרים לא יכולים להתקדם.
בתבנית הקודמת, הת'רדים לא נתקעים ב busy-wait על שורה מסויימת אלא מבצעים שוב ושוב את הלולאה, עד אשר יהיו הראשונים לבצע את ה cas. אם לאחד מהם נגמר זמן המעבד והוא עובר למצב runnable, שאר הת'רדים יכולים להמשיך לנסות, ואחד מהם יצליח.

המונח 'סנכרון lock free' אומר:
- אין מעבר של הת'רדים הממתינים למצב blocked
- אין חסימה של ת'רדים אחרים על שורה מסוימת עד אשר הת'רד הראשון יסיים את קטע הקוד המסונכרן.

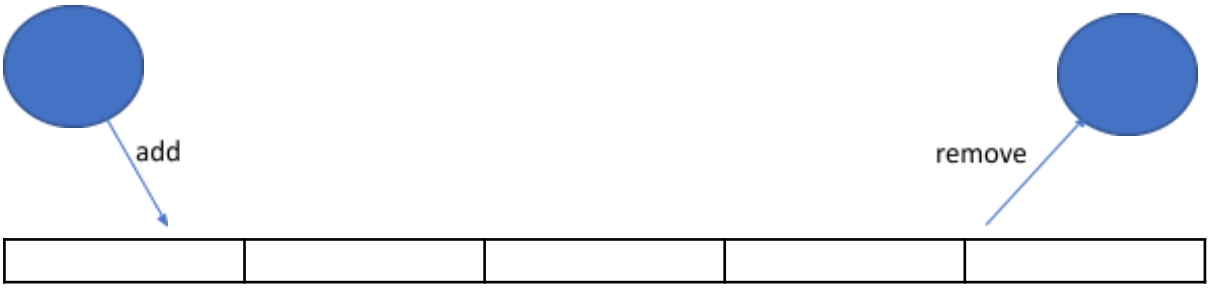
3.5 מנגנוני המתנה

פעולות בתוכנית עשויות להיכשל (ולזרוק Exception) בגלל מגוון סיבות. לדוגמא, בשל אי קיום תנאי ההתחלה.

דרכי התמודדות עם כישלון הפעולה (מה ממומש בבלוק של ה catch):

- עצירת התוכנית (System.exit)
- המשך כרגיל (בלוק ריק)
- העברת האחריות ל'מזמין' הפעולה (אי תפיסה של ה Exception או זריקה מחודשת שלו)
- ניסיון חוזר

בסעיף זה נתמקד באפשרות האחרונה (פגשנו בה כבר ב printAll בגישת Optimistic Try & Fail).
נשים לב כי געשה זו רלבנטית רק לתוכנית מקבילית (אם מנסים לקרוא מתור והתור ריק: עבור תוכנית סדרתית, אין מה לנסות שוב עוד 3 שניות כי אין ת'רד אחר המוסיף איברים לתור, אך עבור תוכנית מקבילית כן)




```
class BoundedQueue<T> {
	List<T> lst;
	int capacity;

	BoundedQueue (int capacity) { this.capacity = capacity; this.lst =
new ArrayList<T>(); }

	public void add(T obj) {
		if (lst.size() == capacity)
		    throw new Exception(“full queue”);
```

```
            lst.add(obj);
        }
        public T remove() {
            if (lst.size() == 0)
                throw new Exception("empty queue");
            return lst.remove(0);
        }
}
```

3.5.1 מנגנון לניסיון חוזר של ביצוע פעולה

<u>ניסיון ראשון</u>: busy wait חמור (**לא לנסות בבית!**)

```
class BoundedQueue<T> {
        List<T> lst;
        int capcity;

        BoundedQueue (int capacity) {
            this.capacity = capacity;
            this.lst = new ArrayList<T>(); }

        public synchronized void add(T obj) {
            while (lst.size() == capacity);
            lst.add(obj);
        }
        public synchronized T remove() {
            while (lst.size() == 0);
            return lst.remove(0);
        }
}
```

<u>חסרונות</u> (בלשון המעטה):
- אין שחרור של הנעילה של האובייקט בזמן ההמתנה לשינויים בו, כך ששום ת'רד אחר לא יוכל לשנות אותו, כפי שאנו רוצים. ניתן לפתור זאת על ידי שימוש במנגנון הנעילה Semaphore (בשיעור הבא)
- תפיסה מיותרת של זמן המעבד.

<u>ניסיון שני</u>: busy wait

```
class BoundedQueue<T> {
        List<T> lst;
        int capcity;
```

```
        BoundedQueue (int capacity) {
              this.capacity = capacity;
              this.lst = new ArrayList<T>();
        }

        public synchronized void add(T obj) {
               while (lst.size() == capacity)
                  Thread.sleep(1000);
               lst.add(obj);
        }
        public synchronized T remove() {
               while (lst.size() == 0)
                      Thread.sleep(1000);
               return lst.remove(0);
        }
}
```

<u>חסרונות</u>:
- אין שחרור של הנעילה של האובייקט בזמן ההמתנה לשינויים בו, כך ששום ת'רד אחר לא יוכל לשנות אותו, כפי שאנו רוצים. ניתן לפתור זאת על ידי שימוש במנגנון הנעילה Semaphore (בשיעור הבא)
- אין כל אינדיקציה לכך כי לאחר פרק הזמן שצוין ב sleep תנאי ההתחלה אכן יתקיימו.

 נדרש מנגנון מהסביבה המעביר ת'רד למצב blockedעד אשר יחולו שינוי כלשהו באובייקט.

סביבת הריצה תומכת במנגנון שכזה, ע"י שתי פעולות:

wait(T,O)
הת'רד T מבקש לעבור למצב blocked עד אשר יחולו שינוי כלשהו באובייקט O (ובפרט, עד אשר תבוצע על O הפעולה notify המציינת זאת), תוך שחרור הנעילה של T על O.

notify(T,O)
הת'רד T שהמתין לשינוי באובייקט O יחזור למצב runnable, כאשר הפעולה הראשונה שלו היא תפיסה מחודשת של הנעילה של O.

מבחינת המימוש, הסיבה מתחזקת לכל אובייקט רשימה של ת'רדים הממתינים לשינוי בו, כאשר פעולת wait מוסיפה אותו לרשימה ופעול notify מסירה אותו מהרשימה.

ב Java קיימות במחלקה Object שלוש מתודות עבור מנגנון המתנה זה:

o.wait()
הת'רד הנוכחי ממתין לשינוי באובייקט o (כלומר, עובר למצב blocked עד אשר תופעל על o המתודה notify/notifyAll), תוך שחרורו הנעילה של o.

o.notify()
בחירה **אקראית** של **אחד** הת'רדים הממתינים לשינוי ב o והעברתו למצב runnable (תוך דרישה לתפיסה מחודשת את הנעילה של o)

o.notifyAll()
בחירת **כל** הת'רדים הממתינים לשינוי ב o והעברתם למצב runnable (תוך דרישה לתפיסה מחודשת את הנעילה של o)

<u>שימוש במנגנון</u>

קטגורית:
- בכל פעם שתנאי ההתחלה אינם מתקיימים נבצע wait
- לאחר שינויים באובייקט נבצע notifyAll

על מה נבצע wait/notifyAll?

<u>פרוטוקול א:</u> על this (תואם לפרוטוקול א בסנכרון, סנכרון מלא)

```
class BoundedQueue<T> {
       List<T> lst;
       int capcity;

       BoundedQueue (int capacity) {
           this.capacity = capacity;
           this.lst = new ArrayList<T>(); }

       public synchronized void add(T obj) {
                while (lst.size() == capacity)
                   this.wait();
                lst.add(obj);
                this.notifyAll();
       }
       public synchronized T remove() {
              while (lst.size() == 0)
                  this.wait();
              T ret = lst.remove(0);
              this.notifyAll();
              return ret;
       }
}
```

q: <T2, [], **[]>** {}

T1: q.add(1)
T2: q.remove()

<u>פרוטוקול ב:</u> על אובייקט המייצג חלק של this (תואם לפרוטוקול ב בסנכרון, סנכרון חלקי)

```
class Point {
     int x,y;
     final int minX, maxX, minY, maxY;
     Object lockX, lockY;

    Point(int x, int y, int minx, int maxX, int minY, int maxY;) {
```

```
        this.x = x;
        this.y = y;
        this.minX = minx; this.maxX= maxX; this.minY = minY; this.maxY =
maxY;
        lockX = new Object();
        lockY = new Object();
    }

     pubic void getY() {
         synchronized(lockY) {
            return y;
          }
     }
     pubic void up() {
         synchronized(lockY) {
            while (y == maxY)
                 lockY.wait();
            y++;
            lockY.notifyAll();
          }
     }
     pubic void down() {
          synchronized(lockY) {
              while (y == minY)
                 lockY.wait();
              y--;
              lockY.notifyAll();
          }
     }
     pubic void getX() {
         synchronized(lockX) {
            return x;
          }
     }
     pubic void right() {
         synchronized(lockX) {
              while (x == maxX)
                 lockX.wait();
              x++;
              lockX.notifyAll();
         }
     }
     pubic void left() {
         synchronized (lockX) {
              while (x == minX)
                 lockX.wait();
              x--;
              lockX.notifyAll();
         }
     }
```

```
}
```

p: <_, [], **[]>** lockX, lockY, 1,5,2,6 2,6
lockX: <_, [], **[]>**
lockY: <T1, [], **[]>**

T1: p.up()
T2: p.left()
T3: p.down()

<u>פרוטוקול ג</u>: המתנה לשינוי באחד מקבוצת אובייקטים (מתאים לפרוטוקול ג בסנכרון)

```
List<Integer> l1, l2;
Object lockL1L2;
…
synchronized (lockL1L2) {
   while (!preCond())
       lockL1L2.wait();
   … // do something with both L1 and L2
   lockL1L2.notifyAll();
}
```

<u>אטומיות</u>

בכל אחד מהפרוטוקולים, פגשנו תבנית קבועה שנראית כך:

```
synchronized (o) {
       while (!checkPreCond())
            o.wait();
        … // code which changes the state
        o.notifyAll();
}
```

ההבדל בין הפרוטוקולים היה רק בזהות של האובייקט o.

תבנית זו נועדה למנוע שתי בעיות מרכזיות:

1. ביצוע של הקוד כאשר תנאי ההתחלה אינם מתקיימים

   - If במקום while

```
public synchronized T remove() {
              if (lst.size() == 0)
                     this.wait();
              T ret = lst.remove(0);
              this.notifyAll();
```

```
        return ret;
    }
```

q: <T2, [], **[]>** {}

T1: q.add(1)
T2: q.remove()
T3: q.remove()

- חייבים לבדוק את תנאי ההתחלה שוב לאחר ההמתנה
- פיצול הסנכרון בין לולאת הבדיקה וביצוע הקוד

```
public void add(T obj) {
        synchronized (this) {
            while (lst.size() == capacity)
            this.wait();
        }
        synchronized (this) {
            lst.add(obj);
            this.notifyAll();
        }
    }
```

q: <T1, [], **[]>** {2,3,5}, capacity = 3

T1: q.add(4)
T2: q.add(5)
T3: q.remove()

- חייבים רצף סנכרון הכולל את בדיקת התנאי והפעולה בעקבותיו

2. המתנה לשווא כאשר תנאי ההתחלה מתקיימים

- פיצול הסנכרון בין לולאת הבדיקה ל wait וכל השאר (נניח שזה היה אפשרי טכנית, ע"י Semaphore לדוגמא (להלן))

```
public void add(T obj) {
        synchronized (this) {
            while (lst.size() == capacity)
        }
```

```
        synchronized (this) {
              this.wait();
              lst.add(obj);
              this.notifyAll();
        }
  }
```

q: <_, [], **[]> {1’2,3}, capacity = 3
T1: q.add(4)
T2: q.remove()

- חייבים רצף סנכרון הכולל את בדיקת התנאי (ב while) והמעבר ל wait (והפעולה בעקבותיו)
- ביצוע notify במקום notifyAll

q: <_, [], **[T1,T2]>** {}, capacity = 1
T1: q.add(4)
T2: q.remove()
T3: q.remove()

- רק אם כל הת'רדים ממתינים על אותו תנאי ניתן לייעל ולבצע notify במקום notifyAll (נראה להלן דוגמא ב Semaphore)

<u>הערה</u>: כל זאת בסמנטיקת wait/notify בסביבתה ריצה של Java. בקורס במערכות הפעלה נראה סמנטיקות אחרות בהן ניתן להשתמש ב if/notify.

3.5.2 מימוש מנגנון סנכרון מונחה blocked על ידי wait/notify

ראינו עד כה שני מימושים של מנגנון הסנכרון:
- מימוש הסנכרון ע"י synchronized, כאשר הת'רדים הממתינים עוברים למצב blocked
- מימוש הסנכרון בעזרת הפעולה האטומית CompareAndSet, כאשר הת'רדים הממתינים נשארים במצב running.

בסעיף זה נממש בדרך שלישית את מנגנון הסנכרון ע"י שימוש ב wait/notify (השימוש המרכזי של wait/notify הוא אמנם בכלל עבור המתנה לקיום תנאי התחלה, אך ניתן לנצל אותו גם לשם סנכרון).
במנגנון זה, הת'רדים הממתינים יעברו למצב blocked, כמו במנגנון ה synchronized, תוך פתרון של מספר חסרונות במנגנון ה synchronized הקיים:

1. לא ניתן לנעול או לשחרר את האובייקט באופן דינאמי בזמן ריצה. מיקום פעולת הנעילה והשחרור נקבע כבר בזמן קומפילציה (היכן שמצוינת המילה synchronized, והיכן שמסתיים הבלוק.

2. לא ניתן 'לנסות' לנעול את האובייקט, כך שאם זה לא אפשרי נמשיך לרוץ ונבצע משהו אחר.
3. לא ניתן לממש מדיניות אחרת מלבד זו שרק ת'רד אחד מבצע את קטעי הקוד הקריטיים.
4. אין הוגנות בתפיסת המשאב הנדרש.

Semaphore 3.5.2.1

מחלקת הסמפור מממשת מנגנון סנכרון בעזרת wait/notify תוך טיפול בחסרונות מנגנון ה synchronized שצוינו למעלה:

1. ניתן להחליט בזמן ריצה, באופן דינאמי בהתאם לתנאים 'בשטח', מתי נועלים את האובייקט ומתי משחררים אותו.
   בפרט, נגדיר שתי מתודות acquire, release הממשות את הפעולות lock, unlock.
2. נגדיר מתודה tryAcquire ה'מנסה' לנעול את האובייקט, ואם זה לא אפשרי מחזירה false אך לא תעביר את הת'רד למצב blocked.
3. נאפשר כניסה לקטע הקוד הקריטי למספר ת'רדים בו זמנית, בהתאם לפרמטר.
4. נממש הוגנות על ידי תור, כך שהת'רד שיתפוס את הנעילה יהיה זה שממתין לכך הכי הרבה זמן.

```
class Semaphore {
      protected final int permits;  // how many threads can work together
(ג)
      protected int free; // how many places are currently left for other
threads working together

     Semaphore(int permits) { this.permits = permits; this.free = permits;
}

     public synchronized void acquire()  throws InterruptedException {
// implementation of lock
              while (free == 0)
                   wait();
              free--;
     }

     public synchronized boolean tryAcquire()  {
              if (free == 0)
                   return false;
              free--;
              return true;
     }

     public synchronized void release() {  // implementation of unlock
            if (free < permits)
                   free++;
              notify();
     }
```

```
}
```

נשתמש בסמפור לשם סנכרון המחלקה Even, לדוגמא:

```
class Even {

   int val;
   Semaphore sem;

   Even(int val)  throws Exception {
        if (val % 2 != 0)
            throw new Exception(…);
        this.val = val;
        this.sem = new Semaphore(1);
   }
   public int get() {
        sem.acquire();
        int ret = val;
        sem.release();
        return ret;
     }
   public void next() {
        sem.acquire();
        val++;
        val++;
        sem.release();
    }
}
```

e: 2, sem
sem: 1,1 <_,[],[]>
T1: e.next()
T2: e.next()

נרחיב את מחלקת הסמפור כך שתממש ההוגנות:

```
class FairSemaphore {
      protected final int permits;  // how many threads can work together
(ג)
      protected int free; // how many places are currently left for other
threads working together
      List<Thread> waitingForLockThreads;

      Semaphore(int permits) { this.permits = permits; this.free =
permits;
```

```
                waitingForLockThreads = new LinkedList<Thread>();
        }

        public synchronized void acquire()  {   // implementation of lock
                waitingForLockThreads.add(Thread.currentThread());
                while (free == 0 || waitingForLockThreads.get(0) !=
    Thread.currentThread())
                     wait();
                free--;
                waitingForLockThreads.remove(0);
                if (free > 0 && waitingForLockThreads.size() > 0)
                   notifyAll();
        }

        public synchronized boolean tryAcquire()  {
                if (free == 0 || waitingForLockThreads.size() > 0)
                     return false;
                free--;
                return true;
        }

       public synchronized void release() {  // implementation of unlock
                if (free < permits)
                      free++;
                notifyAll();
        }
}
```

הערות:

- נדרש notifyAll ולא notify, כי תנאי ההמתנה שונה לכל ת'רד. בפרט, ייתכן מצב שבו הת'רד האקראי שייבחר ב notify יהיה זה הנמצא שני ברשימה, כך שתנאי ההתחלה לא יתקיים עבורו והוא יחזור לישון.
- ניתן לחשוב על מימוש יעיל יותר, בו 'מעירים' רק את הת'רד הראשון ברשימה (במקום לשווא את כולם, רק הראשון ימשיך לרוץ), אך יש קושי טכני רב מבחינת ניהול הסנכרון של הפעולות על הרשימה (לא ניכנס לזה).

ReaderWriter 3.5.2.2

- בתבנית עיצוב זו, נטפל באופן רחב ואפקטיבי יותר בחיסרון השלישי: מימוש מדיניויות סנכרון שונות מתי מותר לכמה ת'רדים לעבוד יחדיו (ניתן היה פשוט להרחיב את המחלקה Semaphore, אך שלא לסבך נאמץ את תבנית העיצוב המקובלת של המחלקה ReaderWriter)

המחלקה מסדירה את הגישה לביצוע פעולות קריאה וכתיבה על פי מדיניות סנכרון כלשהי, על ידי השלד הבא:

```
class ReaderWriter {
     public synchronized void readLock() {
           while (!allowRead())
                wait();
           …
     }
     public synchronized void readUnlock() {
            …
            notifyAll();
     }
     public synchronized void writeLock() {
           while (!allowWrite())
                wait();
           …
     }
     public synchronized void writeUnLock () {
          …
          notifyAll();
     }
     protected boolean allowRead() {
            return …;
     }
     protected boolean allowWrite() {
            return …;
     }
  }
```

נממש מדיניות סנכרון ספציפית בשלד הנתון:
- ת'רדים המבצעים פעולות קריאה יכולים לעבוד יחדיו ללא הגבלה
- הכתיבה דורשת בלעדיות: לא ניתן לבצע קריאה וכתיבה במקביל, וכן לא ניתן לבצע כתיבה וכתיבה במקביל.

כדי לממש מדיניות סנכרון, נדרש:
- הגדרת מבנה נתונים לשם קבלת ההחלטה האם מותר או אסור להיכנס לקטע הקוד המוגן (עבור קריאה או כתיבה)
- תחזוק מבנה הנתונים על פי התרחישים השונים בשלד (לפני הכניסה, אחריה)
- מימוש מתודות המותר/אסור (allowRead, allowWrite) בהתאם למצב מבנה הנתונים במקרה שלנו, המבנה יכלול מידע על מספר הקוראים ומספר הכותבים.

```
class ReaderWriter {
```

```
    int activeReaders = 0, activeWriters = 0;
    public synchronized void readLock() {
          while(!allowRead())
               wait();
          activeReaders++;
    }
    public synchronized void readUnlock() {
          activeReaders--;
          notifyAll();
    }
    public synchronized void writeLock() {
          while(!allowWrite())
               wait();
          activeWriters++;
    }
    public synchronized void writeUnlock() {
         activeWriters--;
         notifyAll();
    }
    protected boolean allowRead() {
          return activeWriters == 0;
    }
    protected boolean allowWrite() {
          return activeWriters == 0 && activeReaders == 0;
    }
}
```

חיסרון: ניתן לחשוב על תרחיש בו יש כל הזמן ת'רדים שקוראים, כך שהת'רד הכותב לא יזכה להגיע לכך (למרות שהוא הגיע לפני מרביתם). יש במימוש הנוכחי יתרון מובהק לת'רדים הקוראים.
נרחיב את מדיניות הסנכרון, כך שתיקח בחשבון מספר מספר הת'רדים הממתינים לכתוב.

```
class ReaderWriter {
    int activeReaders=0, activeWriters=0, waitingWriters=0;
    public synchronized void readLock() {
          while(!allowRead())
               wait();
          activeReaders++;
    }
    public synchronized void readUnlock() {
          activeReaders--;
          notifyAll();
    }
    public synchronized void writeLock() {
          waitingWriters++;
          while(!allowWrite()) {
               wait();
```

```
            waitingWriters--;
            activeWrites++;
      }
      public synchronized void writeUnlock() {
            activeWrites--;
           notifyAll();
      }
      protected boolean allowRead() {
            return activeWriters == 0 && waitingWriters == 0;
      }
      protected boolean allowWrite() {
            return activeWriters == 0 && activeReaders == 0;
      }
  }
```

במידה ויש כעת יתרון לת'רדים הכותבים, שאנו רוצים לצמצם, נוסיף שדה נוסף waitingReades, נתחזק אותו, ונכלול אותו במערך שיקולי ה allowRead, allowWrite.

```
class ReaderWriter {
     int activeReaders=0, activeWriters=0, waitingWriters=0,
waitingReaders=0;
    public synchronized void readLock() {
            waitingReaders++;
            while(!allowRead())
                 wait();
            waitingReaders--;
            activeReaders++;
      }
      public synchronized void readUnlock() {
            activeReaders--;
            notifyAll();
      }
      public synchronized void writeLock() {
            waitingWriters++;
            while(!allowWrite()) {
                 wait();
             waitingWriters--;
             activeWrites++;
      }
      public synchronized void writeUnlock() {
           activeWrites--;
           notifyAll();
      }
      protected boolean allowRead() {
            return activeWriters == 0 && (waitingReaders > waitingWriters);
      }
      protected boolean allowWrite() {
            return activeWriters == 0 && activeReaders == 0;
```

```
    }
  }
```

דוגמא לשימוש במנגנון הסנכרון של ReaderWriter:

```
class Even {

   int val;
   ReaderWriter rw;

   Even(int val)  throws Exception {
        if (val % 2 != 0)
            throw new Exception(...);
        this.val = val;
        this.rw = new ReaderWriter();
   }
   public int get() {
        rw.readLock();
        int ret = val;
        rw.readUnlock();
        return ret;
     }
   public void next() {
        rw.writeLock();
        val++;
        val++;
        rw.writeUnlock();
    }
}
```

Cancellation 3.6

ראינו כי המחלקה Thread כוללת מתודה stop() העוצרת באופן חד את הריצה של הת'רד. ציינו כי גישה זו אינה מומלצת בלשון המעטה, וכי הדרך הנכונה להפסיק את הריצה של הת'רד היא 'להודיע' שצריך לסיים.

<u>דוגמא</u>: תבנית Producer-Consumer




```
abstract class Producer<T> implements Runnable {
      BoundedQueue<T> blockingQ;
      public abstract T produce();
      Producer(BoundedQueue<T> q) { blockingQ = q; }

      public void run() {
             while (true)
                blockingQ.add(produce());
      }
}
abstract class Consumer<T> implements Runnable {
      BoundedQueue<T> blockingQ;
      public abstract void consume(T obj);

      Consumer(BoundedQueue<T> q) { blockingQ = q; }

      public void run() {
             while (true)
                  consume(blockingQ.remove());
      }
}
class Test {
  public static void main(String[] args) {
      BoundedQueue<Integer> q = new BoundedQueue<Integer>();
      Runnable producer = new Producer<Integer> (q) {
               public Integer produce() { return Random.nextInt(); }
      };
      Runnable consumer = new Consumer<Integer> (q) {
               public void consume(Integer i) { System.out.println(i); }
      };
      Thread tProducer = new Thread(producer);
      Thread tConsumer = new Thread(consumer);
      tProducer.start();
      tConsumer.start();
      tProducer.stop();
      tConsumer.stop();
      ???
      ???
  }
}
```

כיצד 'נודיע' לת'רדים השונים שיש לסיים?

<u>פיתרון א</u>: נרחיב את הממשק Runnable עם המתודות הבאות:

```
void interrupt();
boolean isInterrupted();
```

וננמשן בעזרת שדה בוליאני המציין האם צריך לסיים:
- ערך התחלתי false
- פעולת ה interrupt תמומש כהשמת השדה ל true
- פעולת isInterrupted תחזיר את ערכו הנוכחי

```
abstract class Interruptable implements Runnable {
      boolean volatile bInterrupted = false;
      public boolean isInterrupted() { return bInterrupted; }
      public void interrupt () { bInterrupted = true; }
}

class Producer<T> extends Interruptable implements Runnable {
      public abstract T produce();
      public void run() {
            while (!isInterrupted())
               blockingQ.add(produce());
      }
}
class Consumer<T> extends Interruptable implements Runnable {
      public abstract void consume(T obj);
      public void run() {
            while (!isInterrupted())
                 consume(blockingQ.remove());
      }
}
class Test {
    public static void run(String[] args) {
      Producer producer = new Producer<Integer> () {
               public Integer produce() { return Random.nextInt(); }
      };
      Consumer consumer = new Consumer<Integer> () {
               public void consume(Integer i) { System.out.println(i); }
      };
      Thread tProducer = new Thread(producer);
      Thread tConsumer = new Thread(consumer);
      tProducer.start();
      tConsumer.start();
      ~~tProducer.stop();~~
      ~~tConsumer.stop();~~
      producer.interrupt();
      consumner.interrupt();
   }
}
```

בעיה: ייתכן שהת'רדים של ה producer וה consumer נמצאים במצב blocked (בעקבות ה wait על התור), כך שהם אינם מודעים להוראה לסיים.
נדרש שירות של הסביבה, הדואג לכך שאם הת'רדים 'ישנים', הם יחזרו למצב running וייזרק

InterruptedException שייאלץ אותם לדעת זאת.
שרות כזה אכן ניתן ע"י הסביבה, עם ממשק במחלקה Thread.

```
class Producer<T> implements Runnable {
      BoundedQueue<T> blockingQ = new BoundedQueue<T>();

      public abstract T produce();

      public void run() {
             while (!Thread.currentThread().isInterrupted())
                blockingQ.add(produce());
      }
}
class Consumer<T> implements Runnable {

      BoundedQueue<T> blockingQ = new BoundedQueue<T>();

      public abstract void consume(T obj);

      public void run() {
             while (!Thread.currentThread().isInterrupted())
                  consume(blockingQ.remove());
      }
}
class Test {
  public static void run(String[] args) {
      Runnable producer = new Producer<Integer> () {
               public Integer produce() { return Random.nextInt(); }
      };
      Runnable consumer = new Consumer<Integer> () {
               public void consume(Integer i) { System.out.println(i); }
      };
      Thread tProducer = new Thread(producer);
      Thread tConsumer = new Thread(consumer);
      tProducer.start();
      tConsumer.start();
      ~~tProducer.stop();~~
      ~~tConsumer.stop();~~
      tProducer.interrupt();
      tConsumner.interrupt();
 }
}

class BoundedQueue<T> {
      …
       public synchronized void add(T obj) {
              try {
                while (lst.size() == capacity)
                   this.wait();
                lst.add(obj);
```

```
                this.notifyAll();
            } catch (IterruptedException e) {
                    ...
            }
        }
        public synchronized T remove() {
           try {
               while (lst.size() == 0)
                      this.wait();
               T ret = lst.remove(0);
               this.notifyAll();
               return ret;
            } catch (IterruptedException e) {
                    ...
            }
        }
}
```

התגובה ל InterruptedException:

- פעולות סיום לוגיות
- החלטה האם להעביר הלאה את המידע על הפסקת המשימה

השדה bInterrupt מתאפס עם תפיסת ה InterruptedException, כלומר חוזר להיות false, כך שאם רוצים להעביר הודעה זו למעלה יש לבצע על ה'תרד הנוכחי שוב interrupt.

```
public synchronized void add(T obj) {
            try {
                 while (lst.size() == capacity)
                 this.wait();
                 lst.add(obj);
                 this.notifyAll();
            } catch (IterruptedException e) {
                  ... // closing actions
                  Thread.currentThread().interrupt();
            }
        }
        public synchronized T remove() {
           try{
               while (lst.size() == 0)
                      this.wait();
               T ret = lst.remove(0);
               this.notifyAll();
               return ret;
            } catch (IterruptedException e) {
                    ... // closing actions
                    Thread.currentThread().interrupt();
            }
        }
}
```

Liveness 3.7

3.7.1 מבוא

ניתן להערך ביצועים של מערכת בזמן ריצה על פי מדדים שונים:

<u>תפוקה (Throughput)</u>: מספר פעולות ליחידת זמן (או זמן ביצוע של פעולה)

<u>Latency</u>: כמה זמן עובר מהעלאת בקשה ועד לביצועה.

<u>יעילות (Efficiency)</u>: טיב התפוקה/ה-latency ביחס למשאבים הקיימים (מעבדים, זיכרון...)
תפוקה / 'מספר המשאבים'

<u>Scalability</u>: בכמה תשתפר התפוקה/ה-latency אם יתווספו משאבים למערכת.

 ניתן לבדוק יעילות של מערכת קיימת שרצה, על פי מדדים כאלה ואחרים.

מדוע שמדדים אלו יהיו נמוכים עבור מערכת נתונה הממומשת לוגית באופן יעיל, אילו גורמים פוגעים בהם?

- מערכת סדרתית או מקבילית
  - o המתנה ל i/o (קלט מהמקלדת, מקבצים, מידע מהרשת)
  - o המתנה לזמן מעבד (עקב מדיניות תזמון לא הוגנת)
- מערכת מקבילית
  - o סנכרון: מעבר למצב blocked עקב ניסיון נעילה (או במקרה של CompareAndSet סוג של busy wait עבורם)
  - o המתנה לקיום תנאי התחלה: מעבר למצב blocked ע"י wait

במקרים שכאלה אנו אומרים כי המערכת סובלת מבעיות liveness (חַיּוּת). נבחן מספר תרחישים קלאסיים של בעיות שכאלה.

3.7.2 תרחישים

LiveLock

בתרחיש זה, קבוצת ת'רדים עובדת ללא הרף אך הת'רדים אינם מתקדמים במשימתם, כי הם חוסמים זה לזה ללא הרף את תנאי ההתחלה.
<u>קורה כאשר</u>:
- קוד המשימות של הת'רדים סימטרי
- יש תזמון ספציפי, חוזר ונשנה, בו סדר הפעולות גורם לכך שהת'רדים מבטלים את תנאי ההתחלה זה לזה.
<u>דוגמאות</u>:
- שני אנשים במסדרון
- הפילוסופים הרעבים (להלן)
<u>פתרון שכיח</u>: הכנסת ממד רנדומי לתזמון (כמו ביצוע sleep בקוד המשימה לפרק זמן אקראי)

Starvation

מאחר שמנגנוני הסנכרון וההמתנה אינם הוגנים (לפחות ב Java), ייתכן מצב שבו אחד הת'רדים תמיד מצליח לתפוס את הנעילה, או מתעורר ראשון מהמתנה, בעוד ת'רד אחר נכשל בכך (מגיע שני) שוב ושוב, כך שהוא 'מורעב' מבחינת קבלת זמן CPU / היות במצב running.
<u>דוגמא</u>: הפילוסופים הרעבים.
<u>פתרון</u>: שימוש במנגנון סנכרון/המתנה הוגנים, כמו ה FairSemaphore.

DeadLock (חֶבֶק)

קבוצה של ת'רדים נמצאת במצב blocked, ואין שום ת'רד אחר שיכול להוציא אותם ממצב זה, אלא רק ת'רדים מהקבוצה (שאינם רצים כי הם במצב blocked)

שני תרחישים:

1. כל אחד מהת'רדים נמצא במצב blocked עקב המתנה לנעילה של משאב (בעקבות synchronized וכדומה)

דוגמא:

```
class Even {
      int val;
      …
      public void swap(Even other) {
          synchronized (this) {
             synchronized(other) {
               int tmp = other.val;
               other.val = val;
               val = tmp;
          }
        }
      }
}
```

e1: <T1,[T2],[]> 2
e2: <T2,[T1],[]> 4
T1: e1.swap(e2);
T2: e2.swap(e1);

<u>פתרונות</u>:

i. ננעל את שני האובייקטים בפעולה אחת, ע"י 'פרוטוקול ג' בו מיוצגים מספר אובייקטים ע"י אובייקט סנכרון/המתנה אחד. לא כל כך יעיל, כל גישה לאחד האובייקטים בנפרד תנעל את שניהם.

ii. ננעל את האובייקט הראשון, ואת האובייקט השני 'ננסה' לנעול (על ידי tryAcquire במחלקה Semaphore), אם הוא כבר נעול נשחרר את האובייקט הראשון וננסה שוב אחר כך. סכנת LiveLock - דוגמא בהמשך, בעיית הפילוסופים הרעבים.

iii. נימנע משיטה המבטיחה את הנכונות והבטיחות על ידי סנכרון המעביר את הת'רד הממתין למצב blocked (כמו ImmutableObjects)

iv. Resource ordering [דרך המלך]
נקבע סדר קבוע לתפיסת המשאבים, כך שכל אחד מהת'רדים יקפיד לתפוס את המשאבים בסדר זה, ויימנע ה deadlock שנגרם תמיד ע"י סימטריה בסדר תפיסת המשאבים.

בדוגמא שלנו, נקבע כי סדר התפישה של אובייקטים מסוג Even נקבע על פי זהותם הייחודית במערכת (כפי שניתן על ידי המתודה identityHashCode במחלקה System)

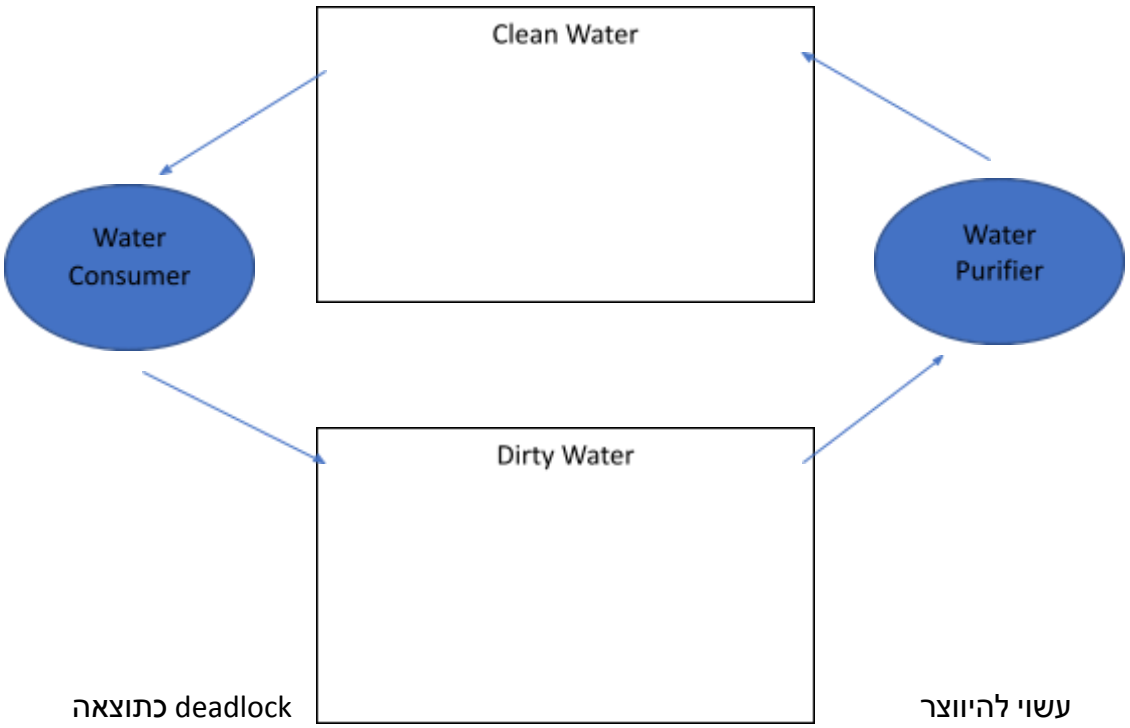
```
public void swap(Even other) {

        if (System.identityHashCode(this) <
            System.identityHashCode(other))
             synchronized (this) {
                 synchronized(other) {
                    int tmp = other.val;
                    other.val = val;
                    val = tmp;
                }
             }
       else
           synchronized (other) {
                 synchronized(this) {
                    int tmp = other.val;
                    other.val = val;
                    val = tmp;
                }
           }
{
```

```
e1: <T2,[],[]>  2     ihc 100
e2: <T2,[],[]>  4     ihc 200
T1: e1.swap(e2);
T2: e2.swap(e1);
```

2. הת'רדים בקבוצה במצב blocked עקב המתנה הדדית לקיום תנאי ההתחלה, כאשר שהת'רד שאמור לשנות את תנאי ההתחלה ממתין אף לתנאי התחלה משלו התלויים בת'רד אחר בקבוצה

דוגמא: [ממבחן] סימולציה של מערכת לטיהור מים ביתית

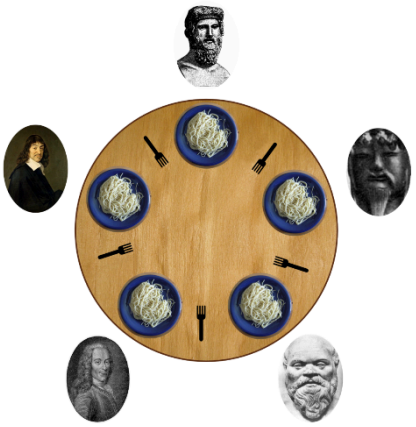


עשוי להיווצר deadlock כתוצאה מהמתנה לקיום תנאי התחלה, אם שני המיכלים מלאים ושני הת'רדים מנסים לצקת את המים שלהם, כל אחד למיכלו.

<u>פתרון אפשרי</u>: שבירת הסימטריה. לדוגמא, ת'רד נוסף שמתעורר מדי פעם ובודק האם יש תצורה של תרחיש זה (סוכם את כמות המים הכוללת במיכלים ואצל הת'רדים, ובודק האם הם חורגים מהאפשר), ואם כן הוא 'שופך' מים מאיזה מיכל.

<u>נדגים את כל התרחישים שציינו, על ידי בעיית 'הפילוסופים הרעבים'</u>:

אורח חייו של הפילוסוף מורכב מחשיבה, ואכילה. פעולת האכילה דורשת תפיסה של שתי המזלגות לצד הצלחת של הפילוסוף.



```
class Philosopher implements Runnable {
     Fork left, right;
    Philosopher(Fork left, Fork right) { this.left = left; this.right =
right; }
```

```
    public void run() {
       while (!Thread.currentThread().isInterrupted()) {
          think();
          synchronized (left) {
               synchronized  (right) {
                    eat();
               }
          }
       }
    }
```

בעיה א: deadlock

פתרון א1: resource ordering

```
class Philosopher implements Runnable {
    Fork left, right;
    Philosopher(Fork left, Fork right) {
       this.left = left; this.right = right; }
     public void run() {
       while (!Thread.currentThread()isInterrupted()) {
           think();
           if (System.identityHashCode(left) <
               System.identityHashCode(right)) {
                synchronized(left) {
                    synchronized(right) {
                        eat();
                    }
                }
         else
               synchronized (right) {
                    synchronized (left) {
                        eat();
                    }
               }
    }
  }
```

בעיה ב: starvation

פתרון: נשתמש בסמפור הוגן

```
class Fork {
    public FairSemaphore sem = new FairSemaphore(1);
    …
}

class Philosopher implements Runnable {
     Fork left, right;

     public void run() {
       while (!Thread.currentThread()isInterrupted()) {
           think();
           if (System.identityHashCode(left) <
               System.identityHashCode(right)) {
                left.sem.acquire();
                right.sem.acquire();
                eat();
                right.sem.release();
                left.sem.rerlease();

              }
         else {
                right.sem.acquire();
                left.sem.acquire();
                eat();
                left.sem.release ();
                right.sem.release();
            }
          }
         }
```

<u>פתרון א2</u> (לבעיית החבק): נתפוס את המזלג הראשון, ננסה לתפוס את המזלג השני, ואם לא ילך נוותר על האכילה בפעם זו

```
class Philosopher implements Runnable {
     Fork left, right;

     public void run() {
       while (!Thread.currentThread()isInterrupted()) {
           think();
           left.sem.acquire();
           if (right.sem.tryAcquire()) {
                eat();
                right.sem.release();
                left.sem.rerlease();
           } else
                left.sem.release();
     }
}
```

בעיה: LiveLock
פתרון: נוסיף אלמנט רנדומי

```
class Philosopher implements Runnable {
     Fork left, right;

     public void run() {
      while (!Thread.currentThread().isInterrupted()) {
           think();
           left.sem.acquire();
             Thread.sleep(Random.nextInt(1000));
           if (right.sem.tryAcquire()) {
                eat();
                left.sem.release();
                right.sem.rerlease();
           } else
                left.sem.release();
     }
   }
```

4. תקשורת בין תהליכים (Interprocess Communication)

4.1 מבוא

4.1.1 מוטיבציה

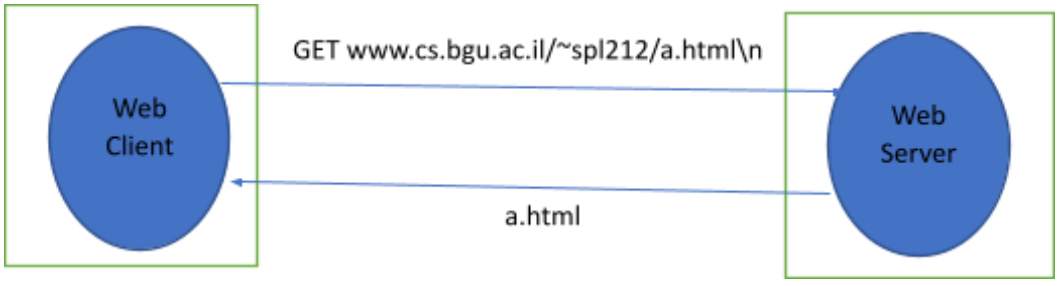
עד כה עסקנו במודל מקביליות של מספר ת'רדים בתהליך אחד:
- מרחב זיכרון אחד לכל הת'רדים (Heap משותף)
- לכל ת'רד מחסנית משלו
- אוטונומיה על המעבד (גם כאשר המעבד משותף לכמה ת'רדים, הערכים ברגיסטרים ייחודיים לכל ת'רד)

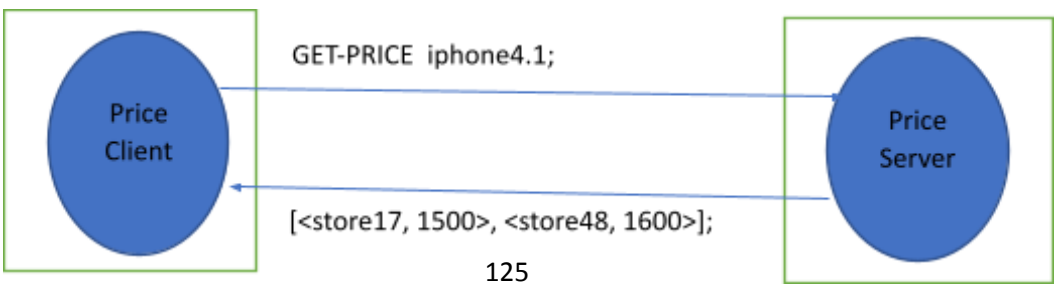
מסתבר כי מודל מקבילי זה, בו רצות מספר משימות במקביל בתהליך אחד, אינו מספק עבור אפליקציות שכיחות הדורשות מקביליות.
לדוגמא:

1. מספר תהליכים על אותו מחשב הנדרשים לשתף פעולה, ובפרט 'לדבר זה עם זה' (נדיר יותר):
   - o אפליקציה לבניית מערכת שעות (מקרה שהיה באמת)
     - מנוע השיבוץ (מימוש האלגוריתם לפתרון כל האילוצים) הוא תהליך בשפה הלוגית פרולוג.
     - כל הממשק עם המשתמש מומש בג'אווה
     - נדרשת תקשורת שוטפת בין שני התהליכים
   - o אפליקציה לניהול חדר חדשות תקשורתי
     - אפליקציית C++, שכללה בין היתר מעבד תמלילים
     - בדיקת האיות נעשית על ידי תהליך עצמאי (בגלל הסיבוכיות של בדיקת איות בפינית), ispel/aspel

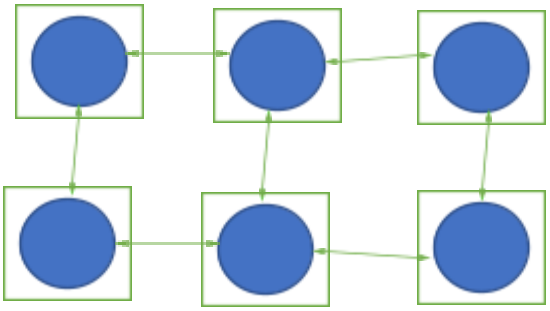
2. שני התהליכים רצים על מחשבים שונים

   - o ארכיטקטורת Client-Server





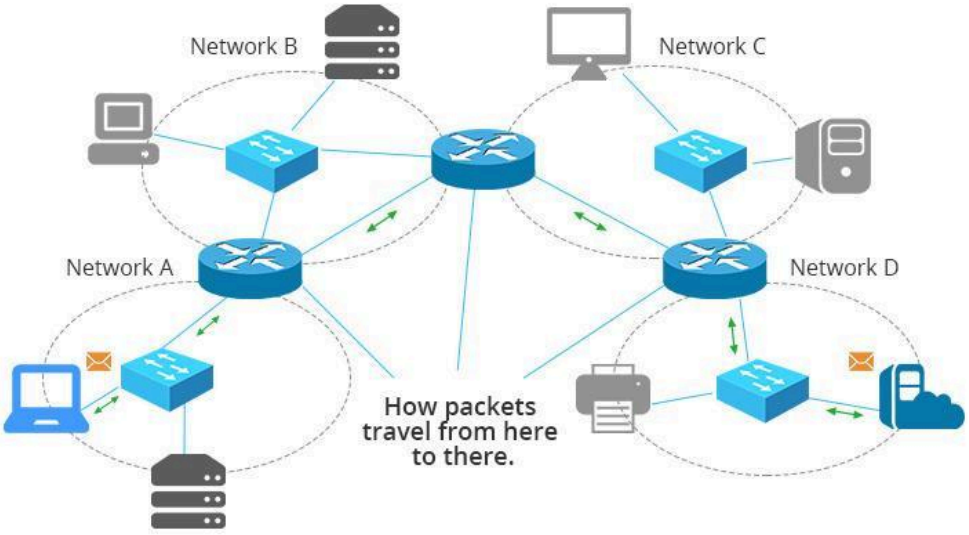
- ארכיטקטורת peer-2-peer



3. קלאסטר של מחשבים בענן המבצעים יחדיו ג'וב על ביג-דאטה (cloud computing), או אלגוריתם סבוך במיוחד הדורשת מקביליות מעבר למספר הת'רדים האפשרי בתהליך בודד.

בפרק זה נתמקד בארכיטקטורת Client-Server השכיחה ביותר.
בפרט נעמוד על בעיות שונות הקשורות לעיצובה, עד להגדרת תבנית כללית של שרת יעיל.

4.1.2 הארכיטקטורה של הרשת



התקשורת בין מחשבים שונים ברשת שכזו, מבוססת על רמות שונות של תקשורת ושירותים, או אם תרצו ארכיטקטורה של כמה 'שכבות'.

כדי להמחיש זאת, נתאר לעצמנו מה קורה כאשר אנחנו שולחים מכתב (פיזי, בדואר) לחבר או חברה:

ניסוח מכתב
כתיבתו על דף
הכנסתו למעטפה

הוצאת מכתב מתיבת הדואר הנכנס
קריאת המכתב, הבנתו
תגובה בהתאם

ציון כתובת השולח והנמען
הטלת המכתב לתא הדואר היוצא

איסוף מכתבים מתיבות ביתיות והעברתם לסניף הדואר המקומי תוך מתן שירותים שונים (שחזור מכתב שהלך לאיבוד)

הפצת מכתבים שהגיעו לסניף לתיבות בבתים תוך מתן שירותים שונים (שמירת מכתבים בסניף כאשר הלקוח בחו"ל)

העברת מכתב מסניף דואר אחד לסניף דואר מרוחק (קביעת מרחב הכתובות, המסלול המתאים כרגע,...)

העברת מכתב מסניף דואר אחד לסניף הסמוך לו (בדיקה ששום ביט לא הולך לאיבוד,...)

מסתבר שגם הרשת בנויה על בסיס שכבות שכאלו:

Application Layer
בשכבה זו אנו כותבים אפליקציות, תוכניות המבוססות על תקשורת עם תהליכים אחרים. התקשורת עם תהליכים אחרים מבוססת על ממשק הניתן ע"י שכבת הטרנספורט (כשירות של סביבת הריצה), הכולל פעולות של קריאה/קבלה וכתיבה/שליחה.

Transport Layer
שכבה זו מספקת את ההפשטה של הקשר הישיר בין התהליכים לשכבת האפליקציה. בפרט, היא אחראית להעביר את המידע המתקבל בראוטר למקום המתאים בתהליך ברשת המקומית, ומצד שני מעבירה את המידע הנשלח מתהליך במחשב החוצה אל שכבת הרשת. שכבה זו מספקת גם שירותים נלווים נוספים (דוגמאות בהמשך). מימושים קיימים לשכבה זו: הפרוטוקלים TCP,UDP (נראה מיד)

Network Layer
שכבה זו אחראית להעביר יחידות מידע ברשת, מצומת (נניח 'ראוטר') אחת לצומת מרוחקת (מממשת אלגוריתמי ניתוב). שכבה זו מגדירה גם את הכתובות ברשת (כתובות IP), ועוד. מימוש קיים, פרוטוקול IP.

DataLink Layer
העברת ביטים מצומת אחת לצומת הסמוכה לה. זיהוי שגיאות, תיקון שגיאות, ועוד.

בסעיף הבא נתמקד בשירות הניתן על ידי שכבת הטרנספורט.

4.2 קשרים בין תהליכים

4.2.1 מושגים

Host: תחנה ברשת, מחשב המחובר לרשת.

IP Address: כתובתו של host ברשת.
פרוטוקול שכבת הרשת, פרוטוקול IP, מגדיר את מרחב הכתובות כמספר בן 32 ביטים, המתואר כארבע מספרים בתחום 0-255. לדוגמא: 132.23.45.107
(ניתן גם לתאר כתובת כמחרוזת דוגמת www.cs.bgu.ac.il, כאשר יש שרת שמות, DNS, המתרגם מחרוזות אלו לכתובות IP)

Communication (תקשורת): החלפת מידע בין שני תהליכים, הרצים על hosts (יחידת המידע הבסיסית המועברת, דרך השכבות מתחת, מכונה packet)

Socket: הממשק הניתן על ידי שכבת הטרנספורט לתהליכים המעוניינים לנהל תקשורת בשכבת האפליקציה. ממשק זה יכלול בדרך כלל פעולות של שליחה/כתיבה ושל קבלה/קריאה.
ניתן לחשוב על socket כ'פתח' בתהליך כלפי חוץ, המאפשר קבלת או שליחת מידע (כמו תיבת דואר, או שפופרת טלפון)

[מבחינת המימוש, כאשר אנו מקבלים Socket משכבת הטרנספורט, בעקבות קריאה למערכת, מערכת ההפעלה מגדירה עבורו שני מערכי בתים, 'באפרים', האחד כולל את הבתים למשלוח, והשני את הבתים שהגיעו מהרשת. באופן זה, שכבת הטרנספורט מעבירה בתים מהרשת לבאפר הקריאה, ושולחת בתים לרשת מבאפר הכתיבה]

Port: כדי לאפשר לתהליך לנהל מספר שיחות במקביל, כלומר להחליף מידע עם כמה תהליכים, נדרש לסמן את ה sockets השונים שהתהליך משתמש בהם (כל אחד לשיחה אחרת). מספר ה port משמש לכך (כמו מספר שלוחה בטלפון, או מספור של תיבות דואר של בני משפחה שונים).
הערה: למעשה, תקשורת של שני תהליכים בעזרת שני sockets, ניתנת לזיהוי ייחודי ע"פ הרביעיה הבאה:

<source host address, source host port, target host address, target port>

4.2.2 שני סוגי הקשרים הניתנים ע"י שכבת הטרנספורט

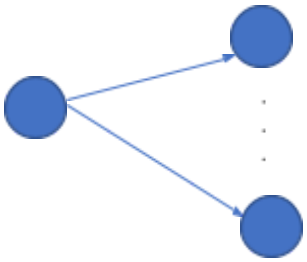
ציינו כי כבר כי שכבת הטרנספורט מספקת (כיום) שני סוגי קשרים בין תהליכים.
ניתן לאפיין קשרים אלו, ע"פ מספר פרמטרים:

- האם הקשר הין התהליכים הוא מונחה שיחה (session-oriented) הוא מונחה שליחת הודעה אחת (datagram-oriented).
  - מונחה שיחה (כמו שיחת טלפון)
    - פרוצדורת אתחול קשר (הכוללת את הכתובת ועוד). נדרש שהצד השני זמין כרגע.
    - העברה חופשית, בשני כיוונים, של 'זרם בתים'. אין צורך לציין שוב ושוב את הכתובת.
    - האחריות על פרשנות הבתים כ'הודעות' מוטלת על המשוחחים
    - ההחלטה מתי מסתיימת השיחה מוטלת על המשוחחים
  - מונחה הודעה (כמו שליחת מכתב בדואר)
    - אין אתחול של 'קשר'. לא נדרש שהצד השני כרגע מחובר.
    - העברת יחידה של 'הודעה', לא סתם כתיבה של בתים. יש צורך לציין את הכתובת בכל משלוח.
    - הנמען אינו צריך לפרש מתי הסתיימה הודעה. הוא מקבל את הבתים כבר במבנה של הודעה.
    - הקשר מסתיים לאחר כל קבלת הודעה. אין 'שיחה' במובן הפשוט, השיחה היא סדרה של יצירת קשר להעברת הודעות.
- מהי 'טופולגיית' הקשר בין התהליכים (מבחינת מספר התהליכים המשתתפים בקשר, ומבחינת היחסים ביניהם)

- Unicast



- Multicast



- מהי רמת ה'שירותים נוספים' הנלווים לקשר (דוגמאות להלן)

שני סוגי הקשרים הניתנים על ידי שכבת הטרנספורט הם:

- (UDP (User Datagram Protocol
  - מונחה שליחת הודעה, datagram-oriented
  - תומך גם ב unicast וגם ב multicast
  - שירותים נלווים מצומצמים
    - אמינות: אם נשלחו הודעות A,B – ייתכן שהן לא יגיעו (יתקבל A או B או כלום), ייתכן שהן יגיעו בסדר הפוך (יתקבל B,A), אך מה שמגיע הוא ללא שגיאות (לא יתקבל C)
- (TCP (Transmission Control Protocol
  - מונחה שיחה (session-oriented)
  - תומך רק ב unicast
  - שירותים נרחבים יותר
    - אמינות: ההודעות שנשלחו יגיעו (אלא אם כן הרשת נפלה, או התהליכים נסגרו), ובסדר שבו הן נשלחו, ללא שגיאות (אם נשלח A,Bיתקבל בצד השני A,B)
    - בקרה על קצב שליחת ההודעות (flow control) בהתאם לעומס ברשת, ולמספר ההודעות שלא טופלו עדיין.
    - ...

בשורה התחתונה, כאשר אנחנו כותבים אפליקציה/תוכנית, ומעוניינים לשלוח מידע לתהליך אחר, אנחנו יכולים לבקש משכבת הטרנספורט socket על פי אחד משני סוגי הקשרים, TCP או UDP.
(ברוב המקרים נבחר TCP, בשל אמינותו, אך קיימות כמה וכמה אפליקציות העושות שימוש ב UDP, הפחות כבד, לדוגמא: שידור סרט, שליחת חיווי שהתהליך עדיין 'חי' לקבוצת תהליכים המעוניינת במידע זה)

4.3 עיצוב אפליקציית שרת-לקוח (מעל שכבת הטרנספורט)

4.3.1 מימוש שרת הדפסה ב TCP ו UDP

נממש תוכנית Client-Server להדפסת מחרוזות על המסך – הלקוח שולח מחרוזת לשרת הוא מדפיס אותה – עבור שני סוגי הקשרים: TCP, UDP

UDP

```
class UDPPrintClient {
       public static void main(String[] args) throws Exception {
             // 1. Get the address (host:port) of the server
             String serverIP = args[0];  // 132.4.5.6
             int serverPort = Integer.parseInt(args[1]); // 7000

            // 2. Define the [udp] interface of the server (the 'Mail
Box')
             DatagramSocket socket = new DatagramSocket();
          // 3. Define a packet for sending (the 'letter')
          String msg = "Hello World";   // the content
          DatagramPacket packet = new DatagramPacket(msg.getBytes(),
msg.getBytes().length,
              InetAddress.getByName(serverIP), serverPort ); // (the
'envelope')
             // 4. Send the packet to the server ('put the envelope in the
mail box')
           socket.send(packet);
           socket.close();
       }
}
class UDPPrintServer {
       public static void main(String[] args) throws Exception {
          // 1. Get the address port of which the server listens
          int port = Integer.parseInt(args[0]); // 7000
          // 2. Define the [udp] interface of the clients (the 'Mail Box')
          DatagramSocket socket = new DatagramSocket(port);
          while (!Thread.currentThread().isInterrupted()) {
            // 3. Define an empty packet for receiving (an empty 'letter')
            //part of the application protocol,limited msg length
            byte[] content = new byte[256];
            DatagramPacket packet =
               new DatagramPacket(content, content.length);
            // 4. Receive a packet content from the client
            socket.receive(packet);
            String msg = new String(content,…);
            System.out.println(msg);
          }
          socket.close();
    }
```

```
}
```

TCP

```
class TCPPrintClient {
       public static void main(String[] args) throws Exception {
             // 1. Get the address (host:port) of the server
             String serverIP = args[0];  // 132.4.5.6
             int serverPort = Integer.parseInt(args[1]); // 7000
             Socket socket = new Socket(serverIP, serverPort);
             String msg = "Hello World";
             for (byte b : msg.getBytes())
                   socket.getOutputStream().write(b);
             // indicate end of message (for the case of TCP)
             socket.getOutputStream().write("\n");
             socket.close();
      }
}

class TCPPrintServer {
       public static void main(String[] args) throws Exception {
             // 1. Get the port of the server
             int port = Integer.parseInt(args[0]); // 7000
             // 2. Define an interface for accepting new client
                   connections ('switchboard')
            ServerSocket serverSocket = new ServerSockert(port);
            while (!Thread.currentThread().isInterrupted()) {
                      // wait for some client connection
                  Socket socket = serverSocket.accept();
                  List<Byte> receivedBytes = new LinkedList<Byte>();
                  while (true) {
                         byte b = socket.getInputStream().read();
                         if (b == '\n')
                            break;
                         else
                             receivedBytes.add(b);
                  }
                  String msg = new String(receivedBytes.toArray(),…);
                  System.out.println(msg);
                  socket.close();
            }

      }
}
```

חסרונות בעיצוב הקיים:

- קידוד המחרוזת לבתים נעשה בצורה ספציפית (ברירת המחדש של getBytes ב Java)
- [עבור TCP] ההחלטה כיצד מסומנים גבולות ההודעות (על ידי תו מפריד של סוף שורה)
- המבנה הפנימי של ההודעה (פרמטר אחד: המחרוזת להדפסה)
- תגובת השרת לקבלת ההודעה (הדפסה על המסך) ספציפית
- מתי נגמרת שיחה עם לקוח (אחרי הודעה אחת)
- אופן הטיפול של השרת במספר לקוחות (על ידי ת'רד אחד, לקוח אחרי לקוח, סדרתית) ספציפית
- ...

□ נגדיר תבנית כללית של שרת, המקבל את כל ההחלטות הנ"ל כפרמטרים.

4.3.2 עיצוב תבנית כללית של שרת

התבנית הכללית של השרת מורכבת מהפשטה של שני ההבטים המרכזיים שצוינו לעיל:
- ה'שפה' שמדבר השרת, הפרוטוקול.
- האופן שבו השרת מנהל מספר שיחות במקביל עם לקוחות שונים, מודל המקביליות של השרת.

4.3.2.1 פרוטוקול

הפרוטוקול מגדיר את נהלי השיחה בין שני תהליכים, כלומר את 'השפה המשותפת' ששניהם דוברים.
מה כולל הפרוטוקול?
לשם כך, נתבונן לרגע ממה מורכבת שפה אנושית:
- האופן שבו צלילים הופכים למילים (מורפולוגיה)
- האופן שבו מילים הופכות למשפט, כולל מבנה המשפט (תחביר)
- המשמעות של המשפט הנאמר (סמנטיקה), כיצד צריך להגיב
- היחס בין משפטים שונים בשיחה (פרגמטיקה, תורת השיח)
- ההחלטה מתי השיחה הסתיימה (פרגמטיקה, תורת השיח)

□ הפרוטוקול קובע:
Encoding – האופן שבו אובייקט ההודעה הופך למערך של בתים.
Framing – האופן שבו הבתים ממוסגרים כ'הודעה' למקבל
Syntax – המבנה הפנימי של ההודעות השונות
Semantics – מה מצופה שיתבצע בעקבות הודעה זו
Synchronization – אילוצי סדר על ההודעות המתקבלות

דוגמאות:

1. השרת הנוכחי
   Encoding – על פי מימוש ה getBytes של Java
   Framing - שורה חדשה כתו מפריד
   Syntax - מבנה פנימי, רק מחרוזת להדפסה

Semantics - משמעות אחת בלבד: להדפיס את ההודעה
Synchronization - אין אילוצים, ניתן לשלוח תמיד מחרוזת להדפסה

2. שרת מחירים (שרת המברר מחירים של מוצר נתון בחנויות שונות)
Encoding – הודעות מסוג מחרוזת, המקודדת לבתים בשיטת UTF-8
Framing – שני הבתים הראשונים בכל הודעה מציינים את האורך שלה
Syntax & Semantics

QUERY product
 [<store price> …]

BUY product store
 SUCCESS/FAIL

Synchronization – לא ניתן לקנות מוצר לפני שבררנו את המחיר שלו

3. http (נהלי השיחה בין שרת רשת ללקוחותיו)
4. Stomp (בתרגיל)

<u>נגדיר הבטים אלו כשני ממשקים</u>:

MessageEncoderDecoder – מטפל ב encoding וב framing

MessagingProtocol – מטפל ב syntax, semantics, synchronization

כך שהשרת יוגדר על בסיס ממשקים אלו.

```
public interface MessageEncoderDecoder<T> {
    byte[] encode(T message);
    T decodeNextByte(byte nextByte);
}
```

ממשק זה קובע כיצד הופכת הודעה למערך בתים, הכולל מנגנון עבור הקורא כיצד לזהות שההודעה הסתיימה - המתודה encode. קורא מערך הבתים יכול לחלץ מהם הודעות בעזרת המתודה decodeNextByte.

נממש ממשק זה עבור פרוטוקול ההדפסה לעיל:

```
public class LineMessageEncoderDecoder implements
MessageEncoderDecoder<String> {

    private List<Byte> bytes = new LinkedList<Byte>();

    @Override
    public byte[] encode(String message) {
        return (message + "\n").getBytes("UTF-8");
```

```
    }

    @Override
    public String decodeNextByte(byte nextByte) {

        if (nextByte == '\n') }
             String ret = new String(Bytes.toArray(bytes),"UTF-8");
             bytes.clear();
             return ret;
{
        bytes.add(nextByte);
        return null;
    }
 }
```

הממשק השני קובע מה נדרש לבצע עבור הודעה נתונה (המתודה process), וכן כיצד יודעים שהשיחה הסתיימה (shouldTerminate):

```
public interface MessagingProtocol<T> {
    T process(T msg);
    boolean shouldTerminate();
}
```

נממש את הממשק עבור פרוטוקול ההדפסה:

```
public class PrintingProtocol implements MessagingProtocol<String> {

    private boolean shouldTerminate = false;

    @Override
    public String process(String msg) {
        if (msg.equals(“bye!”))
             shouldTerminate = true;
        else
            System.out.println(msg);
        return null;
    }

    @Override
    public boolean shouldTerminate() {
        return shouldTerminate;
    }
}
```

על בסיס שני ממשקים אלו, ניתן להגדיר כעת מחלקה המממשת ניהול שיחה עם לקוח נתון ע"פ פרוטוקול נתון - המחלקה ConnectionHandler, המממשת שיחה עם לקוח במתודת ה run שלה:

```
public class ConnectionHandler<T> implements Runnable {
    private final Socket sock;
    private final MessageEncoderDecoder<T> encdec;
    private final MessagingProtocol<T> protocol;


   public ConnectionHandler(Socket sock,
         MessageEncoderDecoder<T> reader, MessagingProtocol<T> protocol) {
        this.sock = sock;
        this.encdec = reader;
        this.protocol = protocol;
    }

    @Override
    public void run() {
           int b;
            while (!protocol.shouldTerminate() &&
                   (b = sock.getInputStream().read()) >= 0) {
                T msg = encdec.decodeNextByte((byte)b);
                if (msg != null) {
                    T ans = protocol.process(msg);
                    if (ans != null)
                        sock.getOutputStream().write(encdec.encode(ans));
                }
            }
       }
}
```

כעת ניתן להגדיר את תוכנית השרת, ע"פ הדפוס של: חיבור לקוח, הגדרת ConnectionHandler לניהול שיחה עם הלקוח, תוך הפעלת מתודת ה run שלו, וחוזר חלילה.
דפוס זה ממומש במתודת ה serve:

```
class BaseServer {

    private final int port;
    private final Supplier<MessagingProtocol> protocolFactory;
    private final Supplier<MessageEncoderDecoder> encdecFactory;

    public BaseServer(
            int port,
```

```
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encdecFactory)
{

        this.port = port;
        this.protocolFactory = protocolFactory;
        this.encdecFactory = encdecFactory;
    }

    public void serve() {
           ServerSocket serverSock = new ServerSocket(port);
           while (!Thread.currentThread().isInterrupted()) {
                Socket clientSock = serverSock.accept();
                ConnectionHandler handler = new ConnectionHandler(
                        clientSock,
                        encdecFactory.get(),
                        protocolFactory.get());
                handler.run();
            }
    }
}
```

נריץ את תבנית השרת עבור שרת ההדפסה:

```
class TCPPrintServer {
       public static void main(String[] srgs) {
              int port = Integer.parseString(args[0]);
              new BaseServer(port,
                            () -> new PrintingProtocol(),
                            () -> new LineMessageEncoderDecoder())
               .serve();
       }
}
```

4.3.2.2 מודל המקביליות של השרת

ציינו כבר קודם, כי הרכיב השני של תבנית השרת הוא האופן שבו השרת מנהל במקביל שיחה עם מספר לקוחות.
בקוד הקיים של BaseServer השרת מדבר כל פעם עם לקוח אחד בלבד (על ידי קריאה למתודת ה run של ה ConnectionHandler שלו). יש להפוך החלטה זו לפרמטר של השרת הנתון לבחירה על ידי המשתמש.
- נגדיר את אופן ניהול השיחה עם לקוח במקביל ללקוחות אחרים, בהינתן ה ConnectionHandler של הלקוח, כמתודה אבסרקטית execute, וניממש אותה באופנים שונים, על פי מודלים שונים של מקביליות.

```
abstract class BaseServer {

    private final int port;
    private final Supplier<MessagingProtocol> protocolFactory;
    private final Supplier<MessageEncoderDecoder> encdecFactory;

    abstract protected void execute(ConnectionHandler handler);

    public BaseServer(
            int port,
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encdecFactory)
{

        this.port = port;
        this.protocolFactory = protocolFactory;
        this.encdecFactory = encdecFactory;
    }

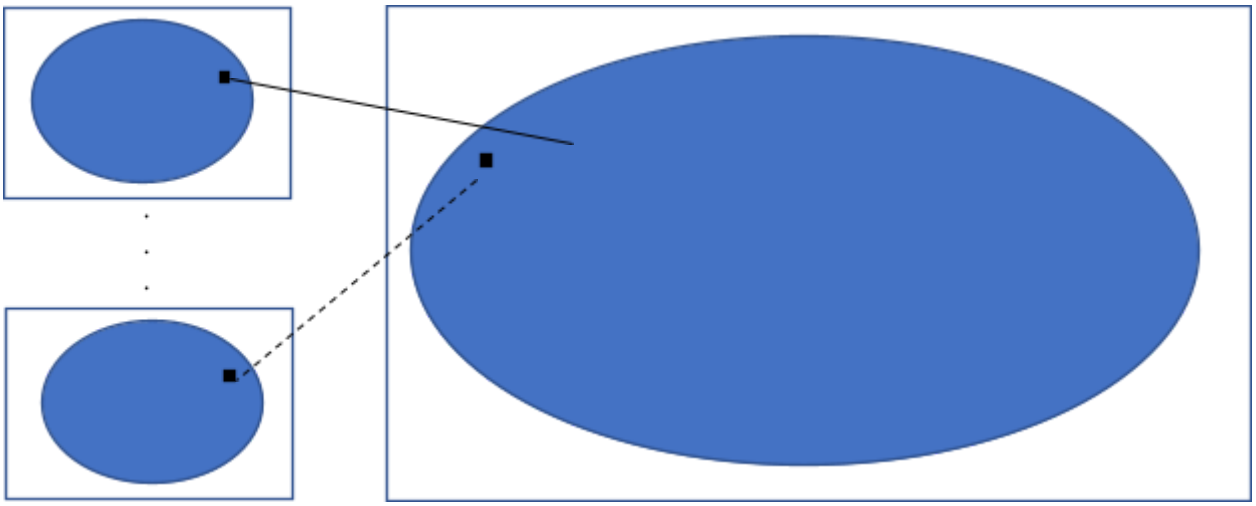
    public void serve() {
           ServerSocket serverSock = new ServerSocket(port);
           while (!Thread.currentThread().isInterrupted()) {
                Socket clientSock = serverSock.accept();
                ConnectionHandler handler = new ConnectionHandler(
                        clientSock,
                        encdecFactory.get(),
                        protocolFactory.get());
                execute(handler);
            }
    }
}
```

נבחן מימושים אפשריים שונים של מתודת ה execute, כלומר מימושים שונים של מודל המקביליות של השרת:

1. Single Thread

השרת מבוסס על ת'רד אחד. ת'רד זה ממתין להתחברות לקוח, משרת אותו עד תום השיחה, וחוזר חלילה.

```
class SingleThreadServer extends BaseServer {
      protected void execute(ConnectionHandler handler) {
            handler.run();
      }
}
```



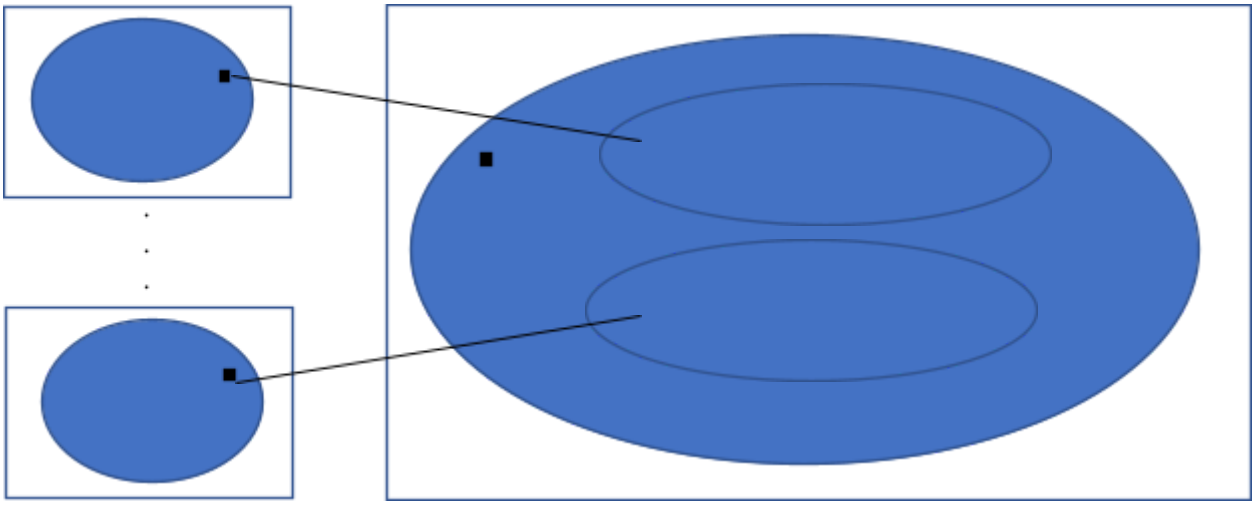
יתרון: פשטות, לא דורש משאבים

חיסרון: זמינות, רק לקוח אחד מקבל שירות בכל נקודת זמן.

עשוי להתאים לשרת המממש פרוטוקול פשוט ביותר כמו TimeServer, כאשר אין מספר גדול שלקוחות בכל נקודת זמן. אך בדרך כלל הוא לא מספק.

2. ת'רד לכל לקוח

```
class ThreadPerClientServer<T> extends BaseServer<T> {
	protected void execute(ConnectionHandler<T> handler) {
		new Thread(handler).start();
	}
}
```



יתרון: זמינות מוחלטת (לכאורה), לכל לקוח יש ת'רד משלו המטפל בו עד תום השיחה.

חסרונות: אין שימוש חוזר בת'רדים קיימים (לאחר שת'רד מסיים לטפל בלקוח הוא מסיים את חייו, כך שעבור הלקוח הבא ייווצר ת'רד חדש), סקלביליות – עבור כמות גדולה של לקוחות, השרת יקרוס מחוסר יכולת לייצר ת'רדים נוספים.

מודל שכזה מתאים רק למקרים שבהם מספר הלקוחות חסום ואינו גדול יותר מדי (ביחס למשאבי החומרה ומערכת ההפעלה). אך באופן כללי, פחות מעשי.

3. מאגר ת'רדים

גישה זו מאזנת בין הזמינות והסקלביליות, תוך שימוש חוזר בת'רדים קיימים, על ידי הגדרת מאגר קבוע של ת'רדים, הרצים כל הזמן, ומטפלים כל פעם בלקוח בראש התור.
כמות הת'רדים במאגר תיקבע בהתאם לכמות הצפויה של הלקוחות, ובהתאם למשאבי השרת (חומרה, מערכת הפעלה). מספר הת'רדים במאגר עשוי להשתנות בהתאם לתנאים, אך בכל מקרה הוא קבוע, במובן שלא מייצרים ת'רד לכל לקוח חדש.
ניתן לממש מאגר קבוע שכזה של ת'רדים, על פי סכמת הקוד הבאה:

```
class ThreadPool {
     BoundedQueue<Runnable> taskQueue;
     List<Thread> threads;
     ThreadPool(int numThreads) {
            taskQueue = new BlockingQueue<Runnable>();
            threads = new LinkedLIst<Thread>();
           for (int i=0; i<numThreads; i++) {
               Thread t = new Thread(() ->
                  while (!Thread.interrupted()) taskQueue.remove().run());
               threads.add(t);
               t.start();
           }
     }

     public void submit(Runnable task) { taskQueue.add(task); }

     public void shutdown() {
          for (Thread t : threads)
               t.interrupt();
     }
}
```

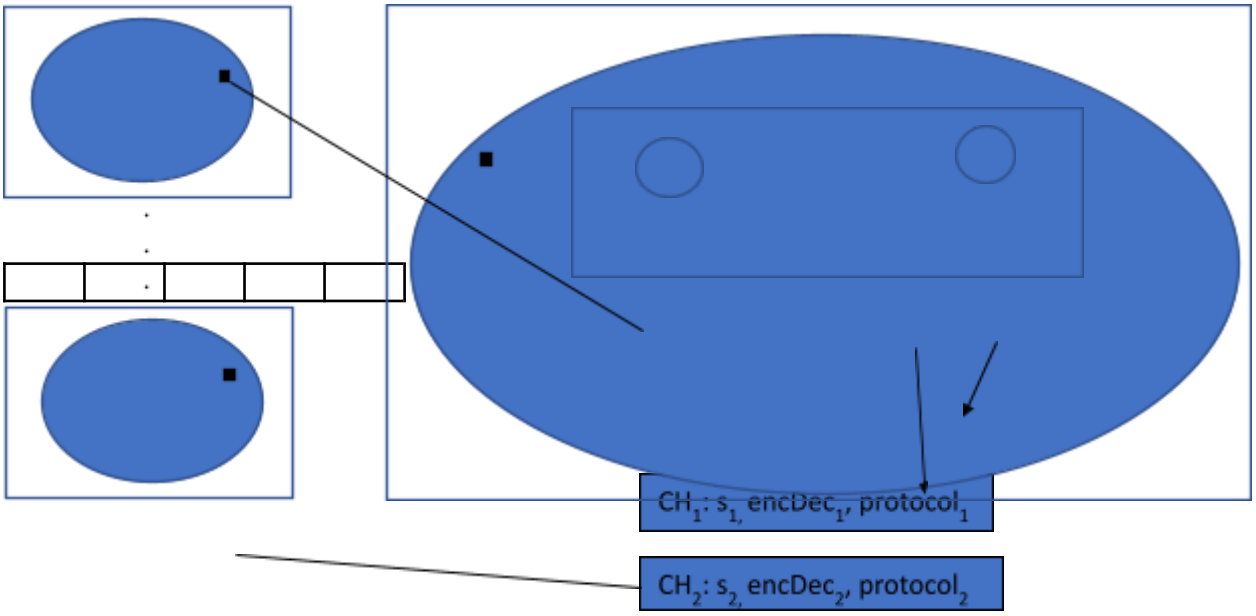
מאגר ת'רדים שכזה ממומש כיום ב JDK הסטנדרטי של Java – המחלקה ExecutorService.

```
class FixedThreadPoolServer<T> extends BaseServer<T> {
      ExecutorService pool;

      FixedThreadPoolServer(int port,
```

```
            Supplier<MessagingProtocol> protocolFactory,
            Supplier<MessageEncoderDecoder> encdecFactory)
            int numThreads)
     {
            super(port, protocolFactory, encdecFactory);
            pool = Executors.newFixedThreadPool(numThreads);
    }

    protected void execute(ConnectionHandler<T> handler) {
            pool.submit(handler);
    }
}
```



יתרונות: זמינות מבוקרת בהתאם ליכולות של המערכת, תוך שימוש חוזר בת'רדים קיימים.

חסרונות: ניצולת, הוגנות.

דוגמא להמחשה: מתן השירות בסניף הבנק, העובדת על פי מודל ג' של מאגר ת'רדים/פקידים.

ניצולת – מאחר שהמשימה המוטלת על הת'רדים (מתודת ה run שהוגדרה עבורם ב ConnectionHandler) היא ניהול שיחה עם הלקוח הכולל גישה חוזרת ונשנית ל i/o (בעיקר קריאה וכתיבה ל socket), הת'רדים עוברים באופן תכוף למצב blocked למרות שיש לקוחות אחרים בממתינים לשירות בתור המשימות של מאגר הת'רדים. כך שייתכן מצב שבו יש מעבד פנוי שאינו מבצע דבר, למרות שיש לקוחות רבים בתור.

הוגנות – מודל המקביליות של השרת מעוצב כך, שקבוצה קטנה של לקוחות מקבלת שירות מוחלט (עד שהם יחליטו שהשיחה הסתיימה) בעוד שאר הלקוחות לא מקבל שירות כלל.

מודל זה, מצוי בחיים האמיתיים, אצל כל המערכות שנותנות שירות לקוחות. המודל הרביעי שנציג להלן, מתעלה על מודל זה ופותר את שני החסרונות של ההוגנות וניצולת המעבד.

4. מודל הריאקטור

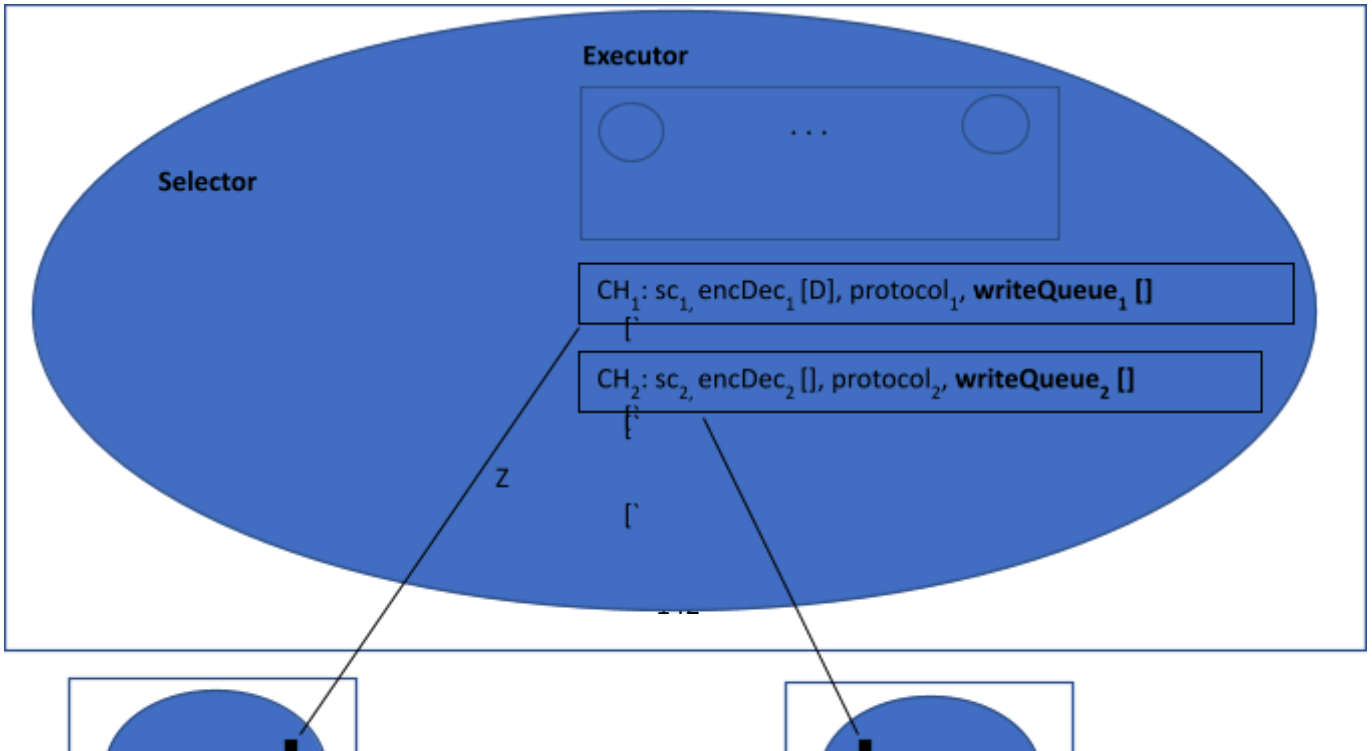
כפי שציינו, מודל הריאקטור מתמודד עם בעיית ההוגנות וניצלות המעבד:

קטגורית:

הוגנות – נצמצם את המשימה הניתנת כל פעם לת'רד במאגר מניהול שיחה שלמה על לקוח, לביצוע הודעה אחת של לקוח (או מספר מצומצם של הודעות).

ניצולת – נפריד בין ניהול התקשורת עם הלקוחות לבין ביצוע בקשות על פי הפרוטוקול. כלומר, אחד הת'רדים ינהל את כל התקשורת עם הלקוחות (ובפרט, יקרא מהם בתים ויכתוב אליהם בתים), וכל שאר הת'רדים יבצעו בקשות של לקוחות בזיכרון.

בעיה: קיים עומס רב על ת'רד התקשורת:
- צריך לקבל לקוחות חדשים (לבצע accept על ה ServerSocket)
- צריך לקרוא בתים מלקוחות מחוברים (ביצוע read על ה InputStream ב Socket שלהם)
- צריך לכתוב בתים ללקוחות מחוברים (ביצוע write על ה OutputStream ב Socket שלהם)
הבעיה מחריפה לאור העובדה, שכל ניסיון של קריאה/כתיבה/קבלה מול לקוח אינו מסתיים עד אשר הוא מצליח (לחבר לקוח, לקרוא/לכתוב לפחות בית אחד), זאת כאשר יש לנהל תקשורת עם לקוחות נוספים.
- נשנה את ערוצי הקלט/הפלט/ההתחברות ל non-blocking i/o. במימוש זה, פעולות אלו מסתיימות מיד: קוראים מה שיש כרגע (אם אין כלום חוזר 0), כותבים מה שאפשר כרגע (אם לא ניתן חוזר 0), מקבלים את מי שמתחבר כרגע (אם אין מישהו שמתחבר חוזר null)
עדיין, יהיה קשה מאוד לת'רד התקשורת לרוץ בלולאה על מספר רב של לקוחות, ולבדוק כל הזמן האם יש מה לקרוא מהם / לכתוב אליהם / לקבל אותם.
- נצייד את ת'רד התקשורת במכשיר, המכונה Selector, השולח התרעה כאשר נדרש לקבל/לקרוא/לכתוב אצל אחד הלקוחות.
באופן זה, ת'רד תקשורת יחיד יוכל לעמוד במעמסה של ניהול התקשורת עם כל הלקוחות, תוך השארת ביצוע ההודעות בזיכרון לת'רדי העבודה במאגר הת'רדים.





תיאור הרכיבים + קוד

- מחלקות עזר מהספריה nio של Java

Channel, SelectableChannel

ServerSocketChannel, SocketChannel, [DatagramSocketChannel]

אירועי I/O המתרחשים ב-SelectableChannel מזוהים על ידי מערכת ההפעלה / ה-Selector:

- אירוע ACCEPT מתרחש כאשר שכבת הטרנספורט מחברת לקוח חדש.
- אירוע READ מתרחש כאשר בבאפר הבתים הנכנסים יש לפחות byte אחד.
- אירוע WRITE מתרחש כאשר בבאפר הבתים היוצאים יש לפחות תא פנוי אחד.

Selector, SelectionKey

ByteBuffer

- מחלקת ה Reactor, המחלקה המרכזית

<u>שדות מרכזיים</u>:

Selector

Executor

readerFactory

protocolFactory

<u>אתחול</u>:

הגדרת ה Selector וה Executor

הגדרת ServerSocketChannel, במוד non-blocking, על הport נדרש

רישום ה ServerSocketChannel ב Selector עבור אירועי ACCEPT (ללא אובייקט מצורף).

<u>מתודות</u>:

Serve

מבוצעת על ידי ת'רד התקשורת

לולאה נצחית:

- המתנה עד אשר יתרחש אירוע כלשהו באחד הערוצים בסלקטור
- עבור כל ערוץ שאירע בו אירוע כלשהו: טיפול באירוע על פי הסוג שלו - קבלה, קריאה, כתיבה

handleAccept

- מבוצעת על ידי הת'רד של התקשורת כאשר יש אירוע OP_ACCEPT
- ביצוע accept על ה ServerSocketChannel
- רישום ה SocketChannel החוזר ב Selector עבור אירוע OP_READ עם אובייקט מצורף - ConnectionHandler שנוצר עבור לקוח זה.

handleRead

- מבוצעת על ידי הת'רד של התקשורת כאשר יש אירוע OP_READ
- קריאה למתודת continueRead ב ConnectionHandler של הלקוח, והוספת המשימה החוזרת לתור המשימות ב Executor

handleWrite

- מבוצעת על ידי הת'רד של התקשורת כאשר יש אירוע OP_WRITE
- קריאה למתודה continueWrite ב ConnectionHandler של הלקוח.

updateInterestOps

- מבוצעת על ידי הת'רד של התקשורת כאשר הוא משנה את הרישום מ OP_READ | OP_WRITE ל OP_READ (לאחר שנכתבו כל התשבות ללקוח וה writeQueue שלו ריק)
- מבוצעת על ידי ת'רד עבודה (מה Executor) לאחר שהוא מוסיף תשובה ל writeQueue של הלקוח, שינוי הרישום ל OP_READ | OP_WRITE
- שינוי רישום האירועים להתראות ב Selector

```
public class Reactor {

    private int port;
    private Supplier<MessagingProtocol<T>> protocolFactory;
    private Supplier<MessageEncoderDecoder<T>> readerFactory;
    private ExecutorService pool;
    private Selector selector;

    …

    public void serve() {
             // Initialization
            // 1. Create ServerSocket channel on the given port, in
non-blocking   mode
            ServerSocketChannel serverSock = ServerSocketChannel.open();
            serverSock.bind(new InetSocketAddress(port));
            serverSock.configureBlocking(false);
             // 2. Registr the socket to the Selector for ACCEPT event
```

```
            serverSock.register(selector, SelectionKey.OP_ACCEPT);

            // Main loop
            while (!Thread.currentThread().isInterrupted()) {
                selector.select();  // wait for some event (in ‘blocked’
state)
                for (SelectionKey key : selector.selectedKeys()) {    //
for each event
                    if (!key.isValid()) {
                        continue;
                    } else if (key.isAcceptable()) {
                        handleAccept(serverSock, selector);
                    } else {  // read and/or write event
                        handleReadWrite(key);
                    }
                }
                selector.selectedKeys().clear(); //clear the selected keys
set
            }
        pool.shutdown();
    }

    private void handleAccept(ServerSocketChannel serverChan,
                              Selector selector)
           throws IOException {

        SocketChannel clientChan = serverChan.accept();
        clientChan.configureBlocking(false);
        final NonBlockingConnectionHandler handler =
           new NonBlockingConnectionHandler(
                readerFactory.get(),
                protocolFactory.get(),
                clientChan,
                this);
        clientChan.register(selector, SelectionKey.OP_READ, handler);
    }

    private void handleReadWrite(SelectionKey key) {
        // Get the attached ConnectionHandler
        NonBlockingConnectionHandler handler =
(NonBlockingConnectionHandler) key.attachment();
        // Case 1: read event
        if (key.isReadable()) {
            Runnable task = handler.continueRead();
            if (task != null)
                pool.submit(task);
        }
        // Case 2: write event
        if (key.isWriteable())
            handler.continueWrite();
```

```
    }

    void updateInterestedOps(SocketChannel chan, int ops) {
            SelectionKey key = chan.keyFor(selector);
            key.interestOps(ops);
            selector.wakeup();
        }
    }
}
```

- המחלקה NonBlockingConnectionHandler

<u>שדות</u>:

Socket**Channel**
encDec
protocol
Queue<ByteBuffer> writeQueue

<u>מתודות</u>:

continueWrite

- **o** מבוצעת על ידי הת'רד של התקשורת כאשר יש אירוע OP_WRITE
- **o** הוצאת ByteBuffers מה writeQueue וכתיבתם ל SocketChannel, עד אשר לא ניתן יותר לכתוב (הכל ב non-blocking)
- **o** אם לא נשארו יותר בתים ב writeQueue החזרת הרישום ב Selector ל OP_READ בלבד

continueRead

- o מבוצעת על ידי הת'רד של התקשורת כאשר יש אירוע OP_READ
- o קריאת ByteBuffer מה SocketChannel של הלקוח, ע"י ביצוע המתודה read
- o החזרת משימת טיפול במערך הבתים, כאובייקט Runnable, עם משימת ה run הבאה:
  - מבוצעת על ידי ת'רד עבודה (מה Executor)
  - שדות: ByteBuffer, encDec, protocol, writeQueue
  - פעולה
    - חילוץ הודעות מה ByteBuffer בעזרת ה encDec (בית אחרי בית)
    - ביצוע כל הודעה ע"פ ה protocol
    - העברת התשובה של כל הודעה כ ByteBuffer ל writeQueue, תוך עדכון הרישום ב Selector של ערוץ הלקוח ל READ | WRITE

```
public class NonBlockingConnectionHandler {

    private static final int BUFFER_ALLOCATION_SIZE = 1 << 13; //8k
    private final MessagingProtocol<T> protocol;
```

```
    private final MessageEncoderDecoder<T> encdec;
    private final Queue<ByteBuffer> writeQueue = new
LinkedList<ByteBuffer>();
    private final SocketChannel chan;
    private final Reactor reactor;

    …

   public Runnable continueRead() {
        ByteBuffer buf =
ByteBuffer.allocateDirect(BUFFER_ALLOCASTION_SIZE);

    if (chan.read(buf) > 0) {  // 1. Read bytes from the client's channel
        buf.flip();
        return () -> {
//2. Return a task for the executor, based on the bytes read from the
client
            while (buf.hasRemaining()) {
              T nextMessage = encdec.decodeNextByte(buf.get());
               if (nextMessage != null) {
                  T response = protocol.process(nextMessage);
                  if (response != null) {
                        writeQueue.add(ByteBuffer.wrap(
                              encdec.encode(response)));
                        reactor.updateInterestedOps(chan,
                         SelectionKey.OP_READ | SelectionKey.OP_WRITE);
                  }
               }
          }
        };
      } else {  // read byte is -1 ◊ client disconnected
     // (0 is not possible, since the read is applied after OP_READ event)
            close();
            return null;
        }
    }

    public void close() {
        try {
            chan.close();
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    public void continueWrite() {
        while (!writeQueue.isEmpty()) {
                ByteBuffer top = writeQueue.peek();
                chan.write(top);
                if (top.hasRemaining())
```

```
                    return;
                else
                    writeQueue.remove();
            }
            // the while-loop can be break due exception
            // (see the full code in the website)
            if (writeQueue.isEmpty()) {
                if (protocol.shouldTerminate()) close();
                else reactor.updateInterestOps(chan, SelectionKey.OP_READ);
            }
        }
    }
```

נבחן מספר 'סוגיות' סביב התבנית הנוכחית:

סנכרון

מאחר שמדובר במערכת מקבילית, המבוססת על מספר ת'רדים הרצים במקביל, נדרש לבחון תרחישים שונים בהם שני ת'רדים ניגשים בו זמנית למידע משותף.

ניתן קטגורית לזהות שתי נקודות חיכוך בין ת'רדים:

1. ת'רד התקשורת ות'רד עבודה (מה Executor)

ה writeQueue ב NonBlockingConnectionHandler, המאחסן תשובות למשלוח עבור לקוח מסוים, נגיש בו זמנית לת'רד התקשורת ולת'רד העבודה: בזמן שת'רד התקשורת מסיר ByteBuffer שנשלח ללקוח דרך ה Socket שלו מה writeQueue, ייתכן שת'רד העבודה מוסיף תשובה ללקוח כ ByteBuffer חדש ל writeQueue של אותו לקוח.
ה writeQueue צריך להיות מסונכרן.

```
public class NonBlockingConnectionHandler {

   ...
   private final Queue<ByteBuffer> writeQueue =
            new ConcurrentLinkedQueue<ByteBuffer>();
    …
}
```

כזכור, סנכרון מחלקה מבטיח הפעלה בטוחה של כל אחת מהמתודות בפני עצמה, אך אם יש קשר לוגי בין מספר פעולות על המבנה (כמו ביצוע if המתייחס למצבו ולאחר מכן ביצוע פעולה בהתאם) נדרש עדיין סנכרון מבחוץ על רצף הפעולות.
קיימים שני מקרים כאלה במתודה continueWrite:

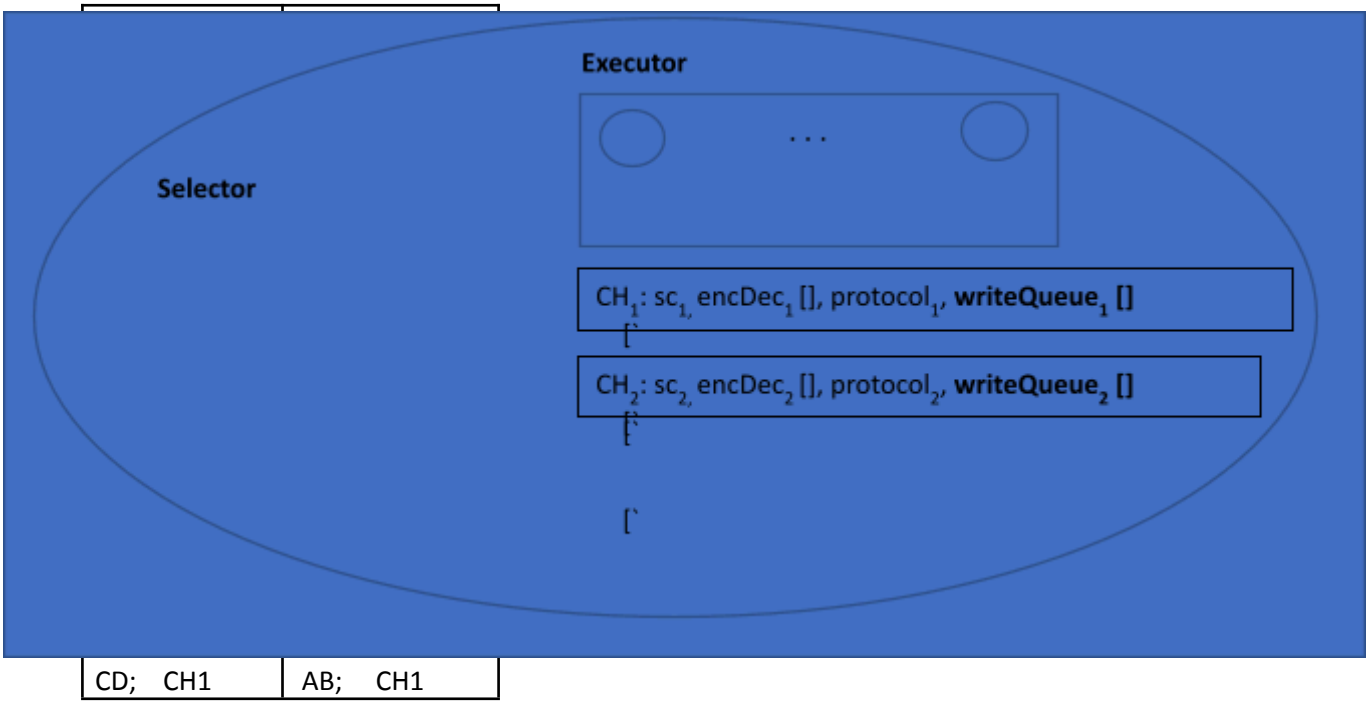
```
while (!writeQueue.isEmpty()) {
   // take bytes from writeQueue and write them to the client’s socket
}
```

אם ת'רד העבודה יוסיף תשובה חדשה ללקוח זה, בין בדיקת התנאי לבין היציאה מהלולאה, זה לא נורא. כי הרישום של הלקוח יהיה בעקבות כך READ | WRITE כך שת'רד התקשורת יילך לישון ב select הבא ויתעורר מיד לאחר שהלקוח יהיה זמין לכתיבה, ויגיע שוב ל continueWrite

```
if (writeQueue.isEmpty()) {
        // change the registration to READ only
}
```

כאן עשויה להיות בעיה: ייתכן שלאחר הבדיקה שהתור ריק, ת'רד עבודה הוסיף תשובה ללקוח זה תוך שינוי הרישום ל READ | WRITE, בעוד ת'רד התקשורת דורס את הרישום ל READ בלבד. באופן זה, הת'רד של התקשורת לא יתעורר מה select עד אשר יגיעו בתים מהלקוח (לא בטוח שזה בכלל יקרה, כי הלקוח מחכה מן הסתם לתשובה לפני שהוא שולח בקשה נוספת), ובכל מקרה לא יטפל באירוע של WRITE.
מן הדין היה לסנכרן את בלוק ה if הזה, אך נראה בהמשך שתרחיש זה ייפתר בדרך אחרת (בעקבות מנגנון שנוסיף לתבנית מסיבה אחרת)

2. שני ת'רדים של עבודה (מה Executor)
תרחיש זה אפשרי, כאשר מתקבלים כמה ByteBuffer מהלקוח בקריאות שונות, המטופלים באופן מקביל על ידי שתי משימות ב Executor:
- ייתכן והבתים של ההודעות יתערבבו ב encDec המשותף ב CH של הלקוח.



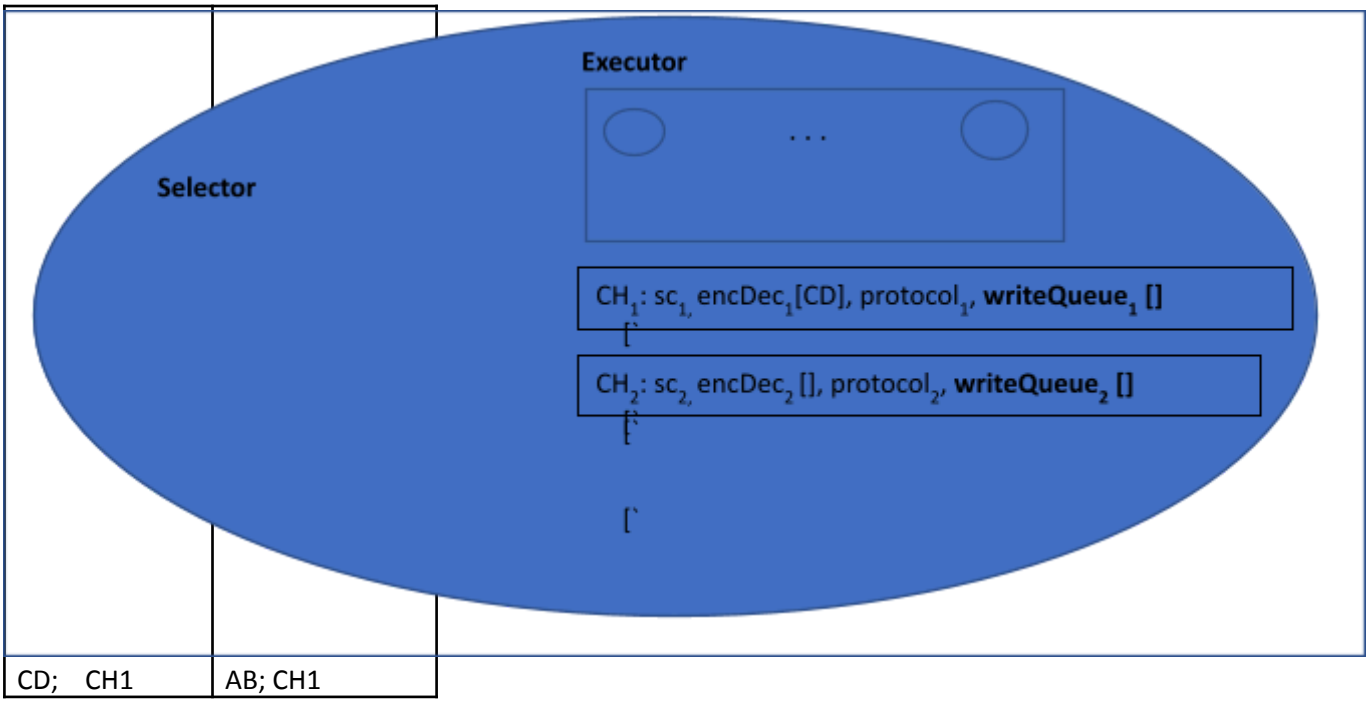
יש לסנכרן את הגישה המשותפת ל'תיק' הלקוח (ל CH שלו)

<u>ניסיון ראשון</u>: נסנכרן את ה CH במתודת ה run של משימת ת'רד העבודה

```
public Runnable continueRead() {
            ...
            return () -> {//2. Return a task for the executor, based on
the bytes read from the client
              synchronized (NonBlockingConnectionHandler.this) {
                while (buf.hasRemaining()) {
                   T nextMessage = encdec.decodeNextByte(buf.get());
```

```
                    if (nextMessage != null) {
                        T response = protocol.process(nextMessage);
                        if (response != null) {
                            writeQueue.add(ByteBuffer.wrap(
                                                encdec.encode(response)));
                            reactor.updateInterestedOps(chan,
                              SelectionKey.OP_READ | SelectionKey.OP_WRITE);
                          }
                      }
                  }
              }
          };
          …
    }
```

חיסרון: עדיין אין שמירה על ביצוע ההודעות על פי סדר הבתים שנשלחו. גם אם הבתים אמנם ייקראו על פי סדרן (פרוטוקול TCP) ואף יגיעו על פי סדרן כמשימות ל Executor (ת'רד תקשורת אחד), ואף יילקחו מתור המשימות ב Executor על פי סדרן על ידי ת'רדים של עבודה, מכאן מאילך אין כל הבטחה לתזמון שבו הת'רד שלקח את מערך הבתים הראשון יבצע את משימתו לפני הת'רד שלקח את מערך הבתים השני.
יתרה מכך, קיים תרחיש שבו השרת יבצע הודעות שלא נשלחו כלל על ידי הלקוח, ומצד שני לא יבצע הודעות שכן נשלחו: אם ה-ByteBuffer הראשון הוא AB;C והשני הוא D;. אם הת'רד שלקח את המשימה השניה יתוזמן ראשון, הוא יבצע את ההודעה D שלא נשלחה, וההודעה CD לא תתבצע.



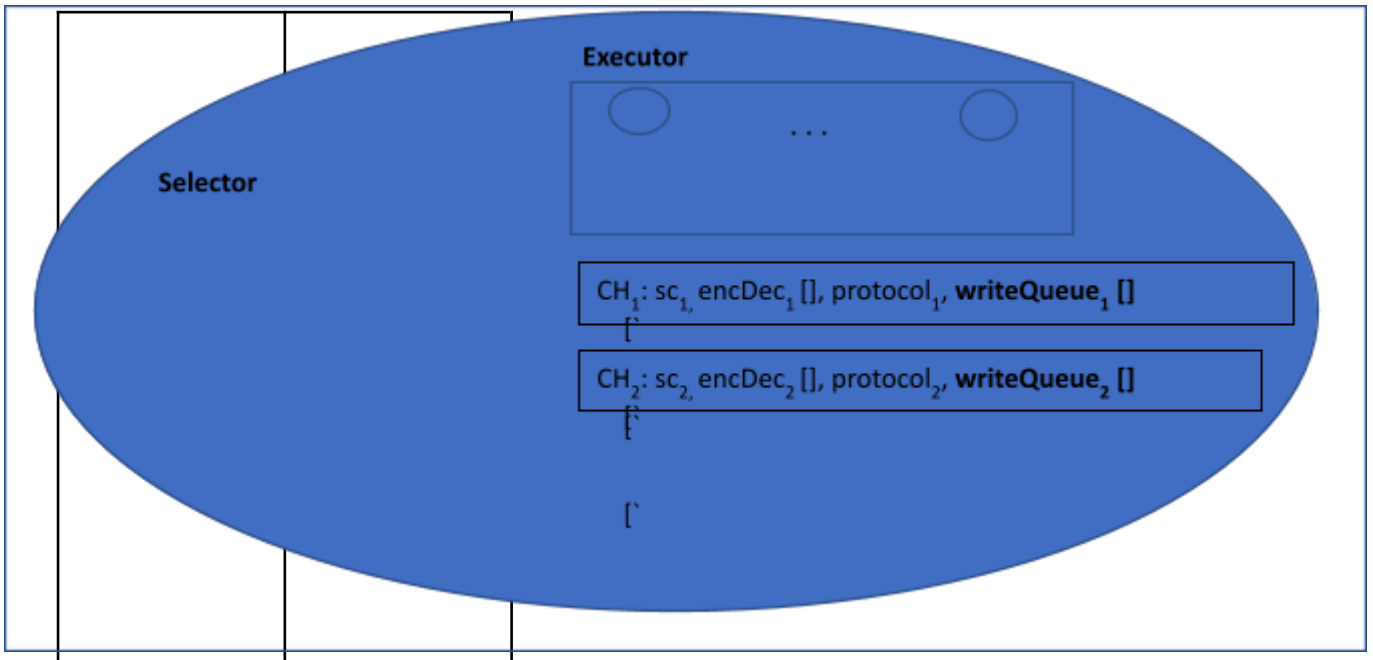
ניסיון שני: הרחבת סנכרון הלקוח, ה CH, כך שיכלול גם את ההסרה של משימותיו מהתור.
לשם כך, נתחזק מבנה נתונים הממפה CH לרשימת המשימות שיש לבצע עבורו. ונגדיר את המשימה עבור הת'רדים ב executor כהסרה, תחת סנכרון ה CH, של המשימה הראשונה עבור הלקוח, וביצועה (הרצת ה run שלה)

```
class Reactor {
   ...
   private Map<NonBlockingConnectionHandler, Queue<Runnable>>
    handler2tasks =
       new ConcurrentHashMap
           <NonBlockingConnectionHandler, Queue<Runnable>>();
    ..

   private void handleAccept(ServerSocketChannel serverChan,
                             Selector selector)
     throws IOException {
        SocketChannel clientChan = serverChan.accept();
        clientChan.configureBlocking(false);
        final NonBlockingConnectionHandler handler =
           new NonBlockingConnectionHandler(
                readerFactory.get(),
                protocolFactory.get(),
                clientChan,
                this);
        clientChan.register(selector, SelectionKey.OP_READ, handler);
        handler2tasks.put(handler, new ConcurrentLinkedQueue<Runnable>());
    }

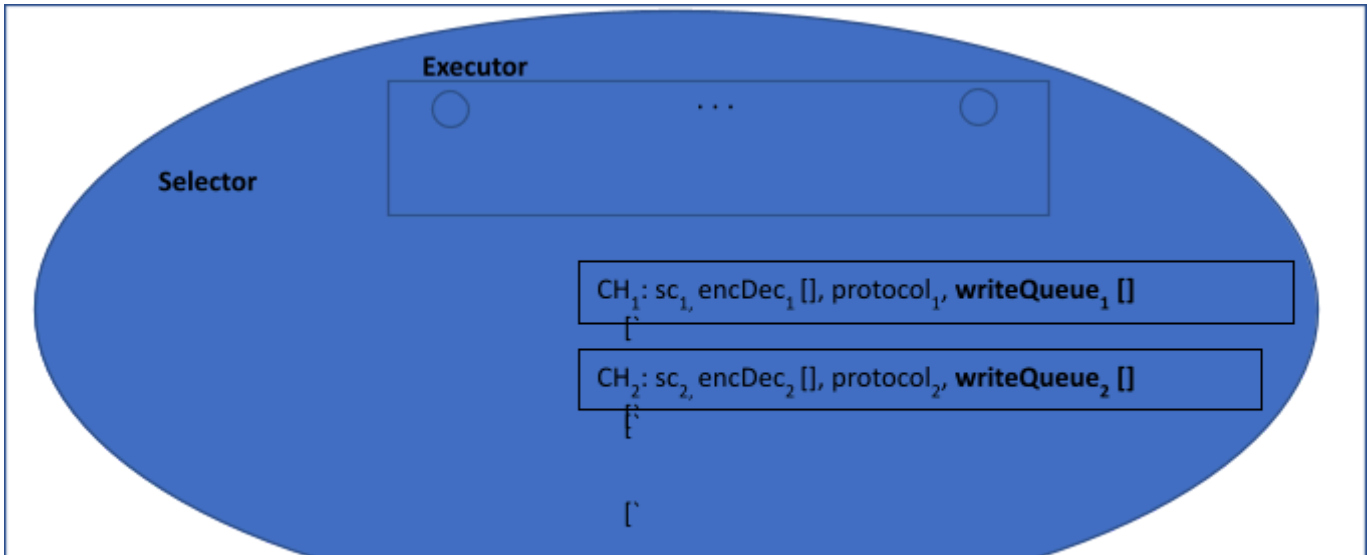
    private void handleReadWrite(SelectionKey key) {
        NonBlockingConnectionHandler handler =
              (NonBlockingConnectionHandler)key.attachment();
        if (key.isReadable()) {
            Runnable task = handler.continueRead();
            if (task != null) {
                pool.submit(task);
                handler2tasks.get(handler).add(task);
                pool.submit(() -> {
                     synchronized(handler) {
                        tasks.remove(0).run();
                     }
                });
            }
        } else {
            handler.continueWrite();
        }
    }
```

המשימה המוגדרת לת'רדים ב executer היא: להוריד, תחת סנכרון של ה CH, את המשימה הראשונה ברשימת המשימות של הלקוח ולבצע אותה (להריץ את ה run שלה). באופן זה, אין חשיבות לתזמון הת'רדים ב executor, מי שירוץ ראשון ייקח את המשימה הראשונה ויבצע אותה, מבלי לאפשר לת'רדים אחרים לבצע משימות אחרות של אותו לקוח.



Synch, remove, run	Synch, remove, run
CH1	$T_{1,1}$ $T_{1,2}$
CH2	

חיסרון: מבטיח את הנכונות, אך לא יעיל מבחינת liveness: ביצוע במקביל של מספר משימות של אותו לקוח, יעביר את כל הת'רדים חוץ מאחד להמתנה, למרות שיש לקוחות אחרים שניתן לטפל בהם בזמן זה.



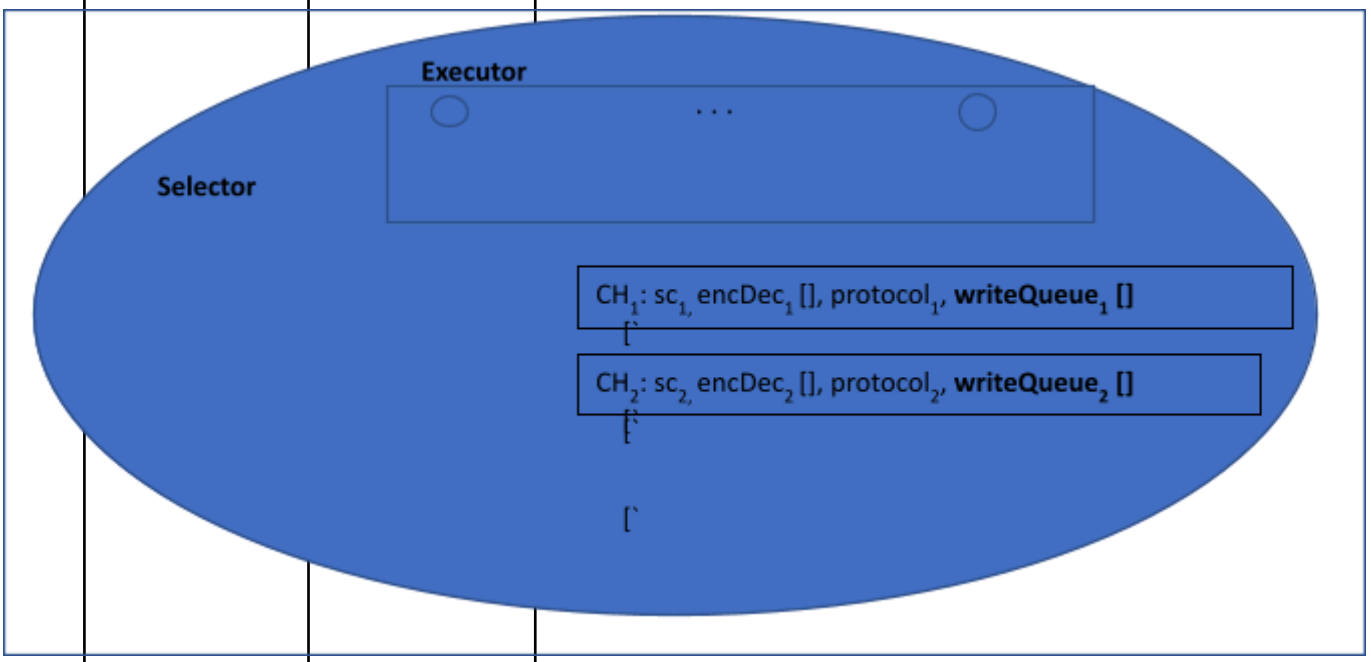
CH_2: synch, rem, run	CH_1: synch rem, run	CH_1: synch, rem, run
CH_1	$T_{1,1}$ $T_{1,2}$	
CH_2	$T_{2,1}$	

<u>ניסיון שלישי</u>: ניהול ביצוע משימות הלקוחות ב Executor כך שלא יבוצעו בו זמנית שתי משימות של אותו לקוח. כלומר, מגדיר פרוטוקול פשוט, ה'מטפטף' משימות של לקוחות מהטבלה שבה הם נשמרים לרשימת המשימות של ה Executor, באופן שבו לא יופיעו שתי משימות של אותו לקוח בתור המשימות של ה Executor.

נממש זאת באופן כללי (לאו דווקא עבור התרחיש שלנו בהקשר של משימות בשרת עבור ConnectionHandler) בעזרת תבנית ה ActorThreadPool, המממשת ExeccutorService תחת אילוץ מסוג זה:

- מיפוי 'שחקן' (במקרה שלנו לקוחות, CHs) למשימות הנוכחיות שלו Tasks שצריך לבצע
- רשימה המציינת אלו 'שחקנים' (במקרה שלנו לקוחות, CHs) מטופלים כרגע ע"י ת'רדים ב executor
- הוספת משימה של שחקן ל executor תתבצע רק אם אין משימות אחרות שלו בתור המשימות של ה Executor

CH_1: $T_{1,2}$ complete	CH_2: $T_{2,1}$ complete			CH_2: $T_{2,1}$ complete
CH_1				
CH_2				



- המשימה המוגדרת לת'רדים ב executor: א) ביצוע המשימה הראשונה הנוכחית של שחקן נתון; ב) הוספת המשימה הבאה של לקוח זה לסוף תור המשימות. אם אין יותר משימות לשחקן, הוא יוסר מרשימת השחקנים הפעילים

עיצוב: המחלקה ActorThreadPool

submit (T actor, Runnable task)

- הוספה של המשימה task לרשימת המשימות של הלקוח
- אם הלקוח אינו פעיל, כלומר אף משימה שלו אינה מבוצעת על ידי ה executor: נוסיף משימה ל executor המורכבת מביצוע ה run של המשימה task, ולאחריה ביצוע של complete עבור לקוח זה (אשר תוביל להוספת המשימה הבאה של לקוח זה ל executor). הוספת משימה שכזו מוגדרת על ידי המתודה execute.

execute(Runnable task, T actor)

- פונקציית עזרה המוסיפה משימה חדשה ל executor הכוללת:
  - ביצוע מתודה ה run של task נתון
  - קריאה ל complete, אשר תוסיף את המשימה הבאה של הלקוח ל executor

complete(T actor)

- אם אין עוד משימות ל actor, הסרתו מרשימת השחקנים הפעילים (מהרשימה playingNow)
- אחרת, נוסיף משימה חדשה ל executor הכוללת את הביצוע של המשימה הבאה ולאחריה complete (כלומר, קריאה למתודה execute)

```
public class ActorThreadPool<T> {

    private final Map<T, Queue<Runnable>> acts;
    private final Set<T> playingNow;
    private final ExecutorService threads;  // the 'executor'

    public ActorThreadPool(int threads) {
        threads = Executors.newFixedThreadPool(threads);
        acts = new HashMap< T, Queue<Runnable>>();
        playingNow = HashSet<T>();
    }

    public void submit(T act, Runnable task) {
            if (!playingNow.contains(act)) {
                playingNow.add(act);
                // add a new task to the executor: <r.run();
complete(…);>
                execute(task, act);
            } else {
                // add the new task r to the tasks of act
                pendingRunnablesOf(act).add(task);
            }
    }
```

```
    public void shutdown() {
        threads.shutdownNow();
    }

    private Queue<Runnable> pendingRunnablesOf(T act) {
        // get the task list of the given actor
        Queue<Runnable> pendingRunnables = acts.get(act);
        if (pendingRunnables == null)
            acts.put(act, pendingRunnables = new LinkedList<Runnable>());
        return pendingRunnables;
    }

    private void execute(Runnable task, T act) {
        threads.submit(() -> {
            try {
                task.run();
            } finally {
                complete(act);
            }
        });
    }

    private void complete(T act) {
      Queue<Runnable> pending = pendingRunnablesOf(act);
      if (pending.isEmpty())
          playingNow.remove(act);
      else
         //add new task for this actor, in the task queue of the executor
         execute(pending.poll(), act);
    }

}
```

סנכרון

בשימוש שלנו ב ActorThreadPool:

- ת'רדים שונים ניגשים לטבלה acts, בפרט במתודה pendingRunnablesOf.
 נגדיר אותה כ **ConcurrentHashMap**

- התרד של התקשורת מבצע submit, כלומר מוסיף משימת עיבוד של ByteBuffer שנקרא עבור לקוח, ConnectionHandler, מסוים.
ת'רד של עבודה מבצע complete לאחר שהוא מסיים לבצע משימה של לקוח מסוים.
 שתי המתודות מסונכרנות על ה actor

```
public class ActorThreadPool<T> {
```

```
private final Map<T, Queue<Runnable>> acts;
private final Set<T> playingNow;
private final ExecutorService threads;  // the ‘executor’

public ActorThreadPool(int threads) {
    threads = Executors.newFixedThreadPool(threads);
    acts = new ConcurrentHashMap< T, Queue<Runnable>>();
    playingNow = ConcurrentHashMap.newKeySet();
}

public void submit(T act, Runnable r) {
    synchronized (act) {
        if (!playingNow.contains(act)) {
            playingNow.add(act);
            execute(r, act);
        } else {
            pendingRunnablesOf(act).add(r);
        }
    }
}
public void shutdown() {
    threads.shutdownNow();
}

private Queue<Runnable> pendingRunnablesOf(T act) {
    Queue<Runnable> pendingRunnables = acts.get(act);
    if (pendingRunnables == null)
        acts.put(act, pendingRunnables = new LinkedList<>());
    return pendingRunnables;
}

private void execute(Runnable r, T act) {
    threads.submit(() -> {
        try {
            r.run();
        } finally {
            complete(act);
        }
    });
}

private void complete(T act) {
    synchronized (act) {
        Queue<Runnable> pending = pendingRunnablesOf(act);
        if (pending.isEmpty()) {
            playingNow.remove(act);
        } else {
            execute(pending.poll(), act);
        }
    }
```

```
    }

}
```

<u>בעיה טכנית</u>:
הכתובת של כל ConnectionHandler מאוחסנת בשדה ה attachment ב SelectionKey שלו ב Selector (בשדה של 'האובייקט המצורף')
עד כה, כאשר לקוח התנתק, ה Selector, במימוש הפנימי שלו, הוריד את ה SelectionKey שלו אוטומטית מה Selector, כך שלא יהיו יותר רפרנסים ל ConnectionHandler ו garbage collector ינקה את הזיכרון שלו.
אולם כעת, יש לו עדיין רפרנס במבנה acts בטבלת ה acts ב ActorThreadPool
כדי לא לנהל את ההסרה של ConnectionHandler כאלו שאינם רלבנטיים יותר, נשתמש ב **WeakHashMap** של Java המסיר אוטומטית רשומות שלמפתח שלהן יש רק רפרנס אחד.
מאחר ש WeakHashMap אינו Thread-Safe נבצע את הגישות אליו תחת סנכרון, עם ReaderWriterLock

```
public class ActorThreadPool<T> {

    private final Map<T, Queue<Runnable>> acts;
    private final Set<T> playingNow;
    private final ExecutorService threads;  // the 'executor'
    private final ReadWriteLock actsRWLock;

    public ActorThreadPool(int threads) {
        this.threads = Executors.newFixedThreadPool(threads);
        acts = new WeakHashMap< T, Queue<Runnable>>();
        playingNow = ConcurrentHashMap.newKeySet();
        actsRWLock = new ReentrantReadWriteLock();
    }

    public void submit(T act, Runnable r) {
        synchronized (act) {
            if (!playingNow.contains(act)) {
                playingNow.add(act);
                execute(r, act);
            } else {
                pendingRunnablesOf(act).add(r);
            }
        }
    }
    public void shutdown() {
        threads.shutdownNow();
    }

    private Queue<Runnable> pendingRunnablesOf(T act) {
        actsRWLock.readLock().lock();
        Queue<Runnable> pendingRunnables = acts.get(act);
        actsRWLock.readLock().unlock();
```

```
        if (pendingRunnables == null) {
            actsRWLock.writeLock().lock();
            acts.put(act, pendingRunnables = new LinkedList<>());
            actsRWLock.writeLock().unlock();
        }
        return pendingRunnables;
    }

    private void execute(Runnable task, T act) {
        threads.submit(() -> {
            try {
                task.run();
            } finally {
                complete(act);
            }
        });
    }

    private void complete(T act) {
        synchronized (act) {
            Queue<Runnable> pending = pendingRunnablesOf(act);
            if (pending.isEmpty()) {
                playingNow.remove(act);
            } else {
                execute(pending.poll(), act);
            }
        }
    }
}
```

כעת נותר רק לשנות את הטיפוס של pool במחלקה Reactor מ ExecutorService ל
ActorThreadPool, ולהוסיף את ה CH בקריאה ל submit:

```
class Reactor {
     …
     ExecutorService pool;
     ActorThreadPool<NonBlockingConnectionHandler> pool;
    …

    private void handleReadWrite(SelectionKey key) {
        NonBlockingConnectionHandler handler =
                (NonBlockingConnectionHandler) key.attachment();
        if (key.isReadable()) {
            Runnable task = handler.continueRead();
            if (task != null) {
                pool.submit(handler, task);
            }
```

```
        } else {
            handler.continueWrite();
        }
    {
```

<u>הערה</u>: במימוש הנתון של ActorThreadPool יש שימוש ב synchronized לשם סנכרון המתודות submit ו complete. ניתן להימנע משימוש ב synchronized (ראינו כי הוא אינו יעיל בפועל) ולסנכרן עם CAS, אך זה דורש עיצוב מורכב במקצת. **לא נעשה זאת במסגרת הקורס (<u>לא בחומר המבחן</u>), אך כדאי לבחון את המימוש הזה כחומר קריאה**:

```
public class NonBlockingActorThreadPool<T> {

    private ExecutorService pool;
    private final ReadWriteLock actsRWLock = new ReentrantReadWriteLock();
    private Map<T, Actor> acts = new WeakHashMap<>();

    public NonBlockingActorThreadPool(int threads) {
        this.pool = Executors.newFixedThreadPool(threads);
    }

    private Actor getActor(T act) {

        actsRWLock.readLock().lock();
        Actor actor = acts.get(act);
        actsRWLock.readLock().unlock();

        if (actor == null) {
            actsRWLock.writeLock().lock();
            acts.put(act, actor = new Actor());
            actsRWLock.writeLock().unlock();
        }

        return actor;
    }

    public void submit(T act, Runnable r) {
        getActor(act).add(r);
    }

    public void shutdown() {
        pool.shutdownNow();
    }

    private static class ExecutionState {

        //the amount of tasks that should be handled
        public int tasksLeft;
```

```
        //true if there is a thread that handle or designated to handle
these task
        public boolean playingNow;

        public ExecutionState(int tasksLeft, boolean playingNow) {
            this.tasksLeft = tasksLeft;
            this.playingNow = playingNow;
        }

    }

    private class Actor implements Runnable {

        public ConcurrentLinkedQueue<Runnable> tasksQueue = new
ConcurrentLinkedQueue<>();
        public AtomicReference<ExecutionState> state =
                                new AtomicReference<>(new ExecutionState(0,
false));

        public void add(Runnable task) {
            tasksQueue.add(task);
            ExecutionState oldState = state.get();
            //after this operation completed,
            //one additional task will be added to the queue, and
           //a thread for this tasks will be executed if not already is
            ExecutionState newState =
                new ExecutionState(oldState.tasksLeft + 1, true);

            while (!state.compareAndSet(oldState, newState)) {
                oldState = state.get();
                newState.tasksLeft = oldState.tasksLeft + 1;
            }

            if (!oldState.playingNow){
                pool.submit(this);
            }
        }

        @Override
        public void run() {
            //when this function completes,  0 tasks will be in the queue
           // and no one will and the actor will not "play now"
            ExecutionState newState = new ExecutionState(0, false);
            int tasksDone = 0;
            ExecutionState oldState;

            do {
                oldState = state.get();
                for (; tasksDone < oldState.tasksLeft; tasksDone++) {
                    tasksQueue.remove().run();
```

```
                }

            } while (!state.compareAndSet(oldState, newState));
        }
    }
}
```

סוגיית שינוי הרישום המקבילי ב Selector (מקרה מבחן)

נתון כי ניסיון שינוי רישום האירועים עבור ערוץ ב Selector עשוי להעביר את הת'רד המבצע למצב blocked אם ת'רד אחר מבצע כרגע select.
בפרט במערכת שלנו, אם ת'רד עבודה ינסה, לאחר שהוא מוסיף תשובה ללקוח ב writeQueue, לשנות את הרישום של הלקוח ב Selector מ READ ל WRITE | READ, הוא עשוי להיתקע אם ת'רד התקשורת מבצע select.
נגביל את עידכון הרישום ב Selector לת'רד התקשורת בלבד (כמו שהרחקנו את ת'רד העבודה מ i/o)
המנגנון:

- נגדיר תור של משימות 'עדכון רישום' עבור ת'רד התקשורת
- במתודה updateInterestOps, אם הת'רד הנוכחי אינו ת'רד התקשורת (נגדיר משתנה המזהה את ת'רד התקשורת(לא נשנה את הרישום בפועל אלא נוסיף משימת 'עדכון רישום' לרשימה.
- נוסיף ללולאה של ת'רד התקשורת, יתווסף ביצוע משימות של עדכון רישום

```
public class Reactor {
    private final ConcurrentLinkedQueue<Runnable> selectorTasks = new
ConcurrentLinkedQueue<>();
    private Thread selectorThread;
    …
    public void serve() {
             selectorThread = Thread.currentThread();
             …
            // Main loop
            while (!Thread.currentThread().isInterrupted()) {
                selector.select();
                runSelectionThreadTasks();
                for (SelectionKey key : selector.selectedKeys()) {
                          … // handleAccept/ReadWrite
                }
                selector.selectedKeys().clear();
            }
        pool.shutdown();
    }
    …
    void updateInterestedOps(SocketChannel chan, int ops) {
            SelectionKey key = chan.keyFor(selector);
            if (Thread.currentThread() == selectorThread)
               key.interestOps(ops);
```

```
            else {
                selectorTasks.add(() -> {
                    if(key.isValid())
                       key.interestOps(ops);
                 });
                 selector.wakeup();
            }
        }

        private void runSelectionThreadTasks() {
            while (!selectorTasks.isEmpty())
                selectorTasks.remove().run();
        }
    }
```

בחזרה לשאלת סנכרון עדכון הרישום ב continueWrite: לאור מנגנון זה, עדכון הרישום של ת'רד העבודה ל READ | WRITE מוגדר כמשימה לת'רד התקשורת + wakeup. זהו 'notify' נצבר, כך שב select הבא הת'רד התקשורת יתעורר מיד ויבצע את משימות עדכוני הרישום של ת'רדי העבודה, כך שבכל מקרה הם יתבצעו לאחר עדכון הרישום ל READ בערוץ זה של ת'רד התקשורת. בשורה התחתונה מובטח שהרישום יהיה בסופו של דבר READ | WRITE.

שימוש חוזר ב ByteBuffer שעבר זמנם

בקוד הנוכחי אנו מייצרים מחדש ByteBuffer לפני כל פעולת קריאה. ניתן לייעל ולמחזר ByteBuffer קיימים (לא כזה עקרוני):

```
public class NonBlockingConnectionHandler {
    private static final ConcurrentLinkedQueue<ByteBuffer> BUFFER_POOL =
                    new ConcurrentLinkedQueue<ByteBuffer>();
    …
   public Runnable continueRead() {
      ~~ByteBuffer buf = ByteBuffer.allocateDirect(BUFFER_ALLOCASTION_SIZE);~~
      ByteBuffer buf = leaseBuffer();
       …
       releaseBuffer(buf);
    }

    private static ByteBuffer leaseBuffer() {
        ByteBuffer buff = BUFFER_POOL.poll();
        if (buff == null) {
            return ByteBuffer.allocateDirect(BUFFER_ALLOCATION_SIZE);
        }
        buff.clear();
        return buff;
    }

    private static void releaseBuffer(ByteBuffer buff) {
```

```
            BUFFER_POOL.add(buff);
        }
    }
```

ניתן לקרוא את קוד הריאקטור ב-Class Material

4.3.3 שרת מונחה פקודות (Command Invocation Server)

מוטיבציה:

- מימושי הפרוטוקול עד היום התבססו על הגדרה מפורטת וספציפית של בתים, כך שנדרש לממש מקודד (MessageEncoderDecoder) ספציפי ופארסינג ספציפי (במתודת ה process של ה MessagingProtocol) לכל פרוטוקול. מסורבל ומעיק (כפי שנוכחתם בתרגיל)... היינו רוצים לאפשר ללקוח לשלוח לשרת אובייקט ב Java המתאר את הפקודה, והממש אף את ביצועה (בעיצוב הנוכחי נדרוש שהלקוח כתוב אף הוא ב Java). יתרה מכך, גם התשובות של השרת יישלחו באותו אופן.
- [בגישה מרחיקת לכת יותר, ניתן לחשוב על שרת המכיל מבנה נתונים, ומאפשר ללקוחות לבצע כל פעולה שיגדירו על מבנה זה. במודל זה, השרת אינו מגדיר אוסף פקודות מצומצם הניתנות לבקשת הלקוחות, אלא מאפשר ללקוח להגדיר בעצמו כל פקודה שיחפוץ על מבנה הנתונים, השרת יריץ פקודה זו על מבנה הנתונים וישלח ללקוח את התוצאה. במובן זה, בפרוטוקול של השרת יש פעולה אחת בלבד: להריץ את הפקודה שהתקבלה.
לדוגמא: השרת מחזיק מבנה נתונים על הפעולות בשוק ההון. הלקוח יכול לנסח פקודה המריצה אלגוריתם ייחודי שלו על נתונים אלו (לשם קביעה איזו מניה כדאי לקנות), הוא שולח את הפקודה לשרת, השרת מריץ אותה, ושולח ללקוח את הערך שחזר מביצוע הפקודה.
במימוש שלנו, מסיבות טכניות של אבטחה, לא נאפשר ללקוח לשלוח לשרת פקודות (כלומר מחלקות ב Java) שהוא לא מכיר]

- נגדיר תבנית של שרת 'מונחה פקודות', כלומר שרת שהפרוטוקול כולל סוג הודעה אחד בלבד: מקבל 'פקודה' מלקוח, מבצע אותה על הנתונים שלו, ומחזיר ללקוח את הערך המוחזר.
  - הלקוח שולח אובייקט ב Java, ללא כל צורך בקידודו למערך של בתים (יהיה קידוד גנרי שאינו תלוי בסוג הפקודה). השרת לא נדרש לבצע parsing על מבנה ההודעה המתקבלת, אלא סה"כ להפעיל את הפקודה על ידי קריאה למתודה מוסכמת.
  - [הלקוח יכול לשלוח כל פקודה שיחפוץ על הנתונים בשרת]

מימוש:

1. נגדיר 'פקודה' על מבנה נתונים מטיפוס T (טיפוס המידע המאוחסן בשרת)

```
public interface Command<T> extends Serializable {
    Serializable execute(T data);
}
```

הפקודה חייבת להיות Serializable, וכן הערך המוחזר מהמתודה execute, כיוון שזה מפשט את ההכלה של העברת הפקודה או התשובה ברשת.
בפרט, ממשק ה Serializable מגדיר אובייקט שהוא 'בר העברה' ברשת על שתי מתודות:
writeObject – הופכת אובייקט למערך של בתים ('encode')
readObject - בונה מחדש מערך של בתים לאובייקט ('decode')
הבשורה המשמחת: כל מחלקה ב Java שמוצהר עליה כי היא מממשת Serializable, מיוצר עבורה אוטומטית מימוש של הממשק (כלומר, אין צורך לממש את הממשק בעצמנו)

2. הגדרת הפרוטוקול של השרת כביצוע פקודה על מבנה הנתונים שהוא מאחסן

```
public class RemoteCommandInvocationProtocol<T> implements
MessagingProtocol< Serializable > {

    private T data;

    public RemoteCommandInvocationProtocol(T data) {
        this.data = data;
    }

    @Override
    public Serializable process(Serializable msg) {
        return ((Command)msg).execute(data);
    }

    @Override
    public boolean shouldTerminate() {
        return false;
    }
}
```

3. הגדרת ה MessageEncoderDecoder של השרת כמקודד פקודות ותשובות לבתים
כזכור, הפקודות והתשובות הוגדרו כבר כ Serializable, כלומר הן מממשות כבר שיטה להמרת הפקודות/תשובות לבתים (המתודה writeObject), ולבנייתן מחדש מייצוג זה של מערך של בתים (המתודה readObject). כל שנדרש, הוא להתאים את החתימות של readObject ו writeObject בממשק Serializable לאלו של encode ו decodeNextByte בממשק MessageEncoderDecoder של השרת.
טכני משהו:
   - encode
     נגדיר את מבנה ההודעה המקודדת כמערך הכולל:
     - ארבעה בתים לייצוג אורך הפקודה המקודדת בבתים
     - שאר הבתים במערך מתארים את הפקודה, ע"פ המדיניות המומשת ב writeObject
   - decodeNextByte
     - נגדיר שתי רשימות בתים: האחד לארבעת הבתים הראשונים המציינים את גודל האובייקט (lengthBytes), והשני לאחסון הבתים של האובייקט עצמו (objectBytes)

- כאשר מתקבל אחד מארבעת הבתים הראשונים (נזהה זאת ע"פ השדה lengthIndex), הוא יאוחסן במערך הראשון (lengthBytes). כאשר המערך הראשון מתמלא בארבעת הבתים הראשונים שהתקבלו, נמיר אותם למספר שלם כך שנדע לכמה בתים לצפות עבור הפקודה עצמה (השדה objectLength).
- כאשר מגיעים בתים של הפקודה עצמה, נאחסן אותם במערך השני (objectBytes). לאחר שכולם יתקבלו (אנחנו יודעים לכמה לצפות - objectLength) נמיר אותם לאובייקט ע"י קריאה ל readObject של Serializable

```
public class ObjectEncoderDecoder implements
MessageEncoderDecoder<Serializable> {

    private List<Byte> lengthBytes = new LinkedList<Byte>();
    private List<Byte> objectBytes = new LinkedList<Byte>();
    private int objectLength = 0;

    @Override
    public byte[] encode(Serializable message) {

             ByteArrayOutputStream bytes = new ByteArrayOutputStream();

            //placeholder for the object size
            for (int i = 0; i < 4; i++)
                bytes.write(0);

            // encode the message to byte-array according to writeObject
               method of Serializable
            ObjectOutput out = new ObjectOutputStream(bytes);
            out.writeObject(message);
            out.flush();
            byte[] result = bytes.toByteArray();

            //now write the object size
            intToBytes(result.length - 4, result);
            return result;
    }

    @Override
    public Serializable decodeNextByte(byte nextByte) {
             if (lengthBytes .size() < 4) {
                  //indicates that we are still reading the length
                  lengthBytes.add(nextByte);
                  if (lengthBytes .size() == 4)
                       objectLength = bytesToInt(lengthBytes.toArray());
              } else {
                 objectBytes.add(nextByte);
                 if (objectBytes .size() == objectLength) {
                  ObjectInput in = new ObjectInputStream(
```

```
                    new ByteArrayInputStream(objectBytes.toArray()));
                Serializable result = (Serializable) in.readObject();
                objectBytes.clear();
                lengthBytes.clear();
                return result;
            }
            return null;
    }

    private static void intToBytes(int i, byte[] b) {
        b[0] = (byte) (i >> 24);
        b[1] = (byte) (i >> 16);
        b[2] = (byte) (i >> 8);
        b[3] = (byte) i;
    }

    private static int bytesToInt(byte[] b) {
        //this is the reverse of intToBytes,
        //note that for every byte, when casting it to int,
        //it may include some changes to the sign bit so we remove those
          by anding with 0xff

        return ((b[0] & 0xff) << 24)
                | ((b[1] & 0xff) << 16)
                | ((b[2] & 0xff) << 8)
                | (b[3] & 0xff);
    }

}
```

נקודה למחשבה: מה יקרה אם השרת אינו מכיר את מחלקת הפקודה ששלח הלקוח?
בקוד הנוכחי, ייזרק בשרת ClassNotFoundException בפעולת ה readObject מאחר שהמחלקה אינה מוכרת (היא לא ב class-path שלו)
הקוד הנוכחי מניח שהשרת מכיר את כל המחלקות שהלקוח שולח, ובמילים אחרות, הלקוח אינו יכול לשלוח כל פקודה שהוא רוצה (בניגוד למוטיבציה השנייה)
גישה זו נכונה גם בטיחותית: מסוכן לאפשר ללקוח להריץ ללא בקרה כל קוד שיחפוץ על המחשב של השרת.
אם היינו רוצים לתמוך בקונספט:
- נדרש היה להגדיר רכיב 'בטיחות' בשרת, עם הגדרה מה מותר ומה אסור (ניתן לקרוא נתונים אך לא לשנותם, לא ניתן לכתוב לדיסק, וכדומה)
- נדרש היה לכלול בתיאור של הפקודה, לא רק את הסראליזציה של אובייקט הפקודה, אלא גם את קוד המחלקה שלו (ה Class object, שמוגדר כבר כ Serializable). משהו כמו:
encode: ארבעה בתים לגודל הייצוג של המחלקה של הפקודה, ייצוג המחלקה של הפקודה, ארבעה בתים לגודל ייצוג המופע של הפקודה, ייצוג המופע של הפקודה
decodeNextByte: ניתוב הבתים למערך המתאים: הבתים המייצגים את גודל קידוד המחלקה, הבתים המייצגים את קידוד המחלקה, הבתים המייצגים את גודל קידוד הפקודה, הבתים המייצגים את קידוד הפקודה.

בנוסף, לאחר ייצור ה Class Object של המחלקה (לאחר ביצוע readObject על מערך הבתים המתארים את המחלקה של הפקודה), נדרש לטעון אותה לזיכרון של תהליך השרת (ניתן לעשות זאת בעזרת ClassLoader)

אם המחלקה תלויה במחלקות אחרות, צריך יהיה לשלוח גם אותן. מנגנון לא פשוט... קיימות מערכות שתומכות ברמה כזאת של הפשטה, עד כדי תמיכה בלקוחות השולחים פקודות כמחלקות בשפות אחרות.

4. כדי להריץ שרת לדוגמא, נגדיר את מבנה הנתונים שהשרת מאחסן. לדוגמא: ידיעות חדשותיות בנושאים שונים

```
public interface NewsFeed {
   void publish(String category, String news);
   List<String> fetch(String category);
}

public class NewsFeedImpl implements NewsFeed {

    private ConcurrentHashMap<String, ConcurrentLinkedQueue<String>>
       newsPerCategory = new ConcurrentHashMap<>();

    public void publish(String category, String news) {
        ConcurrentLinkedQueue<String> queue =
               newsPerCategory.computeIfAbsent(category, (k) ->
                       new ConcurrentLinkedQueue<>());
        queue.add(news);
    }

    public List<String> fetch(String category) {
         ConcurrentLinkedQueue<String> queue =
                     newsPerCategory.get(category);
            if (queue == null) {
                return new ArrayList<>(0); //empty
            } else
                return new ArrayList<>(queue);
                //copy of the queue, arraylist is serializable
    }
}
```

5. הרצת שרת מונחה פקודות על מבנה נתונים של ידיעות חדשותיות בנושאים שונים

```
public class NewsFeedServer {

    public static void main(String[] args) {

        NewsFeed feed = new NewsFeedImpl(); //one shared object

        new Reactor(
```

```
            7777, //port
            () -> new RemoteCommandInvocationProtocol<Serializable>(feed),
            () -> new ObjectEncoderDecoder< Serializable >(),
            20
        ).serve();
    }
}
```

6. הגדרת פקודות שונות לביצוע על מבנה הנתונים של הידיעות החדשותיות

- קבלת ידיעות בנושא מסוים

```
public class FetchNewsCommand implements Command<NewsFeed> {

    private String category;

    public FetchNewsCommand(String category) {
        this.category = category;
    }

    public Serializable execute(NewsFeed feed) {
        return feed.fetch(category);
    }
}
```

- פרסום ידיעה בנושא מסוים

```
public class PublishNewsCommand implements Command<NewsFeed> {

    private String category;
    private String news;

    public PublishNewsCommand(String category, String news) {
        this.category= category;
        this.news = news;
    }

    public Serializable execute(NewsFeed feed) {
         feed.publish(category, news);
         return "OK";
    }

}
```

7. הגדרת לקוח לדוגמא, המפרסם ומקבל ידיעות בנושאים שונים

מחלקת עזר לשליחה נוחה וקבלה נוחה של בקשות/תשובות ל/מהשרת:

```
public class RCIClient implements Closeable {

    private final ObjectEncoderDecoder encdec;
    private final Socket sock;
    private final BufferedInputStream in;
    private final BufferedOutputStream out;

    public RCIClient(String host, int port) throws IOException {
        sock = new Socket(host, port);
        encdec = new ObjectEncoderDecoder();
        in = new BufferedInputStream(sock.getInputStream());
        out = new BufferedOutputStream(sock.getOutputStream());
    }


    public void send(Command<?> cmd) throws IOException {
        out.write(encdec.encode(cmd));
        out.flush();
    }

    public Serializable receive() throws IOException {
        int read;
        while ((read = in.read()) >= 0) {
            Serializable msg = encdec.decodeNextByte((byte) read);
            if (msg != null) {
                return msg;
            }
        }

        throw new IOException(
              "disconnected before complete reading message");
    }

    @Override
    public void close() throws IOException {
        out.close();
        in.close();
        sock.close();
    }

}
```

תוכנית הלקוח:

```
public class NewsFeedClientMain {

    public static void main(String[] args) throws Exception {

       RCIClient c = new RCIClient(args[0], 7777);
        c.send(new PublishNewsCommand("jobs",
           "System Programmer, knowledge in C++. call 0x134693F"));
        c.receive(); //ok

       c.send(new FetchNewsCommand("jobs"));
       List<String> newss = (List<String>)c.receive();
       for (String news : newss)
              System.out.println(news);

    }
}
```

הקוד המלא נמצא ב-Class Material

5. בסיסי נתונים

5.1 מבוא, מבנה נתונים ביחס למערכת קבצים

<u>מוטיבציה</u>: שמירת נתוני סטודנטים וקורסים באוניברסיטה
- נתוני סטודנטים
- נתוני קורסים
- נתוני מחלקות
- אילוצים: קורסי קדם, שנת לימודים,...
- ...

התוכנית ממומשת כרגיל בזיכרון. כלומר, יש מחלקות כמו Student, Course, Department הכוללות שדות ומתודות המכילים את המידע וממשים בין היתר את האילוצים השונים:

```
Student: getYear(), getCourses(),…
Course: getPreRequirements(), get Students()
```

חסרונות:
- כאשר התהליך מסתיים (לוגית, או בשל תקלה כמו הפסקת חשמל) כל המידע נמחק, שהרי הוא מאוחסן בזיכרון בטכנולוגיה חשמלית.
- לא תמיד ניתן לאחסן את כל המידע בזיכרון. באפליקציות רבות מדובר במידע גדול החורג מגודלו של ה RAM.

 נשמור את המידע בדיסק:
- הוא יישמר שם לאורך זמן גם לאחר שהתהליך יסתיים.
- נפח האחסון של הדיסק גדול לאין שיעור מזה של ה RAM

מימוש:
- נגדיר את המחלקות הרלבנטיות כממשות ממשק של סראליזיה ודה-סראליזציה לדיסק. משהו כמו:

```
toFile(OutputStream), fromFile(InputStream)
```

[ניתן אגב להשתמש לשם כך במחלקה ObjectEncoderDecoder שכתבנו, או לממש זאת בכל פורמט שנרצה (כמו לדוגמא הפורמט השכיח json)]

חסרונות:
- שליפת נתונים: שליפת קבוצת סטודנטים בעלי תכונה מסוימת (מדעי המחשב, שנה ב) אינה טריוויאלית כאשר הם מאוחסנים בכל מיני מקומות בדיסק, כמערכי בתים.
- עדכון נתונים: הוספת ציון לקבוצת סטודנטים בעלי תכונה מסוימת אינה פשוטה כלל ועיקר (תלמידי מערכות, מועד א)
- תמיכה בפעולות מקבילות מכמה תהליכים על הנתונים בדיסק המשותף: מנגנוני נעילה, סנכרון, הרצה מקבילית...
- תמיכה בפעולות מורכבות: לדוגמא טרנזקציה, ביצוע סדרה של פעולות שונות כאשר כישלון של אחת מהן מחזיר את המצב לקדמותו (ביטול רישום נוכחי לקורס בחירה, רישום לקורס מבוקש;
  פדיון קופת גמל, רכישת מניות בשער מסוים)
- אבסטרקציה: חוסר תלות בשפת התכנות, במערכת הקבצים, במערכות ההפעלה, בטכנולוגיית האחסון

- ממשקים נוחים
- אופטימיזציות
- ...

 נדרשת סביבת ריצה ייעודית התומכת במידול, שמירה וביצוע פעולות מגוונות על נתונים, תוך תמיכה במקביליות, מודלריות, אופטימזציות וכדומה

סביבה שכזו מכונה: DBMS (DataBase Management System), או 'מערכת לניהול בסיסי נתונים'

דוגמאות למערכות שכאלה: MySQL, Microsoft SQL Server, Oracle, Sybase,…

כמו בסביבות ריצה אחרות שפגשנו בקורס (מערכת ההפעלה, JVM) לא נעסוק באופן מימוש הסביבה (סוגיות בעיצוב מערכת שכזו נלמד בקורס 'ניהול בסיסי נתונים') אלא בשימוש בו: כיצד מגדירים את מודל הנתונים, וכיצד מבצעים עליו פעולות תחת סביבה זו, מהי שפת הממשק של סביבה זו. בשלב זה, נציין רק את הארכיטקטורה השכיחה של מערכות שכאלו, באופן כללי ביותר:

- ניהול בטיחות
- ניהול טרנזקציות
- שליפת נתונים: ביצוע *שאילתא*, אופטמיזציות
- פעולות על נתונים: הוספה, עדכון, מחיקה
- גישה לקבצים
- גישה לדיסק

5.2 מודל נתונים (Data Model)

מודל נתונים הינו תיאור מופשט של מידע, המבוסס על הגדרת הנתונים ועל הפעולות עליהן, ללא תלות באופן אחסונן ובמימוש פעולות אלו.

קיימים סוגים שונים של מודל נתונים, כלומר שיטות ארגון שונות של מידע.
לדוגמא:

- מודל נתונים רלציוני (יחסי)
  - המידע מאורגן, כפי שנראה, סביב טבלאות עם שדות, קשרים ביניהם, ואפשרויות מפתוח.
- מודל נתונים מבוסס key-value
  - המידע מאורגן כטבלה אחת גדולה, הממפה מפתח לאובייקט (מעין HashMap על הדיסק). קיימות מערכות שונות שכאלה: Berkley DB, Redis, שירות S3 של AWS
- מודל נתונים מונחה אובייקטים
  - בסיס הנתונים כולל אוסף של אובייקטים הניתנים לשליפה על פי תכונותיהם (כל הסטודנטים שהשדה year שלהם הוא 2): object4j, שירות DynamoDB של AWS.

אנו נתמקד במודל רלציוני (המגה-קלאסי)

מודל נתונים רלציוני (Relational Model)

במודל נתונים רלציוני המידע מאורגן בטבלאות (relations, 'רלציות'), כאשר כל טבלה מוגדרת על ידי אוסף של שדות (attributes), עם אילוצי קשרים לטבלאות אחרות (integrity constraints), ואינדקסים.

שדה מוגדר על ידי: שם, טיפוס (integer), תחום הגדרה (מספר 14 ל 60), ותכונות/דרישות נוספות. לדוגמא, ניתן לדרוש שערכו יהיה ייחודי, UNIQUE, בטבלה. לדוגמא: השדה 'מספר זהות' בטבלת 'סטודנטים' צריך להיות ייחודי, כלומר אין שני סטודנטים בעלי אותו מספר זהות. בנוסף, ניתן לדרוש שהדה חייב לקבל ערך NOT NULL (כמו נניח שם הסטודנט).
אוסף של שדות מגדיר טבלה, לטבלה יש גם שם (כמו טבלת 'סטודנטים', הכוללת שדות כמו מספר זהות ושם, ועוד).
אילוצי הקשרים בין הטבלאות מוגדרים על ידי מפתח זר (foreign key) היוצר תלות בין ערך של קבוצת שדות בטבלה אחת לבין מציאות קבוצת שדות בעלי ערך זהה בטבלה אחרת. לדוגמא: השדה 'מחלקה' בטבלת 'סטודנטים' צריך להכיל שם של מחלקה המוגדרת בטבלת 'מחלקות'.
נשים לב, כי ניתן להגדיר שני סוגי אילוצים על התוכן: אילוץ מקומי (טיפוס השדה, תחום ההגדרה שלו, היותו ייחודי בטבלה) ואילוצים חוצי טבלאות (מפתחות זרים)
הגדרת שדה כאינדקס (index) תהפוך את הגישה אליו ליעילה יותר. הגדרת שדה כמפתח תדרוש מסביבה התארגנות בהתאם, בפרט הגדרת מבנה נתונים שתומך בגישה מהירה לשדות אלו (בדומה ל'אינדקס' המצוי לעיתים בדפים האחרונים של ספרים שונים). בדרך כלל, לא נרצה להגדיר את כל השדות כאינדקס, אלא רק שדות שלדעתנו הגישה לנתונים דרכם שכיחה (כמו לדוגמא 'שם הסטודנט').

במודל רלציוני קיימים סוגים שונים של פעולות:

- הוספת רשומה (record) לטבלה
  הוספת 'שורה' חדשה לטבלה נתונה, עם הצבת ערכים לכל אחד מהשדות (הוספת רשומה לטבלת 'סטודנטים' עם פרטי הסטודנט כפי שהוגדרו בשדות השונים, מספר זהות, שם, מחלקה, וכו')
- עדכון רשומה בטבלה
  השמת ערך חדש לשדה אחד או יותר בשורה בטבלה
- מחיקת רשומה מטבלה
- שליפת נתונים מטבלאות:
  - Selection: בחירה של מספר רשומות מטבלה על פי קריטריון מסוים (סטודנטים משנה א במדעי המחשב). חלק מהשורות.
  - Projection: בחירה של מספר שדות מרשומות של טבלה (רק את שמות הסטודנטים ואת שם המחלקה שלהם). חלק מהעמודות.
  - פעולות על קבוצות: איחוד של קבוצות רשומות, חיתוך, משלים, מכפלה קרטזית.
  - פעולת ה Join היא הפעולה המרכזית של שליפת הנתונים. בפעולה זו אנו משדכים, תחת אילוץ תאימות מוגדר, רשומות ושדות מסוימים מטבלאות שונות. כלומר:
    Selection + Projection + Cross Product

הגדרת מודל נתונים רלציוני עבור מידע הדורש מידול

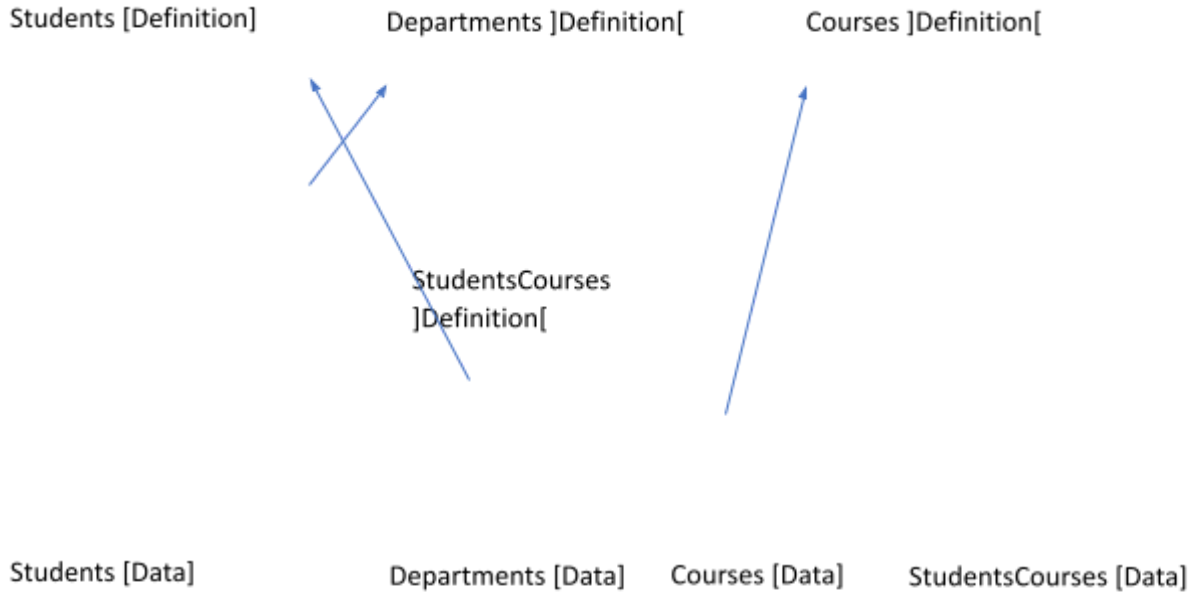
`נתון מידע הנדרש לשומרו לאורך זמן בבסיס נתונים רלציוני. יש להגדיר עבורו מודל נתונים.

לדוגמא: יש להגדיר מודל נתונים רלציוני עבור המידע של הסטודנטים, הקורסים והמחלקות באוניברסיטה. ובפרט, יש לתאר את המידע כאוסף של טבלאות עם שדות, ואת הקשרים ביניהן על ידי מפתחות.

לשם פשטות, ננקוט בגישה המותאמת כבר לדרכי עיצוב המוכרות לנו לעולם התכנות: Object Oriented Design

1. מחלקה ב Java תוגדר כטבלה במודל הנתונים
השדות של המחלקה יהיו השדות של הטבלה, עם טיפוסים תואמים. במידה ושדה מסוים דורש יחודיות בטבלה ו/או השמת ערך, נגדירו כ UNIQUE ו/או NOT NULL
2. אם יש שדה במחלקה מסוג 'אובייקט', נגדיר אותו בטבלה נפרדת, ונכלול בטבלה הנוכחית 'מצביע' אליו, כלומר מפתח זר (אם לא נעשה זאת, כלומר נכלול את הגדרת האובייקט על שדותיו השונים בטבלה המקורית, יהיה אחסון מיותר של נתונים אלו שוב ושוב בכל רשומה)
3. אם השדה הוא מייצג 'מערך' של נתונים, והיחס הוא many-to-many, נגדיר טבלת ביניים הממפה שדה (או קבוצת שדות) ייחודי(ת) בטבלה אחת לשדה או קבוצת שדות בטבלה שניה
4. נוסיף את תכונת האינדקס לשדות מרכזיים בשליפת הנתונים, כלומר שדות שהערך שלהם מהווה קריטריון לשליפת הרשומה שלו.
5. בכל טבלה חייבת להיות שדה (או קבוצת שדות) שהוא גם ייחודי וגם אינדקס. כלומר, הדרישה היא שיש לפחות דרך אחת לשלוף מידע באופן חד משמעי ויעיל. שדה (או קבוצת שדות) זה מכונה 'מפתח ראשי', או primary key (יכולים כמובן להיות שדות נוספים שהם גם אינדקס וגם ייחודיים, אך רק אחד מהם הוא 'מפתח ראשי' (יש לכך השלכות, לא חשוב כרגע))

[המפתח הזר מוגדר גם כן, בילט-אין, כאינדקס, שהרי שאילתות רבות יתבססו עליו, כפי



שנראה]

5.3 SQL (Structured Query Language)

SQL היא שפת הממשק בין אפליקציות לבין מערכת לניהול בסיסי נתונים. זוהי שפה גנרית, כלומר, כל מערכת לניהול בסיסי נתונים אמורה לתמוך בשפת ממשק זה (באופן זה האפליקציה אינה תלויה בסוג המערכת שהיא עובדת מולה בזמן ריצה (התפתחו אגב במשך השנים וואריציות שונות לשפה זו בכל מיני מערכות, אנו נעסוק ב SQL הקלאסי המותאם לכולן)).

שפת SQL תומכת בשתי קטגוריות מרכזיות של פעולות:

- הגדרת מודל הנתונים (Data Definition Language)
  הגדרת טבלאות, שדות, מפתחות, אינדקסים, וכדומה.

נראה כיצד ניתן לתאר/ליצור את מודל הנתונים שהגדרנו עבור הסטודנטים והקורסים בעזרת פעולות SQL (קיימות מערכות המתרגמות אוטמטית תרשימים מעין זה שהגדרנו לקוד SQL)

```
CREATE TABLE Departments (Id INTEGER PRIMARY KEY,
                          Name VARCHAR(20) UNIQUE NOT NULL,
                          Building INTEGER);
CREATE INDEX idx_departments_name ON Departments(name);
```

```
CREATE TABLE Students (Id INTEGER PRIMARY KEY,
                       Name VARCHAR(20) NOT NULL,
                       Login VARCHAR(10) UNIQUE,
                       DeptId INTEGER,
                       FOREIGN KEY (DeptId) REFERENCES Departments(Id));
CREATE INDEX idx_users_name ON Students(name);

CREATE TABLE Courses (Id INTEGER PRIMARY KEY,
                      Name VARCHAR(20) UNIQUE NOT NULL,
                      NAKAZ INTEGER);
CREATE INDEX idx_courses_name ON Courses(name);

CREATE TABLE StudentsCourses (StudId INTEGER,
                              CourseId INTEGER,
                              Year INTEGER,
                              Grade INTEGER,
                              PRIMARY KEY (StudId, CourseId, Year),
                              FOREIGN KEY (StudId) REFERENCES
Students(Id),
                              FOREIGN KEY (CourseId) REFERENCES
Courses(Id));
CREATE INDEX idx_studentscourses_year ON StudentsCourses(year);
```

- פעולות על נתונים (Data Manipulation Language)
  - o הוספת רשומות לטבלה

```
INSERT INTO Students(Id,Name,Login,DeptId)  VALUES(17, 'Dina','din',201)
```

פעולה זו תיכשל, שכן לא קיימת מחלקה שהשדה Id שלה הוא 201

 יש להגדיר קודם את המחלקות השונות:

```
INSERT INTO Departments(Id,Name,Building) VALUES(201, 'math', 58)
INSERT INTO Departments(Id,Name,Building) VALUES(202, 'cs', 37)
INSERT INTO Students(Id,Name,Login,DeptId) VALUES(17, 'Dina','din',201)
INSERT INTO Students(Id,Name,Login,DeptId) VALUES(89, 'Danny','dan',202)
INSERT INTO Students(Id, DeptId) VALUES(5,202)
```

פעולה זו תיכשל, שכן הגדרנו את השדה Name במחלקה Students להיות NOT NULL, כלומר דרשנו שלא יהיה סטודנט ללא שם

```
INSERT INTO Students(Id, Name, DeptId) VALUES(5, 'Ringo', 202)
```

נשים לב כי השדה Login לא קיבל ערך (ערכו נכון לעכשיו הוא NULL), זה בסדר כי לא דרשנו עבורו 'NOT NULL'

```
INSERT INTO Courses(Id,Name,Level) VALUES(7, 'infi', 4)
INSERT INTO Courses(Id,Name, Level) VALUES(8, 'spl',4)
INSERT INTO Courses(Id,Name, Level) VALUES(9, 'ppl',4)
INSERT INTO StudentsCourses(StudId,CourseId,Year,Grade) VALUES(17,7,18,70)
INSERT INTO StudentsCourses(StudId,CourseId,Year,Grade) VALUES(17,8,19,90)
INSERT INTO StudentsCourses(StudId,CourseId,Year,Grade) VALUES(89,7,18,80)
INSERT INTO StudentsCourses(StudId,CourseId,Year,Grade) VALUES(89,8,19,85)
```

- עדכון רשומות קיימות

```
UPDATE Students SET Login = 'ring'
```

פעולה זו משנה את השדה Login לכל הסטודנטים. מלבד זאת שהיא לא הגיונית לוגית, היא גם תיכשל כי הגדרנו את השדה Login כ UNIQUE.
נוסיף קריטריון מדויק יותר בעזרת WHERE

```
UPDATE Students SET Login = 'ring' WHERE Name = 'Ringo'
```

- מחיקת רשומות

```
DELETE FROM StudentsCourses WHERE year < 1900
```

- שליפת נתונים על ידי שאילתא [הנושא החשוב]

כפי שראינו, המידע מאורגן על ידי רשומות עם שדות שונים בטבלאות שונות.
נשים לב לכך שהמידע במודל הרלציוני מאוחסן באופן קומפקטי – אין חזרה על נתון אלא אם כן הוא 'מצביע', כלומר שדות שהם מפתחות זרים. בשיטת ארגון שכזו, שליפת נתונים כרוכה לרוב בשידוך נתונים מטבלאות שונות. בניגוד למודלים אחרים, בחוזרים על נתונים שוב ושוב, כדי לייעל את פעולת השליפה (lookup פשוט), תוך בזבוז משווע של מקום אחסון.
שליפת הנתונים מבוססת בעיקרה על פעולת ה JOIN, המבצעת כפי שאמרנו מכפלה קרטזית בין טבלאות (cross product), על פי קריטריוני סינון שונים (selection - חלק מהרשומות/השורות, projection – חלק מהשדות/העמודות)

ננסח כללים פשוטים ביותר להגדרת שאילתא – 'המדריך השלם'

דוגמא: נדרש לשלוף מבסיס הנתונים שמות של סטודנטים, שמות של קורסים, והציון עבורם

1. נכתוב SELECT
2. נציין את השדות (כולל טבלאותיהם) שאותן אנו רוצים לקבל בשליפה
3. נכתוב FROM
4. נציין את שמות הטבלאות המופיעות בשורת ה SELECT

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  Students, Courses, StudentsCourses
```

מה קורה כאשר מורצת שאילתא שכזאת?
מסתבר, כי מנוע הרצת השאילתות בונה עבור כל שדה בשורת ה SELECT ווקטור עם כל הערכים המצויים בטבלה עבור שדה זה, ומבצע עליהם מכפלה קרטזית.
במקרה שלנו:

Dinna		infi		70
Danny	X	spl	X	80
Ringo		ppl		85
				90

Dinna infi 70
Dinna infi 80
Dinna infi 85
Dinna infi 90
Dinna spl 70
…
Dinna ppl 90
Danny infi 70
…
Danny ppl 90
Ringo infi 70
…
Ringo ppl 90

התוצאות כמובן שגויות – מתקבלים שידוכים רבים של נתונים לא תואמים משלוש הטבלאות.
נדרש לכלול בשאילתא <u>קריטריון תאימות בין הטבלאות</u>, כלומר אילוץ על פעולת השידוך.
נשים לב כי קריטריון השידוך הנכון מוגדר למעשה ע"י המפתחות הזרים.

1. נכתוב SELECT
2. נציין את השדות (כולל טבלאותיהם) שאותן אנו רוצים לקבל בשליפה
3. נכתוב FROM
4. נציין את שמות הטבלאות המופיעות בשורת ה SELECT, **תוך תיאור הקשר ביניהן ע"פ המפתחות הזרים עם המילה השמורה JOIN** (אם קיימת טבלה נוספת בחלק ה JOIN, שאינה מופיעה בחלק ה FROM, נוסיף אותה לשם)

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
       INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
```

Dinna		infi		70
Danny	X_{join}	spl	X_{join}	80

Ringo ppl 85
90

Dinna infi 70
Dinna spl 90
Danny infi 80
Danny spl 85

5. במידה ורוצים לקבל רק חלק מהרשומות ('selection'), ניתן לעשות זאת על ידי הוספת רכיב ה WHERE לשאילתא.
לדוגמא: קבלת ציונים של סטודנטים במחלקה למתמטיקה בלבד.

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
WHERE Students.DeptId = 201
```

Dinna infi 70
Dinna spl 90

דוגמא נוספת: הכללת שם המחלקה בתוצאות

```
SELECT  Students.Name, Departments.name, Courses.Name,
StudentsCourses.Grade
FROM  ((Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
       INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id)
       INNER JOIN Departments ON Students.DeptId = Departments.Id
```

6. אם רוצים שהתוצאות יהיו ממוינות על פי קריטריונים שונים, ניתן לעשות זאת על ידי הוספת חלק ה ORDER BY לשאילתא (ברירת המחדל היא מיון ע"פ המפתח הראשי)
לדוגמא: מיון התוצאות בסדר עולה ע"פ שם הסטודנטים, ובסדר יורד על פי ציונם

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
WHERE Courses.NAKAZ = 4
ORDER BY Students.Name ASC, StudentsCourses.Grade DESC
```

Danny spl 85
Danny infi 80
Dinna spl 90
Dinna infi 70

OUTER JOIN

נניח כי אנו מעוניינים לקבל את כל שמות הקורסים, ואם סטודנטים לומדים בהם את שמות הסטודנטים וציוניהם.
בשאילתא הנוכחית, מתקבלים רק קורסים שקיים סטודנט הלומד אותם (הקורס **ppl** שלא נלמד על ידי אף סטודנט אינו מופיע), שהרי דרשנו ב JOIN תאימות בין טבלת Courses לטבלת StudentsCourses.

נניח כי אנו מעוניינים לקבל את כל שמות הסטודנטים, ואם הסטודנטים לוקחים קורסים את שמות הקורסים ואת ציוניהם.
בשאילתא הנוכחית, מתקבלים רק סטודנטים הלומדים קורס כלשהו (**Ringo**, שבשל עיסוקו כמתופף עמוס לא לוקח אף קורס כרגע, אינו מופיע), שהרי דרשנו ב JOIN תאימות בין טבלת Students לטבלת StudentsCourses.

נגדיר סוג נוסף של JOIN, שאינו דורש תאימות דו צדדית כמו ה INNER JOIN, אלא תאימות חד צדדית בין שתי הטבלאות. כלומר, יופיעו כל הערכים מטבלה אחת, ואם קיימת תאימות גם הערכים מהטבלה השניה בהתאם (אחרת, טכנית, יופיע NULL עבור הערך שאינו קיים בטבלה השניה)

כל הקורסים + סטודנטים שלוקחים קורסים:

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      RIGHT OUTER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
```

Dinna infi 70
Dinna spl 90
Danny infi 80
Danny spl 85
NULL ppl NULL

כל הסטודנטים + קורסים שנלמדים ע"י סטודנטים:

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students LEFT OUTER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
```

Dinna infi 70
Dinna spl 90
Danny infi 80

Danny spl 85
Ringo NULL NULL

כל הסטודנטים + כל הקורסים:

נבצע UNION של שתי השאילתות

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
       RIGHT OUTER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
UNION
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  Students LEFT OUTER JOIN
      (StudentsCourses INNER JOIN Courses ON StudentsCourses.CourseId =
Courses.Id)
      ON Students.Id = StudentsCourses.StudId
```

Dinna infi 70
Dinna spl 90
Danny infi 80
Danny spl 85
NULL ppl NULL
Dinna infi 70
Dinna spl 90
Danny infi 80
Danny spl 85
Ringo NULL NULL

כדי למנוע חזרות, מצמצם את השאילתא השניה כך שתחזיר את התוספות בלבד, בעזרת WHERE

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      RIGHT OUTER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
UNION
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  Students LEFT OUTER JOIN
      (StudentsCourses INNER JOIN Courses ON StudentsCourses.CourseId =
Courses.Id)
      ON Students.Id = StudentsCourses.StudId
WHERE Courses.Name is NULL
```

Dinna infi 70
Dinna spl 90
Danny infi 80
Danny spl 85

NULL ppl NULL
Ringo NULL NULL

GROUP BY

ניתן לאחד מספר רשומות על פי ערך שדה משותף כדי לבצע על ערך שדה אחר פעולות שונות כמו: ספירה, סכימה, ממוצע, וכדומה.

לדוגמא, עבור השאילתא המקורית:

```
SELECT  Students.Name, Courses.Name, StudentsCourses.Grade
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
```

Danny spl 85
Danny infi 80
Dinna spl 90
Dinna infi 70

חישוב ממוצע הציונים של כל סטודנט:

```
SELECT  Students.Name, AVG)StudentsCourses.Grade(
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
GROUP BY Students.Id
```

Danny 82.5
Dinna 80

הציון הנמוך ביותר של כל סטודנט:

```
SELECT  Students.Name, MIN)StudentsCourses.Grade(
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
GROUP BY Students.Id
```

Danny 80
Dinna 70

הציון הגבוה ביותר של כל סטודנט:

```
SELECT  Students.Name, MAX)StudentsCourses.Grade(
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
```

```
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
GROUP BY Students.Id
```

Danny 85
Dinna 90

מספר הקורסים שכל סטודנט לקח (מעל 0 במקרה זה):

```
SELECT  Students.Name, COUNT)StudentsCourses.Grade(
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
GROUP BY Students.Id
```

Danny 2
Dinna 2

סך נקודות הזכות שכל סטודנט צבר בקורסים:

```
SELECT  Students.Name, SUM)Courses.NAKAZ(
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
      INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
GROUP BY Students.Id
WHERE StudentsCourses.grade > 55
```

Danny 8
Dinna 8

הערה: DISTINCT מאחד מופעים זהים של רשומות בתוצאת השאילתא, לא רק עבור שדות ספציפיים כמו ב-GROUP

```
SELECT  Students.Id, Students.Name
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
       INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
```

17 Dinna
17 Dinna
89 Danny
89 Danny

```
SELECT DISTICT Students.Id, Students.Name
FROM  (Students INNER JOIN StudentsCourses ON Students.Id =
StudentsCourses.StudId)
```

```
INNER JOIN Courses ON StudentsCourses.CourseId = Courses.Id
```

17 Dinna
89 Danny

5.4 עיצוב אפליקציה לניהול מידע בבסיס נתונים

נעצב תבנית כללית לניהול מידע בבסיס נתונים, וכן אפליקציה העושה בו שימוש למטרה מוגדרת.

את האפליקציה נכתוב ב Python, ואת הפעולות על בסיס הנתונים ב SQL (נשתמש בספריה של Python המספקת ממשק שכזה מול בסיס הנתונים)

5.4.1 מבוא קצר לPython

- תכונות
  - שפת סקריפט
    - מורצת שורה-שורה ע"י האינטרפרטר (לא הרצה של קובץ מקומפל)
    - פחות יעיל – פארסינג בזמן ריצה, שגיאות זמן ריצה שניתן היה למנוע מראש.
  - לא חובה להגדיר טיפוסים
    - יש טיפוס למשתנה בכל רגע נתון לאור הערך המושם הנוכחי, אך טיפוס עשוי להשתנות בהתאם להשמה הנוכחית

      x = 3
      x = 'abc'
    - שגיאות טיפוסים מתגלות רק בזמן ריצה
  - ניתן לממש OOP, מחלקות וכדומה
    - האובייקטים ב Heap
    - self, בנאי, ניקוי אוטומטי
    - שדות, מתודות
    - ירושה, פולימורפיזם
  - קיימות ספריות רבות התומכות בדברים שונים ומשונים (בפרט חבילה נוחה לעבוד מול בסיסי נתונים – sqlite)
  - נוחה לכתיבה/לפיתוח, פחות לקריאה (וכפי שאמרנו, פחות יעיל בהרצה (ניתן אגב לקמפל מראש לשם יעילות))

- קוד לדוגמא
  - דוגמת קוד באתר
  - כאשר יש הגדרה של פרוצדורה האינטרפרטר שומר אותה בזיכרון
  - אי הרצה של פונקציית ה main כאשר מבוצע import, בעזרת משתנה המערכת __main__
  -]פרטים נוספים בתרגול[

5.4.2 עיצוב האפליקציה

נממש אפליקציה לבדיקה אוטומטית של תרגילי תכנות, כאשר פרטי הסטודנטים, העבודות והציונים מנוהלים בבסיס נתונים.

נתון: בסיס נתונים עם פרטי סטודנטים ותרגילים, תיקיה במערכת הקבצים עם קוד התרגיל (ב Python) שכל סטודנט הגיש.

האפליקציה קוראת כל תרגיל שהוגש, מריצה אותו (את הפונקציה המוגשת run_assignment) ומשווה את התוצאה לתוצאה הנדרש (הניתנת כחלק מהגדרת התרגיל בבסיס הנתונים), מחשבת את הציון בהתאם ושומרת אותו בבסיס הנתונים.

מימוש

ניסיון ראשון

```
import os
import sqlite3
import imp
import atexit

#connect to the database
_conn = sqlite3.connect('grades.db')

#register a function to be called immediately when the interpreter
terminates
def _close_db():
    _conn.commit()
    _conn.close()

atexit.register(_close_db)

#our application API:

def create_tables():
    _conn.executescript("""
        CREATE TABLE students (
            id      INT         PRIMARY KEY,
            name    TEXT        NOT NULL
        );

        CREATE TABLE assignments (
            num                 INT     PRIMARY KEY,
            expected_output     TEXT    NOT NULL
        );

        CREATE TABLE grades (
            student_id      INT     NOT NULL,
```

```
            assignment_num  INT     NOT NULL,
            grade           INT     NOT NULL,

            FOREIGN KEY(student_id)     REFERENCES students(id),
            FOREIGN KEY(assignment_num) REFERENCES assignments(num),

            PRIMARY KEY (student_id, assignment_num)
        );
    """)


def insert_student(id, name):
    _conn.execute("INSERT INTO students (id, name) VALUES (?, ?)", [id,
name])


def insert_assignment(num, expected_output):
    _conn.execute("INSERT INTO assignments (num, expected_output) VALUES
(?, ?)",
          [num, expected_output])


def insert_grade(student_id, assignment_num, grade):
    _conn.execute("INSERT INTO grades (student_id, assignment_num, grade)
VALUES (?, ?, ?)",

                                [student_id, assignment_num, grade])

def grade(assignments_dir, assignment_num):
    query_data = _conn.cursor()
    query_data.execute("SELECT expected_output FROM assignments WHERE num
= ?",[assignment_num])

    # the expected value is the first element in the fetched row
    expected_output = query_data.fetchone()[0]

    #loop over the files in the assignment_dir
    for assignment in os.listdir(assignments_dir):

        #the os.path.splitext split a file name that looks like:
'file.ext' into ['file', 'ext']
        (student_id, ext) = os.path.splitext(assignment)

        #imp.load_source loads python source code (module) into a variable
        #the first argument will be the __name__ that the module will see
when it loads
        #and the second is the path to the module
        code = imp.load_source('test', assignments_dir + '/' + assignment)

        #calculating the grade
```

```
        grade = 100 if code.run_assignment() == expected_output else 0

        insert_grade(student_id, assignment_num, grade)


def print_grades():
    cur = _conn.cursor()

    cur.execute("SELECT name as student_name, assignment_num, grade
        FROM students INNER JOIN grades ON students.id =
grades.student_id")

    print 'grades:'
    for row in cur.fetchall():
        print 'grade of student {} on assignment {} is {}'.format(*row)
```

<u>הרצה לדוגמא</u>:

נניח כי הוגשו הקבצים הבאים עבור התרגיל האחרון

```
#1111.py
def run_assignment():
  return 'spl'

#2222.py
def run_assignment():
  return 'spl!'

#3333.py
def run_assignment():
  return 'spl'
```

`הרצת הקוד

```
> python

>>> from vicious import *
>>> create_tables()
>>> insert_student(1111, 'Alice')
>>> insert_student(2222, 'Bob')
>>> insert_student(3333, 'Christ')
>>> insert_assignment(1, 'spl')

>>> grade('assignments', 1)
>>> print_grades()
grades:
grade of student Alice on assignment 1 is 100
grade of student Bob on assignment 1 is 0
```

grade of student Christ on assignment 1 is 100

חיסרון מרכזי: הקוד של הגישה לבסיס הנתונים מעורבב עם הקוד של ה'לוגיקה' – אופן חישוב הציון, הכנת הדו"ח.

- שינוי בלוגיקה (שיטת חישוב הציון) ישנה את תוכנית הגישה לבסיס הנתונים
- שינוי קוד הגישה לבסיס הנתונים (נניח הוספת שכבת caching, או שימוש בספריה טובה יותר מ sqlite3) ישנה את החלק הלוגי
- שימוש בקוד הגישה לבסיס הנתונים לאפליקציה אחרת יגרור טעינה של החלק הלוגי
- שימוש בקוד הלוגי (חישוב אוטומטי של ציון) לאפליקציה אחרת יגרור טעינה של קוד הגישה לבסיס הנתונים.

נפריד את שני הרכיבים:

- שכבה עצמאית של גישה לבסיס נתונים
- לוגיקת מתן הציון והכנת הדוחות

ניסיון שני:

```
#file persistence.py

import sqlite3
import atexit

#we want to make the persistence layer singleton so we are marking the
following class as private
# by prepending an underscore to its name.
class _PersistenceLayer(object):
    def __init__(self):
        self._conn = sqlite3.connect('grades.db')

    def _close(self):
        self._conn.commit()
      self._conn.close()

    def create_tables(self):
        #see code in previous version...

    def insert_student(self, id, name):
        #see code in previous version...

    def insert_assignment(self, num, expected_output):
        #see code in previous version...

    def insert_grade(self, student_id, assignment_num, grade):
        #see code in previous version...

    def get_assignment_expected_output(self, assignment_num):
        c = self._conn.cursor()
        c.execute("SELECT expected_output FROM assignments WHERE num = ?",
[assignment_num])
        return str(c.fetchone()[0])
```

```
    def get_all_grades(self):
        c = self._conn.cursor()
        return c.execute("SELECT name as student_name, assignment_num,
grade
                          FROM students INNER JOIN grades ON students.id =
                               grades.student_id”).fetchall()

#persistence layer is a singleton
psl = _PersistenceLayer()

atexit.register(psl._close)


#file: grading.py

from persistence import psl
import os
import imp


def grade(assignments_dir, assignment_num):
    expected_output = psl.get_assignment_expected_output(assignment_num)

    for assignment in os.listdir(assignments_dir):
        (student_id, ext) = os.path.splitext(assignment)

        code = imp.load_source('test', assignments_dir + '/' + assignment)

        grade = 100 if code.run_assignment() == expected_output else 0

        psl.insert_grade(student_id, assignment_num, result)


def print_grades():
     print 'grades:'
    for grade in psl.get_all_grades():
        print 'grade of student {} on assignment {} is {}'.format(*grade)
```

חיסרון: המידע שמספקת שכבת האחסון הינו ספציפי – הוא כולל רק את המידע הנדרש כעת לחלק הלוגי, ואינו תומך במידע נרחב יותר שהחלק הלוגי עשוי להיזדקק לו בהמשך.

- expected_output

אם נרצה בעתיד לשפר את שיטת הערכת הציון על בסיס שדות אחרים, נצטרך לשנות את השכבה הניגשת לבסיס הנתונים

- get_all_grades

אם נרצה לייצר פלט עם תוכן עשיר יותר מלבד הציון לסטודנט, נצטרך לשנות את השכבה הניגשת לבסיס הנתונים

נגדיר ממשק כללי לאחסון ושליפת נתונים מטבלה בבסיס נתונים

ניסיון שלישי:

(Data Transfer Object (DTO
אובייקט בשפת התכנות (python במקרה זה) המייצג תוכן של רשומה מטבלה בבסיס הנתונים. החלק הלוגי עובד רק עם אובייקטים מסוג זה. כאשר הוא נדרש לקבל מידע מבסיס הנתונים, המידע יחזור אליו מממשק בסיס הנתונים כאובייקט מסוג זה (ולא כרשומה, כפי שהוא מיוצג בבסיס הנתונים)

(Data Access Object (DAO
ממשק התומך בפעולת אחסון (insert) של אובייקט המייצג רשומה (DTO) או בשליפת רשומה (find) כאובייקט שכזה (DTO)

פעולת האחסון מקבל DTO לשמירה בטבלה ובונה שאילתת INSERT המוסיפה רשומה עם ערכי השדות שלו לטבלה.
פעולת השליפה מחלצת באמצעות שאילתת SELECT רשומה מהטבלה, ומחזירה אותה כאובייקט DTO (ערכי השדות ברשומה הם ערכי השדות של אובייקט ה DTO)

במקרה שלנו יש שלוש מחלקות DTO: Student, Grade, Assignment , ושלוש מחלקות DAO: Students, Grades, Assignments

```python
#file: persistence.py

import sqlite3
import atexit

# Data Transfer Objects:

class Student(object):
    def __init__(self, id, name):
        self.id = id
        self.name = name


class Assignment(object):
    def __init__(self, num, expected_output):
        self.num = num
        self.expected_output = expected_output


class Grade(object):
    def __init__(self, student_id, assignment_num, grade):
        self.student_id = student_id
        self.assignment_num = assignment_num
        self.grade = grade


# Data Access Objects:

# All of these are meant to be singletons
```

```
class _Students:
    def __init__(self, conn):
        self._conn = conn

    def insert(self, student):
        self._conn.execute("INSERT INTO students (id, name) VALUES (?,
?)",
                   [student.id, student.name])

    def find(self, student_id):
        c = self._conn.cursor()
        c.execute("SELECT id, name FROM students WHERE id = ?",
[student_id])
        return Student(*c.fetchone())


class _Assignments:
    def __init__(self, conn):
        self._conn = conn

    def insert(self, assignment):
        self._conn.execute("INSERT INTO assignments (num, expected_output)
VALUES (?, ?)”,
                           [assignment.num, assignment.expected_output])

    def find(self, num):
        c = self._conn.cursor()
        c.execute("SELECT num,expected_output FROM assignments WHERE num =
?", [num])
        return Assignment(*c.fetchone())


class _Grades:
    def __init__(self, conn):
        self._conn = conn

    def insert(self, grade):
        self._conn.execute("""
            INSERT INTO grades (student_id, assignment_num, grade) VALUES
(?, ?, ?)
        """, [grade.student_id, grade.assignment_num, grade.grade])

    def find_all(self):
        c = self._conn.cursor()
        all = c.execute("""
            SELECT student_id, assignment_num, grade FROM grades
        """).fetchall()

        return [Grade(*row) for row in all]
```

```
#The Repository
class _Repository(object):
    def __init__(self):
        self._conn = sqlite3.connect('grades.db')
        self.students = _Students(self._conn)
        self.assignments = _Assignments(self._conn)
        self.grades = _Grades(self._conn)

    def _close(self):
        self._conn.commit()
        self._conn.close()

    def create_tables(self):
        #see code in previous version...

# the repository singleton
repo = _Repository()
atexit.register(repo._close)

#file: grading.py

from persistence import repo

import os
import imp

def grade(assignments_dir, assignment_num):
    expected_output =
repo.assignments.find(assignment_num).expected_output

    for assignment in os.listdir(assignments_dir):
        (student_id, ext) = os.path.splitext(assignment)

        code = imp.load_source('test', assignments_dir + '/' + assignment)

        student_grade = Grade(student_id, assignment_num, 0)
        if code.run_assignment() == expected_output:
            student_grade.grade = 100

        repo.grades.insert(student_grade)


def print_grades():
    print 'grades:'
    for grade in repo.grades.find_all():
        student = repo.students.find(grade.student_id)
        print 'grade of student {} on assignment {} is {}'\
            .format(student.name, grade.assignment_num, grade.grade)
```

חיסרון: הקוד של ה DAO השונים - Students, Grades, Assignments – זהה כמעט לחלוטין, עד כדי שמות טבלאות ושדות.
נגדיר תבנית כללית של DAO

ניסיון רביעי:
המחלקה המממשת את התבנית הכללית של ה DAO תקבל בבנאי את מחלקת ה DTO שאותה היא צריך להעביר מ/ל בסיס הנתונים, ועל פיה היא תנסח את השאילתות המתאימות.
הנחות:

- שמות הטבלאות בבסיס הנתונים הם כשמות מחלקות האובייקטים המועברים (ה DTO) בתוספת 's' (Student -> Students, Grade -> Grades, Assignment -> Assignments')
- שמות השדות בבסיס הנתונים הם כשמות השדות של מחלקות האובייקטים המועברים (ה DTO)
- הבנאי של אובייקט ה DTO מקבל את כל השדות (לאו דווקא על פי סדרם בטבלה בבסיס הנתונים – ידרוש המרת אינדקסים מייגעת בפרוצדורה orm)

במימוש התבנית הכללית, על בסיס הנחות אלו נדרש לקבל אינפורמציה על פרטי מחלקת ה DTO (ניתן על ידי החבילה inspect ב python), ועל השדות ברשומה החוזרת מבסיס הנתונים (כלול באובייקט ה cursor של sqlite)

```
#file dbtools.py

import inspect

#we can use our method above in order to start writing a generic Dao
#note that this class is not complete and we will add methods to it next
class Dao(object):
    def __init__(self, dto_type, conn):
        self._conn = conn
        self._dto_type = dto_type

        #dto_type is a class, its __name__ field contains a string
representing the name of the class.
        self._table_name = dto_type.__name__.lower() + 's'

   def insert(self, dto_instance):
        ins_dict = vars(dto_instance)      # { 'id': 3, 'name' : 'danny' }

        column_names = ','.join(ins_dict.keys())     # ['id' , 'name'] =>
'id,name'
        params = ins_dict.values()                   # [3, 'danny']
        qmarks = ','.join(['?'] * len(ins_dict))     # '?,?'

        stmt = 'INSERT INTO {} ({}) VALUES ({})'\
               .format(self._table_name, column_names, qmarks)

        self._conn.execute(stmt, params)

    def find_all(self):
```

```
        c = self._conn.cursor()
        c.execute('SELECT * FROM {}'.format(self._table_name))
        return orm(c, self._dto_type)

  def find(self, **keyvals):
        # keyvals  = { ‘id’ : 17, ‘name’ : ‘danny’}
        column_names = keyvals.keys() # [‘id’, ‘name’]
        params = keyvals.values() # [17, ‘danny’]

        stmt = 'SELECT * FROM {} WHERE {}' \
               .format(self._table_name, ' AND '.join([col + '=?' for col
in column_names]))
        # SELECT * FROM students WHERE id=? AND name=?

        c = self._conn.cursor()
        c.execute(stmt, params)
        return orm(c, self._dto_type)

def orm(cursor, dto_type):

    #the following line retrieve the argument names of the constructor
    args = inspect.getargspec(dto_type.__init__).args    #  [‘self’, ‘id’,
‘name’]

    #the first argument of the constructor will be 'self', it does not
correspond
    #to any database field, so we can ignore it.
    args = args[1:]  #  [‘id’, ‘name’]

    #gets the names of the columns returned in the cursor
    # cursor.description = [ [‘name’, …], [‘id’, … ]]
    #col_names = [‘name’, ‘id’]
    col_names = [column[0] for column in cursor.description]

    #map them into the position of the corresponding constructor argument
    col_mapping = [col_names.index(arg) for arg in args]   # [1, 0]
    # cursor.fetchall() = [ [‘danny’,17], [‘dinna’,45]]
    return [row_map(row, col_mapping, dto_type) for row in
cursor.fetchall()]


def row_map(row, col_mapping, dto_type):
    # row = [‘danny’,17]
    # col_mapping [1, 0]
    #dto_type = Student
    ctor_args = [row[idx] for idx in col_mapping]   # [17, ‘danny’]
    return dto_type(*ctor_args)  # Student(17, ‘danny’)

#Repository
class Repository(object):
```

```
    def __init__(self):
        self._conn = sqlite3.connect('grades.db')
        self._conn.text_factory = bytes
        self.students = Dao(Student, self._conn)
        self.assignments = Dao(Assignment, self._conn)
        self.grades = Dao(Grade, self._conn)

    def _close(self):
        #see code in previous version...

    def create_tables(self):
        #see code in previous version...

# singleton
repo = Repository()
atexit.register(repo._close)
```

חיסרון: כל פעם שמגדירים שדה חדש בטבלה, או טבלה חדשה בבסיס הנתונים, נדרש לעדכן את מחלקת ה DTO המקבילה ב python

 נייצר אוטומטית את מחלקות ה DTO בהינתן בסיס נתונים:

- לכל טבלה יוגדר DTO ששמו כשם הטבלה (פחות ה 's')
- שמות השדות של מחלקת ה DTO יהיו כשדות המחלקה

לשם כך נדרש לקבל מבסיס הנתונים מידע על מודל הנתונים שלו: טבלאות ושדות. ניתן לעשות זאת ב SQL:

```
SELECT    TABLE_NAME, COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS
```

נשאיר את המימוש כתרגיל לבית...

חזרה וסיכום הקורס

1. הנושאים המרכזיים בקורס
   - זיכרון
     - חומרה, מערכת הפעלה
     - מודלים: Stack, Heap
     - משתנים
       - ייצוג (תוכן, כתובת), reference
       - הגדרה, העברה והחזרה
       - const
     - אובייקטים: מבנה, אתחול, מחיקה, העתקה, השמה (+אופטימיזציית &&)
     - ירושה ופולימורפיזם
       - מבנה, אתחול, מחיקה, העתקה, השמה
       - מימוש הפולימורפיזם, dynamic and static binding
   - מקביליות
     - הגדרת ת'רד
     - אינווריאנטה ותנאי התחלה וסיום, הגדרת בטיחות ונכונות

- שיטות לשמירה על בטיחות ונכונות
  - אובייקטים שאינם ברי שינוי, snapshot (דורש מיזוג עותקים בסוף)
  - סנכרון: lock-based (synchronized, Semaphore, ReaderWriter, CAS, lock), lock-free (cas)
  - Optimistic Try & Fail
- מנגנוני המתנה: wait & notify
- Liveness: מדדים, תרחישים ופתרונות

- תקשורת
  - שירותי שכבת הטרנספורט, פרוטוקולי TCP, UDP
  - עיצוב תבנית כללית של שרת (בשכבת האפליקציה)
    - Encoding-Framing-Decoding
    - Protocol
    - ConnectionHandler
    - Concurrency: Single Thread, Thread Per Client, Thread Pool, Reactor
    - שרת מונחה פקודות
- בסיסי נתונים
  - עיצוב מודל נתונים
  - SQL
  - אפליקציה הכוללת לוגיקה ושמירה בבסיס נתונים

2. רשימת תבניות עיצוב

- Memory
  - Rule of 3 and Rule of 5
  - ~~Factory~~, Singleton
  - Smart pointer
  - Double dispatch
- Concurrency
  - Immutable object
  - Synchronized class (fully/partially)
  - CAS
  - Versioned Iterator
  - Guarded suspension (wait-notify pattern)
  - Semaphore
  - Reader-Writer
- Communication
  - BaseServer
  - Reactor
  - ActorThreadPool
  - Remote-Command-Invocation server
- Database
  - Application persistence pattern

3. המבחן

- מבנה
  - 4 שאלות

        - זיכרון 30%
        - מקביליות 30%
        - תקשורת 30%
        - בסיסי נתונים 10%
- המלצת למידה
    - מעבר על החומר
    - מבחינת קוד - דגש על design patterns
    - פתרון מבחנים, קודם את השנים האחרונות. יש מבחנים ופתרונות באתר.
    - מייל, שעות קבלה